Data-Centric Recursive Improvement for Foundation Models: A Survey

Yanyan Zou[α*†✉], **Chao Chen**[β†], **Tung Sum Thomas Kwok**[γ†], **Jiayu Yang**[β†], **Yunyun Hou**[ϵ†], **Jingmin Zhu**[ζ†], **Zehang Luo**[β†], **Shengjie Li**[η†], **Yinhong Liu**[δ†], **Yingjia Wan**[γ†], **Chengzu Li**[δ†], **Tao Yu**[θ], **Zhijiang Guo**[β✉], **Jiachen Liu**[α✉], **Sirui Wang**[α], **Nan Duan**[α]

[α] *JD Future Academy,* [β] *Hong Kong University of Science and Technology (Guangzhou),* [γ] *University of California, Los Angeles,* [δ] *University of Cambridge,* [ϵ] *Communication University of China,* [ζ] *Monash University,* [η] *Peking University,* [θ] *Institute of Automation, Chinese Academy of Sciences*

[*] *Project lead* [†] *Core contributor* [✉] *Corresponding author*

Abstract

Data-centric methods are central to foundation model development, but data construction and evaluation have often been treated as separate stages. Recent model-development pipelines increasingly couple the two: evaluation results expose capability gaps and failure modes, which then guide data selection, filtering, synthesis, and post-training interventions that shape the behavior of subsequent model versions. This survey studies this trend as data-centric recursive improvement, with data-evaluation co-evolution as its core mechanism. In this mechanism, evaluation produces diagnostic signals, orchestration determines whether and how these signals should be trusted and used, and execution updates mutable data-related objects. These updated objects are then used in later training, adaptation, retrieval, or memory stages, enabling the system to change in subsequent rounds. The framework centers on three questions: *what signal diagnoses the current system, who decides how to act on that signal, and what object is updated to affect the next round.* We organize the literature around this signal-decision-update loop, covering feedback signals, control decisions, and data updates across major stages of foundation model development. We further examine failure modes in closed-loop improvement, such as unreliable feedback, feedback overfitting, unstable updates, erosion of data support, and irreproducible system changes. By providing a structured framework for understanding and designing data-evaluation co-evolution, this survey outlines a roadmap toward more adaptive, robust, and accountable foundation model development, where evaluation becomes an active driver of recursive model improvement rather than a passive measure of progress and ultimately sheds light on possible pathways toward Artificial Super Intelligence (ASI), in which a model continually evolve and surpass human-level performance across diverse tasks.

1 Introduction

Foundation-model development is increasingly organized around repeated cycles of diagnosis and repair rather than a single pass of dataset construction followed by post-hoc evaluation. In current practice, an evaluation result is often not the end of an experiment but the beginning of the next data intervention: benchmark failures (Gadre et al., 2023) or learned judges (Mi et al., 2025; Deng et al., 2026) motivate targeted data acquisition. These examples point to a broader shift: evaluation is becoming an operational part of the improvement process, and data is no longer a static input but a mutable state shaped by what the current model fails to do.

This shift is only partially captured by existing views of data-centric AI, evaluation, and model self-improvement. Data-centric work has provided strong tools for documenting datasets (Gebru et al., 2021), tracking provenance (Longpre et al., 2024a), filtering web corpora (Penedo et al., 2024), and designing data

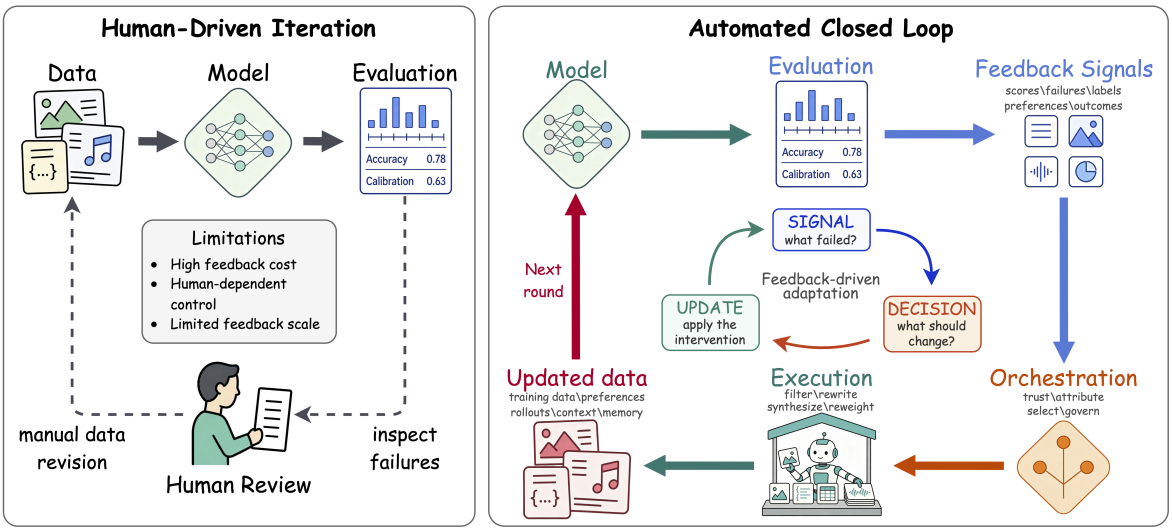

Figure 1: From human-driven iteration to the automated closed loop we formalize as *data–evaluation co-evolution*. Foundation-model development has traditionally followed a linear pipeline (left) in which humans manually inspect evaluation outputs and revise data for the next round. This survey studies the automated regime (right), in which evaluation signals grounded in text, image, audio, or preference feedback close the loop through orchestration and execution over mutable data-related state.

mixtures (Xie et al., 2023; Ye et al., 2025a), but it often studies data quality as a property of a dataset or pipeline before the next model is trained. Evaluation surveys summarize benchmark design, contamination-aware evaluation, and the shift from static to dynamic evaluation (Li et al., 2024d; Chen et al., 2025b), but this line of work often treats evaluation as a measurement layer rather than as a controller of subsequent data changes. Self-improvement and agent-evolution surveys cover broad mechanisms by which models, agents, tools, or memories adapt (Gao et al., 2026), yet they typically do not isolate the data pathway through which feedback changes later training. As a result, methods such as dynamic data construction (Deng et al., 2026), target-aware instruction selection (Xia et al., 2024), and failure-driven tuning (Yao et al., 2025) are discussed as different topics even when they share the same underlying loop. The central lens of this survey is therefore the operational path from evaluation signal to data-centric update. Across these systems, three operational questions recur: what signal diagnoses the current model state, who or what determines whether that signal is trustworthy and translates it into an intervention, and what object is updated to shape future model behavior?

Our scope is intentionally data-centric: we focus on improvement driven by data-related objects. We introduce *data-centric recursive improvement* to describe improvement processes in which a foundation-model system becomes better across rounds by updating such objects. To the best of our knowledge, this is the first systematic survey centered on data-centric recursive improvement, offering a foundational roadmap for understanding how evaluation can drive model improvement through data flows.

Since this line of research is still taking shape, its scope and terminology have not yet converged on a single agreed-upon formulation. We therefore treat this survey not as a catalogue of a settled paradigm, but as an effort to clarify an emerging set of related mechanisms. Rather than drawing overly strict boundaries, we use a unified framework to connect the diverse approaches that are beginning to appear across the literature. Specifically, we define the core mechanism as *data-evaluation co-evolution* and structure our analysis around the three foundational questions introduced above through three connected elements: evaluation, orchestration, and execution. An evaluation signal diagnoses the current system; an orchestration policy decides whether the signal is trustworthy, what failure it indicates, and which intervention it justifies; and an execution mechanism applies the selected intervention to a mutable data-related state that affects a later round. This separation matches the functional roles exposed by recent systems. DataOrchestra (Huang

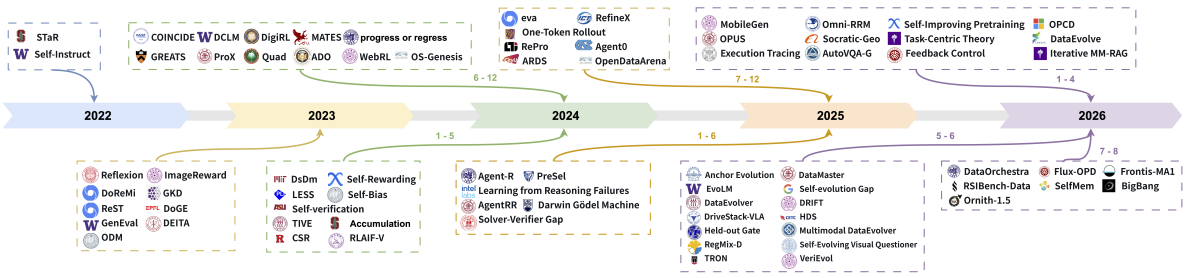

Figure 2: An evolutionary landscape of several representative data-evolution co-evolution frameworks from 2022 to 2026.

et al., 2026) separates tool-execution feedback from chunk-level curation decisions. BigBang (PolarSeeker, 2026) uses critic judgments and post-training outcomes to revise later synthesis and evaluation policies. Frontis-MA1 (Yang et al., 2026a) updates programs and experience records rather than model weights during inference. A method falls within our core scope when this signal-to-decision-to-update path is explicit enough to show how evaluation changes a later data object, context, memory, or control policy. Under this view, an aggregate score, failure cluster, verifier label, preference comparison, judge rationale, or environment outcome becomes more than a report only when it is consumed by a decision process that changes future training or adaptation.

2 Definition and Taxonomy

2.1 Definition

We define *data–evaluation co-evolution* as a feedback loop in which evaluation signals guide updates to data-related objects, and those updated objects affect later model or system behavior that is evaluated again in later rounds.

This concept is our mechanism-level view of *data-centric recursive improvement*. Recursive improvement is the broader phenomenon: a system state produced in one round contributes to later improvement. Data-evaluation co-evolution specifies how this happens through data-centric updates. Evaluation diagnoses the current system, orchestration decides how the signal should be used, and execution changes the data-related object that affects future behavior. These roles answer three questions that define the loop: what has been observed, how the observation should be used, and what object is actually changed.

2.2 Data-Evaluation Co-Evolution as the Improvement Mechanism

We formalize data-evaluation co-evolution as a closed loop with three functional roles: evaluation, orchestration, and execution. Evaluation observes the current model or system state and produces an actionable signal. Orchestration interprets this signal and selects an intervention. Execution applies the intervention to a mutable data-related object. The updated state is then evaluated again, making the next round of improvement depend on the outcome of the previous round. An instance loop is depicted in Figure 3.

Formally, let $\mathcal{E}$ denote the evaluation procedure that produces diagnostic evidence from the current system. It may be implemented by a benchmark or a verifier model; see Section 3 for more details. We denote by M_t the model or model-based system at iteration t. $\mathcal{Q}_t$ denotes the evaluation distribution at iteration t, such as benchmark examples, tool checks, or environment states used to evaluate the current model or model-based system. It may be fixed or refreshed across rounds.

First, an evaluation function maps the current system and an evaluation distribution $\mathcal{Q}_t$ to a feedback signal:

$$s_t = \mathcal{E}(M_t, \mathcal{Q}_t). \tag{1}$$

The signal s_t may summarize aggregate performance or isolate a specific failure. Depending on the setting, it may take the form of human or model feedback, verifier or tool outputs, or external environment signals,

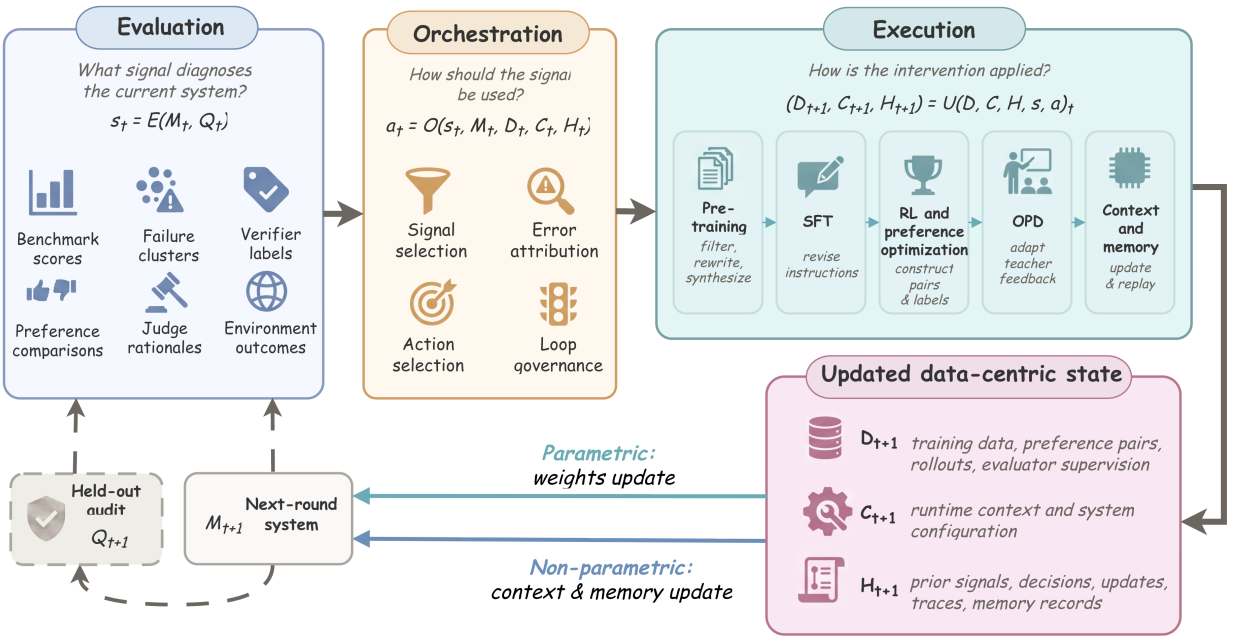

Figure 3: The illustrated signal-to-decision-to-update path shows how evaluation changes a later data-related state. Data–evaluation co-evolution is a state transition rather than a measurement step. Evaluation observes the current system and returns a signal, orchestration decides whether that signal is trusted and which change it justifies, and execution applies the selected action to a mutable data-related object that the next round inherits. The updated object reaches the next system through a parameter-updating path or a non-parametric path. A held-out audit distribution stays outside the update operator, so a gain measured by the controller's own evidence does not by itself establish improvement. Equations (1) to (3) formalize the three roles.

as discussed in Section 3.1. A signal becomes relevant to recursive improvement only when it can guide a later update, rather than merely report performance.

Second, the orchestration function $\mathcal{O}$ interprets the actionable signal into a specific intervention:

$$a_t = \mathcal{O}(s_t, M_t, \mathcal{D}_t, C_t, \mathcal{H}_t), \tag{2}$$

where $\mathcal{O}$ denotes an orchestration policy, such as a judge, a planner, or a scheduler, as discussed in Section 4. $\mathcal{D}_t$ denotes the mutable data-related object available to the system at iteration t, for example, training data, preference pairs, rollout traces, and evaluator-supervision data. C_t denotes runtime context or system configuration, and $\mathcal{H}_t$ denotes the loop history, including previous signals, decisions, updates, traces, and memory records when available. This step decides whether the signal should be trusted, what failure it indicates, and which intervention should follow.

Third, the execution function applies the selected intervention:

$$(\mathcal{D}_{t+1}, C_{t+1}, \mathcal{H}_{t+1}) = \mathcal{U}(\mathcal{D}_t, C_t, \mathcal{H}_t, s_t, a_t), \tag{3}$$

where $\mathcal{U}$ denotes an update operator, such as data filtering, reweighting, rewriting and synthesizing, as formalized in Section 5. The updated memory $\mathcal{H}_{t+1}$ records the interaction trajectory and supports tracing previous loop states. After the intervention is applied, the updated data, context, and memory serve as inputs to later training or adaptation, resulting M_{t+1} that denotes a parameter-updated model or a model-based system consuming $\mathcal{D}_{t+1}, C_{t+1}$, or $\mathcal{H}_{t+1}$.

This formulation separates three questions that are often mixed together in the literature. Evaluation asks what has been observed. Orchestration asks how the observation should be used. Execution asks what

object is actually changed. A method belongs to the core scope of this survey when this signal-to-decision-to-update path is explicit enough to show how evaluation changes a later data-related state. Measurement-only studies are treated as background evidence unless their outputs are used to change a downstream data-related object or control decision. Figure 3 also marks the distribution that the loop must not consume: an audit distribution held outside the update operator, which Section 4 develops as the separation between a controller-facing selection distribution and an untouched audit distribution.

Evaluation, orchestration, and execution are functional roles rather than mutually exclusive architectural modules. The same component may play multiple roles in a concrete system. For example, a learned judge performs evaluation when it scores candidate outputs, but it performs orchestration when its verdict determines whether candidates are retained, regenerated, or escalated. A teacher performs execution when it produces supervision, but it performs orchestration when student failures determine which examples or modalities receive teacher feedback. A memory system performs execution when it writes or deletes records, while its controller performs orchestration when task outcomes revise retrieval, replay, or promotion policies.

2.3 Literature Coverage and Survey Organization

We included works that either use evaluation feedback to update a data-related object, introduce reusable data/evaluation/control operators for recursive improvement, or analyze closed-loop risks such as contamination, reward hacking, evaluator dependence, model collapse, unstable updates, or irreproducibility. The resulting bibliography contains 300 unique cited works from 2020–2026, including 132 preprints or technical reports and 168 venue-resolved or archival publications. The most represented years are 2024–2026, and the largest venue groups are arXiv, ICLR, NeurIPS, ICML, CVPR, ACL, EMNLP, ECCV, AAAI, and TMLR.

The remainder of the survey follows the first three stages in Figure 4 as a signal–decision–update sequence and uses the fourth stage as a cross-cutting reliability audit. Section 3 studies evaluation signals and asks when measurement becomes actionable feedback. Section 4 studies orchestration and asks how signals are trusted, attributed, routed, scheduled, escalated, or stopped. Section 5 studies execution and asks how selected interventions update mutable data-related objects across pre-training, SFT, RL and preference optimization, OPD, and context or memory adaptation. Section 6 then analyzes why a higher score may still fail to establish reliable recursive improvement, focusing on signal validity, signal independence, update stability, data-support integrity, and system identifiability. Section 7 presents future directions worth exploring.

3 Evaluation Signals: From Measurement to Actionable Feedback

This section studies when an evaluation becomes an actionable signal for recursive improvement. An evaluation output contributes to the data–evaluation loop only when it does more than report performance: it must diagnose a model or system state and guide a downstream update to data, context, memory, training policy, or orchestration policy.

3.1 Evaluation Forms as Feedback Signals

We classify feedback by the form of the evaluation output rather than by the training stage that consumes it. The same signal can operate at multiple stages: a detector score can filter pre-training data, select SFT samples, or provide rewards for RL. Following this output-form view, we organize the literature into six recurring forms, each characterized by the mutable object it can change, the trust assumption it introduces, and the failure mode it creates when wrong. Table 1 summarizes the six forms.

3.1.1 Benchmark Scores

An aggregate benchmark score is the coarsest and cheapest evaluation signal: almost every benchmark emits one. When consumed as a capability gap or a mixture objective, it can change dataset-level mutable objects, such as domain or modality mixtures, curricula, and checkpoint-selection policies. DataComp(Gadre et al., 2023) provides a clean example. With the model, code, and compute fixed, a suite of 38 downstream tasks ranks candidate corpora, so the benchmark score selects datasets rather than models. Benchmark-derived

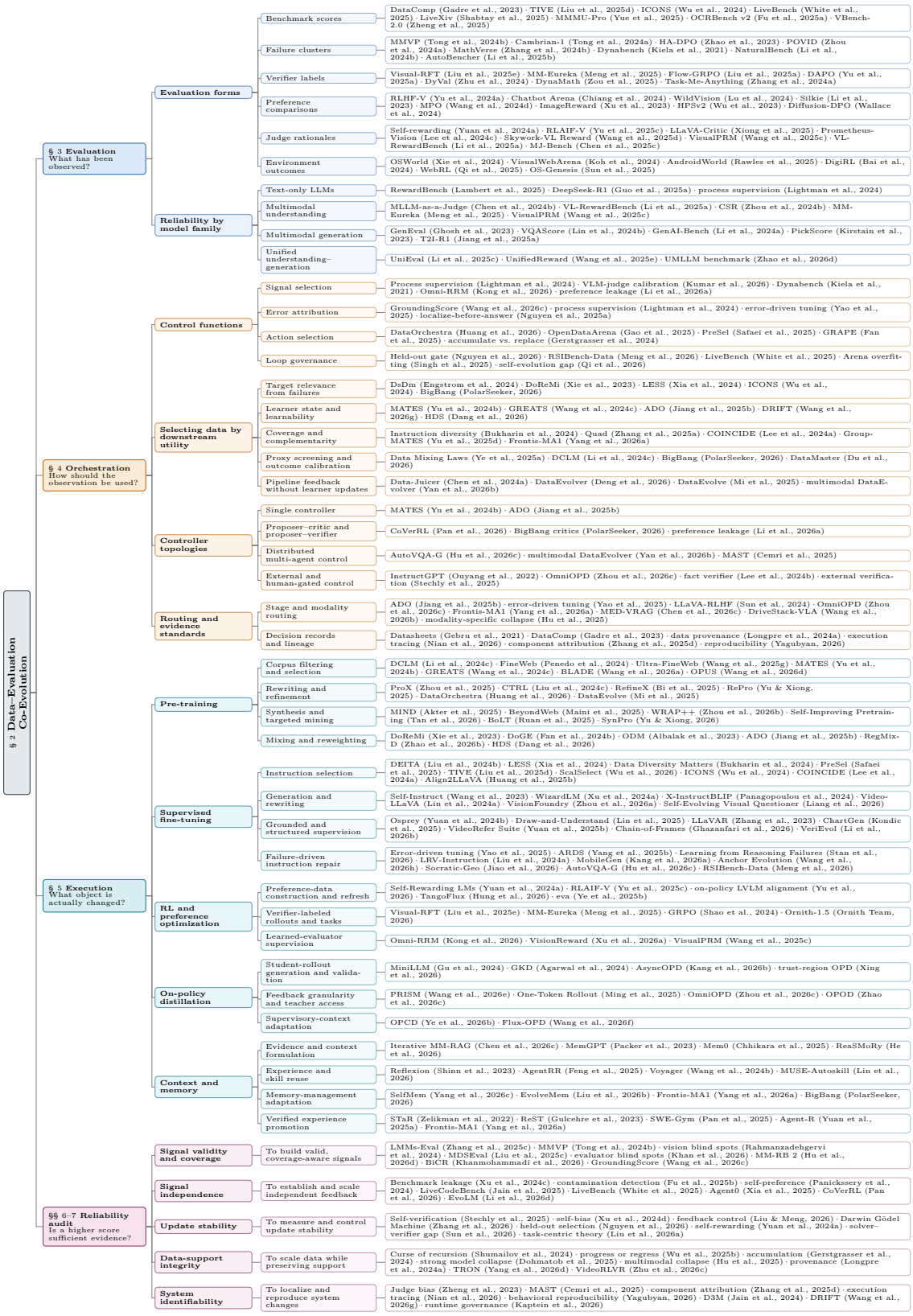

Figure 4: Taxonomy and reading map for data–evaluation co-evolution.

Table 1: Six evaluation forms as feedback signals in data–evaluation co-evolution. Each form is characterized by the loop signal it expresses, the mutable object it can change, the trust assumption it introduces, and the failure mode it creates when wrong.

Evaluation form	Loop signal	Mutable object updated	Trust assumption	Failure mode	Representative works
Benchmark score	Capability gap; mixture objective	Data mixture, curriculum, checkpoint selection	Transfer beyond the source benchmark	Benchmark overfitting	DataComp (Gadre et al., 2023), ICONS (Wu et al., 2024), LiveBench (White et al., 2025)
Failure cluster	Weakness specification	Hard examples, synthetic data, negative samples	Correct causal attribution	Fixing the wrong bottleneck	MMVP→Cambrian-1 (Tong et al., 2024b;a), POVID (Zhou et al., 2024a), NaturalBench (Li et al., 2024b)
Verifier label	Accept/reject; scalar reward	Rollouts, generated samples, reward-labeled data	Coverage and calibration	Reward hacking	Visual-RFT (Liu et al., 2025e), Flow-GRPO (Liu et al., 2025a)
Preference comparison	Pairwise winner–loser supervision	Preference pairs, reward-model data	Rater reliability and calibration	Bias propagation	RLHF-V (Yu et al., 2024a), Chatbot Arena (Chiang et al., 2024), ImageReward (Xu et al., 2023)
Judge rationale	Error explanation; revision target	Rewritten answers, critiques, process traces	Grounded validity	Fluent but false supervision	RLAIF-V (Yu et al., 2025c), LLaVA-Critic (Xiong et al., 2025), VisualPRM (Wang et al., 2025c)
Environment outcome	Task success; tool feedback	Agent traces, memory, curriculum tasks	Environment transfer	Environment overfitting	OSWorld (Xie et al., 2024), DigiRL (Bai et al., 2024), OS-Genesis (Sun et al., 2025)

scores can also guide sample-level selection. TIVE (Liu et al., 2025d) and ICONS (Wu et al., 2024) estimate the influence of instruction examples on validation benchmarks and retain the highest-valued subset; the same signal is consumed at the SFT stage (Section 5.2) and, at corpus scale, in pre-training mixture search (Section 5.1).

The trust requirement is transfer: gains induced by the score must extend beyond the benchmark that supplied it. When they do not, the loop overfits the benchmark signal, often without directly training on any test item. This makes the failure harder to detect than ordinary contamination: the data update may favor examples, formats, or skills that are close to the benchmark without improving broader capability.

Benchmarks have therefore started to evolve in order to preserve the usefulness of their scores as feedback signals. LiveBench (White et al., 2025) refreshes its questions monthly and scores only objectively checkable answers. LiveXiv (Shabtay et al., 2025) generates VQA examples from newly posted arXiv figures and estimates model scores from a small subset, making frequent refresh affordable. When benchmark suites become saturated, successor versions such as MMMU-Pro (Yue et al., 2025), OCRBench v2 (Fu et al., 2025a), and VBench-2.0 (Zheng et al., 2025) restore headroom.

3.1.2 Failure Clusters

A failure cluster is a group of evaluation cases on which a model repeatedly fails for a shared reason, such as hallucinated objects (Zhao et al., 2023; Zhou et al., 2024a) and shortcut use (Zhang et al., 2024b; Chen et al., 2024d). Unlike an aggregate score, a failure cluster identifies a weakness that may guide a data intervention. It suggests what evidence the next round should add, reweight, or contrast. MMVP (Tong et al., 2024b) exposed clusters of CLIP-blind image pairs. Cambrian-1 (Tong et al., 2024a) responded with vision-centric data and model-design changes, later evaluated with CV-Bench. Hallucination taxonomies play a similar role for alignment data: HA-DPO (Zhao et al., 2023) and POVID (Zhou et al., 2024a) synthesize dispreferred responses that instantiate measured hallucination patterns, turning benchmark-defined error categories into preference-optimization data.

The trust requirement is causal attribution: the cluster must identify the bottleneck that a data update can plausibly repair. A cluster that mixes perception failures with language-prior overrides may send data synthesis toward the wrong target. Ablation protocols such as MathVerse's (Zhang et al., 2024b) information variants and MMStar's (Chen et al., 2024d) vision-indispensability filter therefore matter because they test whether the failure actually depends on missing visual evidence, reasoning, or shortcut use before it is converted into training data (Zhang et al., 2024b; Chen et al., 2024d).

Failure clusters also show why benchmark construction and data construction are tightly coupled in recursive improvement. Dynabench (Kiela et al., 2021) pays annotators for model-breaking examples, NaturalBench (Li et al., 2024b) keeps natural images that models fail but humans pass—a failure-mining algorithm whose output happens to be a test set. AutoBencher (Li et al., 2025b) searches for difficult and novel question sets. The same mined failures can either remain held out as evaluation evidence or be admitted into the next training round as repair data. The boundary is therefore not the mining procedure itself, but the governance decision over whether a failure cluster is used for testing, training, or both.

3.1.3 Verifier Labels

A verifier label is a pass/fail decision or scalar score produced by a rule, program, tool, or fixed evaluator, such as exact answer match, IoU against a reference box, an object detector's verdict, or execution success. Among evaluation forms, verifier labels are often the most auditable because they can be recomputed and inspected. When consumed by a loop, they directly change sample-level mutable objects: which rollouts receive reward, which generated samples are retained, which prompts are filtered, or which web pairs enter a corpus.

At pre-training scale, thresholded CLIP scores and trained filtering networks decide corpus membership (Gadre et al., 2023; Fang et al., 2024; Xu et al., 2024b) (Section 5.1). At the RL stage (Section 5.3), verifiable rewards moved quickly from text (Guo et al., 2025a) to multimodal models: MM-Eureka rewards answer matches (Meng et al., 2025). Visual-RFT (Liu et al., 2025e) lifts IoU and classification accuracy directly into GRPO rewards. Flow-GRPO (Liu et al., 2025a) rewards a text-to-image model with the GenEval evaluator itself, raising SD3.5-M from 63% to 95% on GenEval. The same labels double as difficulty estimates: prompts whose rollouts all pass or all fail carry no gradient, so DAPO (Yu et al., 2025a) filters them online.

The trust requirement is coverage and calibration: the verifier must check the property that the loop intends to improve, and its score must remain meaningful under optimization. When this assumption fails, the loop can reward-hack the verifier. Training directly against GenEval, for example, makes GenEval less independent as a measure of generalization: a higher GenEval score may partly reflect fit to the verifier rather than broader improvement, which is why held-out suites and reward-hacking analyses are needed alongside optimized verifier scores (Jiang et al., 2025a).

This risk also explains why benchmark evolution around verifier labels often takes the form of generated or regenerated tasks. DyVal (Zhu et al., 2024) composes reasoning tasks from graph structures. DynaMath (Zou et al., 2025) renders each seed problem as a program that emits fresh visual variants and reports worst-case accuracy. Task-Me-Anything (Zhang et al., 2024a) emits large numbers of verifiable task instances on demand. Regenerated variants can restore score gaps erased by memorization (Wang et al., 2025b). Because these generators produce their own ground truth, they can support both evaluation and training; the governance question is how to keep the instances used for optimization separate from those used for audit.

3.1.4 Preference Comparisons

A preference comparison states, at minimum, that one output is preferred to another under a given prompt, context, or criterion. When consumed by a loop, this signal changes preference pairs, reward-model training data, or winner–loser examples used by preference-optimization methods (Section 5.3).

Preference sources form a cost–fidelity spectrum. At the high-fidelity end, RLHF-V (Yu et al., 2024a) collects segment-level human corrections for hallucinated multimodal answers and converts them into dense preferences. At scale, arena votes are produced as a by-product of evaluation: Chatbot Arena (Chiang

et al., 2024) and WildVision (Lu et al., 2024) rank models live while accumulating preference corpora for later training. Between these endpoints, preferences are manufactured or semi-automated: Silkie (Li et al., 2023) uses GPT-4V to annotate a feedback corpus, and MPO (Wang et al., 2024d) builds pairs from answer matching and image-ablated continuations. For multimodal generation, reward models such as ImageReward (Xu et al., 2023), PickScore (Kirstain et al., 2023), and HPSv2 (Wu et al., 2023) distill human preferences into signals that both rank generators and support reward fine-tuning or Diffusion-DPO (Wallace et al., 2024).

Preference comparisons make the evaluation–training overlap explicit: the same vote, pair, or reward label can rank systems and become mutable supervision for the next update. The trust requirement is rater reliability and calibration. The failure mode is bias propagation: position bias, verbosity bias, style preference, model-family preference, or weak visual grounding can be copied into every pair the rater labels and then amplified by preference optimization. Arena votes (Chiang et al., 2024; Lu et al., 2024) make this overlap especially visible. They are evaluation signals because they rank systems on fresh user prompts, but they also become training signals when the same votes are accumulated into preference corpora. This dynamic stream tracks the moving frontier better than a frozen preference test set, but using it for training requires attention to prompt distribution, rater population, data access, and independence from final evaluation.

3.1.5 Judge Rationales

A judge is an evaluator, usually a learned model, that assesses candidate outputs and returns a score, preference, accept/reject decision, or critique. Here we focus on cases where the judge returns a rationale rather than only a verdict: it may explain what is wrong, identify the violated criterion, localize an unsupported step, or propose a revised answer. This makes judge rationales a scalable but difficult-to-verify feedback form for open-ended outputs. When consumed by a loop, they change mutable supervision objects such as rewritten answers, critique corpora, process traces, preference annotations, data-selection scores, or process-reward labels. Examples include judge-annotated corpora for preference training (Section 5.3), critic models whose scores select data or policies (Xiong et al., 2025; Wang et al., 2025d; Lee et al., 2024c), and process reward models that convert step-level judgments into supervision (Wang et al., 2025c).

The limiting case is self-referential feedback, where a model judges its own outputs and trains on the result (Yuan et al., 2024a). This design reduces annotation cost but couples the generator and evaluator. In multimodal settings, the coupling is more dangerous because the judge may inherit the same perceptual blind spots or hallucinations it is supposed to detect. CSR (Zhou et al., 2024b) mitigates this risk by re-anchoring self-reward in image-conditioned likelihood, while RLAIF-V (Yu et al., 2025c) replaces direct self-judgment with decomposed peer verification.

The trust requirement is grounded validity: the rationale must be supported by the evidence it cites, whether that evidence is text, image regions, video frames, audio, documents, tools, or environment state. The failure mode is fluent but false supervision. An MLLM judge may confabulate image content and then score or revise an answer against that confabulation (Chen et al., 2024b). Once such rationales enter training, they can corrupt preference pairs, revised answers, filters, and process traces downstream.

Because judges and reward models are themselves trained artifacts, this signal requires second-order evaluation: the field must evaluate the evaluators before their outputs are used as feedback. RewardBench established this practice for text (Lambert et al., 2025), and multimodal extensions such as VL-RewardBench, MJ-Bench, Multimodal RewardBench, VideoRewardBench, and CMI-RewardBench audit reward and judge behavior across vision-language, generation, video, and audio settings (Li et al., 2025a; Chen et al., 2025c; Yasunaga et al., 2025; Zhang et al., 2025e; Ma et al., 2026b). These benchmarks matter for co-evolution because an uncalibrated judge can silently corrupt every preference pair, reward, filter, or rationale that depends on it.

3.1.6 Environment Outcomes

An environment outcome is the success or failure of a whole interaction, such as completing a computer-use task, placing a web order, executing a tool call, or reaching a verified task state. OSWorld (Xie et al., 2024), VisualWebArena (Koh et al., 2024), and AndroidWorld (Rawles et al., 2025) define such outcomes

with executable checks in real or realistic environments. When consumed by a recursive loop, the outcome changes trajectory-level mutable objects: agent traces, replay buffers, curriculum tasks, memory entries, tool-use records, or instruction trajectories. DigiRL (Bai et al., 2024) trains device-control agents with RL against an autonomous environment evaluator. WebRL (Qi et al., 2025) turns failed trajectories into new curriculum tasks for web agents. OS-Genesis (Sun et al., 2025) explores the environment, synthesizes instruction trajectories, and filters them with a trajectory reward model. Runtime adaptation consumes the same signal when task success determines which experiences are written to memory, retained, or promoted into later training data (Section 5.5).

The trust requirement is environment transfer: success in the training or evaluation environment must predict success on fresh task instances, unseen layouts, new websites, or real deployment settings. The failure mode is environment overfitting. An agent may memorize a site layout, exploit a simulator artifact, or learn shortcuts that satisfy the environment checker without solving the intended task. This risk changes what benchmark evolution means for environment outcomes. Because an agent can overfit fixed trajectories, layouts, or simulator artifacts, environment benchmarks often need to evolve by regenerating task instances rather than only refreshing question sets. AndroidWorld parameterizes tasks so each run can be instantiated freshly (Rawles et al., 2025), reducing direct memorization of fixed trajectories. In such loops, evaluation infrastructure can also become training infrastructure: the same environment may produce rewards, generate traces, define curricula, and audit success. Environment outcomes therefore make the governance problem explicit: systems must separate the instances used for training or adaptation from those reserved for held-out evaluation.

3.2 Signal Reliability Across Model Types

The six feedback forms above are available across model families, but their reliability changes with the model being improved. The central pattern is a reliability gradient: *the more multimodal, generative, and unified a system becomes, the harder it is for evaluation outputs to serve as trustworthy training signals.* A recursive improvement loop that works for one model family therefore cannot usually be transferred to another by reusing the same signal source; the signal must be revalidated against the evidence, output space, and failure modes of the target model.

Text-only LLMs. Text-only LLMs provide the most favorable case. Many outputs can be checked by rules or programs, culminating in RL with verifiable rewards (Guo et al., 2025a). Less verifiable behavior can be routed through reward models, audited by RewardBench (Lambert et al., 2025), or refined with process reward models (Lightman et al., 2024). The main problems are signal integrity—contamination, reward overoptimization, and judge bias—rather than signal availability.

Multimodal understanding models. Multimodal understanding models inherit the text-side stack but add an evidence-use requirement. A signal must verify not only whether the answer is correct, but also whether it depends on the relevant image, region, OCR token, chart structure, video frame, or document evidence. This narrows the verifiable slice and weakens learned judges, which may share the policy's perceptual blind spots (Li et al., 2025a; Chen et al., 2024b). Current pipelines therefore combine partial verifiers, claim decomposition, visual-likelihood calibration, and multimodal process reward models (Liu et al., 2025e; Meng et al., 2025; Yu et al., 2025c; Zhou et al., 2024b; Wang et al., 2025c).

Multimodal generation models. Multimodal generation removes the assumption of a single correct output. Signals therefore become either *reductions*, which project an output onto checkable properties such as object presence or VQA-based alignment (Ghosh et al., 2023; Lin et al., 2024b; Li et al., 2024a), or *preferences*, which provide broader but more overoptimizable judgments (Xu et al., 2023; Kirstain et al., 2023; Wu et al., 2023). Reductions are auditable but gameable; preferences are holistic but easier to exploit. Generation RL pipelines therefore need multiple rewards and reward-hacking analysis (Jiang et al., 2025a; Liu et al., 2025a).

Unified understanding–generation models. Unified models add a commensurability problem. An MMMU accuracy, a GenEval pass rate, and a preference margin do not share a natural scale, yet loops need

comparable signals for checkpoint selection, data-mixture balancing, reward aggregation, or stage scheduling. Current responses probe understanding–generation consistency directly, use one branch to evaluate the other, or train reward models across both task families (Zhao et al., 2026d; Li et al., 2025c; Wang et al., 2025e). The remaining risk is that self-judgment, metric incompatibility, or cross-task interference can make an apparent gain hard to attribute. Section 6 analyzes the resulting loop-level problems through signal validity, signal independence, update stability, data-support integrity, and system identifiability.

4 Orchestration Decisions: From Feedback Signals to Data Interventions

Recent surveys distinguish the feedback observed by a self-improving system from the component or state that is subsequently adapted (Gao et al., 2026). Multimodal self-improvement further spans training data, preference feedback, and inference-time adaptation rather than one uniform update path (Deng et al., 2025). Section 3 defines the behavioral evidence available to the loop, but a failure signal does not determine its own intervention: the same failure cluster can motivate collecting new pre-training documents, constructing a supervised fine-tuning (SFT) repair set, refreshing preference pairs, requesting teacher feedback, or changing runtime memory. Orchestration is the policy that resolves this underdetermination.

Existing self-evolution taxonomies commonly organize systems by the role or component being changed (Gao et al., 2026). We instead treat orchestration as a *data-decision problem*, rather than as an inventory of teachers, judges, critics, or agents. At minimum, a stage-local rule performs orchestration when a feedback verdict changes which data are retained, rejected, or routed; adaptive orchestration is the stronger case in which accumulated feedback changes the target, operator, allocation, gate, or future decision policy. This section focuses on that adaptive case. We specialize the general orchestration action in Eq. 2 to an explicitly data-centric decision:

$$a_t = (z_t, o_t, \pi_t, b_t, g_t), \tag{4}$$

where z_t identifies the mutable data object and execution stage, o_t chooses a data operator and source, π_t allocates mass across examples, capabilities, domains, or difficulty levels, b_t assigns collection, training, and evaluation budget, and g_t specifies acceptance, escalation, stopping, or rollback. DataMaster exposes source, composition, and transformation as alternative branches of a mutable data state (Du et al., 2026). DataOrchestra exposes a complementary per-example choice among dropping, retaining, and cleaning a chunk (Huang et al., 2026). The tuple is our survey abstraction of these decisions: given the current learner and its downstream failures, which feasible change to the data state has the highest credible marginal value?

We first develop signal selection, error attribution, action selection, and loop governance as decisions over data. We then examine downstream-utility estimation, compare representative systems by the point at which feedback closes their loop, and discuss controller topologies. The final parts route decisions across training stages and modalities and specify the evidence needed to audit an orchestration claim.

4.1 A Data-Centric Decision Space

BigBang separates critic evidence, frontier-task demand, synthesis strategies, and promotion by downstream outcomes even though they participate in one loop (PolarSeeker, 2026). DataMaster similarly separates its DataTree controller from the Data Pool and Global Memory that record candidate and realized data states (Du et al., 2026). We abstract these designs into four coupled functions: signal selection determines which observations may enter the decision; error attribution turns those observations into a data demand; action selection chooses a target, operator, source, and allocation; and loop governance decides whether the proposed transition should be committed. They may be implemented by one controller or distributed across several components. Each function should leave an observable trace in the affected data, control state, loop history, or decision log, including when the result is abstention or no update.

4.1.1 Signal Selection: Building a Decision-Grade Evidence Set

Process supervision demonstrates that feedback may localize an error at a reasoning step rather than only at the final answer (Lightman et al., 2024). Vision–language model (VLM) judge studies show that the same evaluator can support ranking while remaining unreliable for absolute thresholding (Kumar et al., 2026).

These examples define a heterogeneous decision-evidence space: aggregate scores can set a capability-level acquisition budget, verifier labels can gate individual rollouts, judge rationales can suggest a rewrite, and environment outcomes can decide whether a trajectory is retained. Because these signals differ in granularity, coverage, cost, and bias, a controller should preserve their provenance and uncertainty rather than collapse all evidence into one scalar reward.

Dynamic benchmarking prefigured this shift from measurement to intervention. Dynabench placed humans and the current model in a repeated data-collection cycle, so observed failures changed which evaluation examples were acquired next (Kiela et al., 2021). Once an evaluator is allowed to control a data gate, however, its validity becomes action-dependent: scaling-law analyses of reward-model overoptimization show that increasing a learned proxy can eventually reduce the gold objective (Gao et al., 2023). The training data used to construct the gate are themselves mutable. Omni-RRM, for example, constructs rubric-grounded preference pairs and dimension-wise justifications for image, video, and audio reward-model training, with text evaluated as a transfer setting (Kong et al., 2026). This automates evaluator-data construction, but it does not by itself establish that the resulting gate remains calibrated after repeated use.

Signal form and decision role must be considered jointly. VLM-judge uncertainty varies by task and use, so an evaluator that supports model comparison may remain unsafe as a per-example data gate (Kumar et al., 2026). Signal selection is therefore curation of decision evidence: a narrow verifier should abstain outside its support, and disagreement should remain as metadata so that unsupported examples can be routed to another verifier or human review.

The main risk is that a cheap, correlated, or contaminated signal becomes the de facto objective of the data loop. Preference leakage demonstrates a concrete dependence channel: judges favor outputs from generators that are the same model, an ancestor, or a member of the same model family (Li et al., 2026a). Reusing such a signal can progressively concentrate the dataset around its blind spots. Signal provenance, abstention, and an evaluation channel not exposed to the controller must therefore be enforced within the orchestration policy rather than treated as evaluation hygiene outside the loop.

4.1.2 Error Attribution: From Failure to Data Demand

Error attribution maps decision-grade evidence to a demand profile over capabilities, domains, modalities, difficulty levels, and training stages. A useful diagnosis is not merely "the score is low," but, for example, "the current model fails medium-difficulty document questions because relevant regions are not grounded before reasoning." Such a diagnosis can increase demand for region–text alignment examples without indiscriminately adding more reasoning traces. Process supervision illustrates how localization at the reasoning-step level can make a different supervision object actionable (Lightman et al., 2024).

Attribution must distinguish the location where a failure appears from the data mechanism that caused it. An optical character recognition (OCR) error may reflect missing pre-training coverage, poor SFT instruction grounding, a retrieval failure, or an evaluator that misread the image. GroundingScore addresses the last ambiguity by explicitly checking the visual premises used by a process reward model before treating its step score as evidence of reasoning quality (Wang et al., 2026c). Localizing Before Answering makes the relevant image region an explicit evaluation object, supplying finer evidence than an answer-level hallucination label (Nguyen et al., 2025a). A self-evolving visual concept library goes one step further by using vision–language critics to revise the concept substrate that later predictions consult (Sehgal et al., 2025).

Controllers can reduce causal ambiguity by comparing failure slices, intermediate traces, modality ablations, and prior interventions recorded in the loop history. Error-driven multimodal tuning operationalizes this mapping: a teacher identifies erroneous reasoning steps, summarizes missing skills, and uses those skills to retrieve a targeted repair subset (Yao et al., 2025). The attribution output should therefore be a ranked set of repair hypotheses, each paired with the data object it predicts would change the outcome.

The same mapping can drive synthesis rather than retrieval. Learning from Reasoning Failures uses a stronger model to analyze a weaker multimodal learner's errors, propose examples targeted to the inferred failure modes, and filter the resulting repair set (Stan et al., 2026). It demonstrates an attribution-to-data

path, but not an outcome-controlled loop: the reported post-training evaluation does not return to revise a later round of failure analysis.

Error-driven multimodal tuning shows that the diagnosed skill determines which repair examples are retrieved, so a mistaken diagnosis can redirect an otherwise competent data generator toward the wrong bottleneck (Yao et al., 2025). GroundingScore likewise shows that an apparent reasoning error can originate in an unsupported visual premise (Wang et al., 2026c). Because several interventions can yield the same aggregate score change, causal attribution should remain provisional until a controlled data change validates it (Gadre et al., 2023). This training-data attribution question—which data intervention repairs behavior—is distinct from system identifiability, which asks which running component caused an observed change (Zhang et al., 2025d). An auditable controller should record the two as separate hypotheses rather than silently converting a component diagnosis into a data prescription.

4.1.3 Action Selection: Choosing the Data Object, Operator, and Allocation

DataOrchestra demonstrates that one action space can include dropping, retaining, and conditionally cleaning individual chunks (Huang et al., 2026). OpenDataArena demonstrates decisions over difficulty composition and multi-domain mixtures at dataset scale (Gao et al., 2025). We use these concrete designs to instantiate z_t, o_t, π_t, and b_t in Eq. 4. The target z_t can be a corpus subset, instruction–response pair, preference comparison, rollout trace, evaluator-training example, retrieval record, or promotion queue. The operator o_t selects, rejects, rewrites, synthesizes, relabels, deduplicates, reweights, reorders, stores, or deletes; π_t controls relative frequency and sequencing; and b_t chooses between further evidence collection, candidate generation, a training probe, and a full update.

PreSel provides a cost-aware ordering example: it selects unlabeled images and allocates task-specific budgets before instruction generation, avoiding annotation and tuning over the full image pool (Safaei et al., 2025). This is a pre-generation primitive rather than an adaptive loop, but together with DataOrchestra's select-or-repair action space it motivates comparing selection, repair, and synthesis instead of defaulting to generation. Filtering, ranking, mixture adjustment, repair, and pre-generation selection are action primitives; they become an adaptive orchestration loop only when evidence from their outcomes changes a later target, allocation, gate, or policy.

GRAPE shows that optimizing a mixture for a single target can improve that objective while degrading other targets (Fan et al., 2025). The decision should therefore record both its intended target and protected slices on which regression is unacceptable. Recursive synthetic-data experiments provide a complementary data-state warning: replacing the original real data at each round caused collapse in the studied settings, whereas accumulating generated data alongside the original pool avoided that outcome (Gerstgrasser et al., 2024). Without such constraints, a controller may oversample conspicuous hard cases or amplify mislabeled examples; modality-specific collapse further shows that slower or weaker channels need explicit protection (Hu et al., 2025). Support preservation and update stability must therefore be constraints on action selection itself.

4.1.4 Loop Governance: Acceptance, Stopping, and Rollback

Recursive Self-Evolving Agents commits a changed context artifact only when it does not regress on a disjoint held-out split (Nguyen et al., 2026). RSIBench-Data shows why the associated history matters: 18 of 23 searches that continued beyond an observed peak ended with a lower-scoring final attempt (Meng et al., 2026). We abstract these cases as g_t, which can gate an individual sample, batch, data pipeline, or checkpoint. A complete policy records the trajectory, escalation and stopping decisions, and a best-so-far state so that promotion and rollback occur at the same granularity as the update.

LiveBench motivates refreshing benchmark instances when model developers may have observed earlier test content (White et al., 2025). Analysis of Chatbot Arena further identifies overfitting incentives from repeated private testing, selective disclosure, and asymmetric access to platform data (Singh et al., 2025). Adaptive selection creates a related dependence: repeatedly using a downstream slice to choose data turns that slice into controller information even without gradient overlap. Preference leakage shows that dependence can also arise through model identity and lineage (Li et al., 2026a), while strictly self-generated reasoning can

retain a gap to oracle supervision (Qi et al., 2026). Governance should therefore separate controller-facing selection from untouched audit data; independent evidence and rollback are complementary rather than interchangeable safeguards.

4.2 Selecting Data by Downstream Utility

DsDm defines data quality through the target performance of the model produced by a specified learning algorithm rather than through an intrinsic property of records (Engstrom et al., 2024). This motivates our central co-evolution problem: choose among candidate data interventions using evidence from the current learner. Let $\mathcal{A}_t$ denote feasible actions, M_t the current model or model-based system, and M_t^a the system obtained after executing action a. We use the following idealized downstream-utility abstraction for a controller-facing selection distribution $\mathcal{Q}_t^{\mathrm{sel}}$:

$$\Delta_{\mathrm{down}}(a) = J(M_t^a; \mathcal{Q}_t^{\mathrm{sel}}) - J(M_t; \mathcal{Q}_t^{\mathrm{sel}}), \tag{5}$$

OpenDataArena reports several value and dataset-analysis dimensions rather than reducing data utility to one quality label (Gao et al., 2025). Accordingly, J is a pre-specified scalarized, possibly risk-adjusted downstream utility, while the full capability, safety, calibration, and efficiency vector is retained for constraint and Pareto analysis. A cost-aware controller can then use

$$a_t^* \in \arg\max_{a \in \mathcal{A}_t} \left[\widehat{\Delta}_{\mathrm{down}}(a) - \lambda_t \, \mathrm{Cost}(a) \right], \tag{6}$$

where λ_t controls the trade-off between estimated gain and cost. The optimization is subject to coverage, provenance, safety, and protected-slice regression constraints. The Data Provenance Initiative demonstrates why source, licensing, and downstream-use records must remain attached to selected data (Longpre et al., 2024a). DataComp demonstrates the complementary need to hold training and compute fixed when comparing candidate datasets through downstream outcomes (Gadre et al., 2023). Here $\mathcal{Q}_t^{\mathrm{sel}}$ is the controller-exposed evaluation distribution; an adaptive evaluation queue is recorded in the control state or loop history, while a fixed $\mathcal{Q}^{\mathrm{audit}}$ is never used for data selection, controller calibration, or policy updates. These equations are our survey abstraction of how systems estimate counterfactual value without fully training every candidate.

The literature instantiates this counterfactual at several fidelities: LESS uses optimizer-aware influence (Xia et al., 2024), Data Mixing Laws uses proxy training runs (Ye et al., 2025a), and BigBang returns to full post-update evaluation (PolarSeeker, 2026).

An optimal-control view makes the sequential dependence explicit: a useful example changes the learner state against which later examples are valued. PDS formulates pre-training selection as a generalized optimal-control problem and derives conditions linking the selection decision to language-model training dynamics (Gu et al., 2025). It remains an offline selector rather than a recursive feedback policy, but it clarifies why a one-time, pointwise quality score is an incomplete approximation to Eq. 6.

4.2.1 Target Relevance from Downstream Failures

Selection first needs a target distribution. A deployment workload, held-out task set, or capability profile can convert failures into weights over skills and domains; candidates are then valued by how well they address that demand. DoReMi is an important domain-level contrast: it uses group distributionally robust optimization (Group DRO) on a proxy model to learn domain weights without downstream-task knowledge, so it provides a proxy-optimized mixture-allocation baseline rather than failure-conditioned target relevance (Xie et al., 2023). LESS estimates the alignment between candidate instruction data and a downstream target through optimizer-aware influence, providing a representative one-pass approximation to target relevance (Xia et al., 2024). ICONS extends target-aware influence selection to vision–language mixtures and aggregates task-level influence through consensus across validation tasks (Wu et al., 2024). BigBang uses performance on held-out real research tasks to identify capability gaps and adjust the domain distribution, difficulty, and capability requirements of subsequent synthetic tasks, closing target-relevance estimation into an iterative loop (PolarSeeker, 2026).

Across these cases, target relevance is narrower than general data quality: an example for an already-saturated skill may have less marginal value than a verifiable example covering a current failure. GRAPE further shows that single-target mixture optimization can improve one objective while degrading other targets (Fan et al., 2025). The demand profile should therefore include protected coverage and distinguish $\mathcal{Q}_t^{\text{sel}}$, which the controller may adapt to, from an untouched $\mathcal{Q}^{\text{audit}}$.

4.2.2 Learner State, Difficulty, and Marginal Learnability

The value of an example changes as the learner changes. Static quality filtering ignores whether the current model already masters the example, cannot yet learn from it, or would benefit from a nearby prerequisite. At example granularity, MATES uses local probes from the current model to learn a data-influence estimator and refresh selection as pre-training proceeds (Yu et al., 2024b). GREATS moves the refresh inside training by selecting data at every optimization iteration (Wang et al., 2024c). At domain granularity, ADO estimates each domain's remaining learning potential with online scaling laws and adjusts the mixture concurrently with training, without a separate proxy model (Jiang et al., 2025b). DRIFT supplies the analogous instruction-data view by using current-model rollouts as validation anchors for attribution (Wang et al., 2026g). HDS then illustrates that the scheduler may need to negotiate quality, difficulty, diversity, and domain objectives rather than impose one universal ranking (Dang et al., 2026). Despite their different granularities and update schedules, their common contribution is to make utility local to the current learner state.

MobileGen operationalizes a capability frontier by separating structural and semantic difficulty before generating new GUI trajectories (Kang et al., 2026a). Task-centric theory gives conditions under which moderate difficulty and easy-to-hard scheduling improve finite-sample guarantees over fixed mixtures (Liu et al., 2026a). Difficulty should therefore not be equated with utility: easy data can be redundant, while very hard data can be unverifiable or outside the learner's present support. MATES also makes clear that probe-based utility is conditional on a current model and optimizer state (Yu et al., 2024b); it should be refreshed after material updates and treated as local evidence rather than a permanent ranking.

Curriculum systems move one step further by changing the candidate distribution itself. WebRL generates new web-agent tasks from unsuccessful attempts, making current failures a source for the next online curriculum (Qi et al., 2025). MobileGen profiles structural and semantic capability frontiers, then uses the profile to update a challenge-centered difficulty distribution for trajectory synthesis (Kang et al., 2026a). Ouroboros-Spatial operates at a finer granularity: it uses the solver's per-sample confidence to steer a frozen proposer toward spatial questions matched to the solver's evolving ability (Zhao et al., 2026a). A task-centric analysis supplies a complementary theoretical view: under explicit conditions on initialization, difficulty, and sample budget, easy-to-hard self-improvement can dominate a fixed mixture, while the feedback loop still predicts eventual saturation (Liu et al., 2026a). Together these works distinguish a fixed curriculum from learner-conditioned data demand; because self-generated supervision can preserve a gap to oracle feedback, learner-derived success signals can inherit the learner's blind spots (Qi et al., 2026).

4.2.3 Coverage, Complementarity, and Exploration

Data Diversity Matters treats instruction diversity as a robustness objective rather than an incidental by-product of quality filtering (Bukharin et al., 2024). COINCIDE measures vision–language coverage through concept–skill transferability rather than raw visual dissimilarity (Lee et al., 2024a). At selection time, Quad combines influence, clustering, and exploration to reduce the redundancy of taking only the highest-scoring records (Zhang et al., 2025a). Together these works motivate complementarity across capabilities, domains, reasoning patterns, modalities, and trajectory states.

Group-MATES makes the unit of utility a selected set rather than an isolated record: it collects reference-loss influences along sampled training-data trajectories and scores each candidate conditional on the previously selected set (Yu et al., 2025d). This provides a bridge between pointwise influence and set-level coverage, although the learned relationships are local estimates rather than identified causal complementarities.

Frontis-MA1 makes this trade-off explicit in executable machine-learning engineering. At inference, it selects candidate program nodes using quality, parent-relative progress, and method-family novelty, then retrieves bounded, operator-conditioned evidence from their experience cards. The Draft, Improve, Debug, and

Crossover interfaces trained with execution-grounded SFT and reinforcement learning (RL) are composed again during long-horizon search (Yang et al., 2026a). Its experience cards are therefore mutable runtime data for orchestration; the remaining limitation is that fixed hand-designed search weights leave the relative value of quality, progress, and novelty task-independent.

RSIBench-Data shows the opposing risk to this exploration policy: continuing a search after its observed peak frequently yields a worse final attempt (Meng et al., 2026). Exploration is therefore not automatically beneficial. A credible policy should report its exploration budget, the coverage measure that prevents collapse, and the evidence that terminates an unproductive branch.

4.2.4 Proxy Screening and Outcome Calibration

Data Mixing Laws fits small-scale mixture runs as a lower-cost proxy for unseen target-scale mixtures (Ye et al., 2025a). DCLM instead fixes the training recipe and downstream evaluation to compare curation procedures at scale (Li et al., 2024c). These are distinct evidence levels: the former screens candidate allocations, whereas the latter isolates data-pipeline differences under controlled training. Outcome-based recalibration requires the feedback path illustrated next.

BigBang is a clear instance of this outcome-calibrated pattern. It uses critics as fast proxies and periodically selects representative synthesis pipelines for higher-fidelity validation. Model variants trained on their outputs are evaluated on held-out real tasks, and a meta-critic uses discrepancies between critic assessments and observed training outcomes to revise evaluation criteria and generation strategies (PolarSeeker, 2026). DataMaster holds the learning algorithm fixed so that downstream results compare alternative data-engineering branches as a whole, although this comparison does not by itself isolate the contribution of an individual discovery, selection, cleaning, or transformation step (Du et al., 2026).

DataComp provides a precedent for isolating the data contribution under fixed training and compute (Gadre et al., 2023). For adaptive selection, an analogous experiment would compare candidate policies under matched budgets and reserve an audit distribution outside the controller; without that design, evidence supports the selected pipeline as a whole rather than a causal claim about its utility estimator.

4.2.5 Pipeline-Level Feedback without Full Learner Updates

The progression from data tooling to orchestration is a progression in what feedback may change. DataJuicer established a configurable operator and recipe abstraction, together with model-based evaluation of alternative recipes, but the user still specifies the processing policy (Chen et al., 2024a). Later systems make per-example plans, revise the preparation policy from trial outputs, or let post-training outcomes alter the next data strategy. Table 2 separates these feedback depths; sharing an operator library does not make their recursive boundaries equivalent.

Full retraining is not the only way to make data preparation adaptive. The automatic data-preparation DataEvolver builds a profile from sampled raw data, fixed high-quality seeds, and accumulated experience; expands an operator library; assembles operators into an executable directed acyclic graph (DAG); and tests candidate pipelines on trial subsets. Logical and execution checks establish whether a plan can run; discrepancies between trial outputs and the seed examples update the profile and experience memory used by later plans (Deng et al., 2026).

Within this design, logical, execution, and seed-discrepancy checks update the preparation plan, whereas downstream SFT is validation after the inner search. It is therefore a preparation-policy loop, not a fully closed learner–data loop. Seed-level feedback makes search cheaper, but learned-proxy overoptimization shows why proxy improvement need not guarantee downstream utility (Gao et al., 2023). Periodic downstream checks are needed to test when that relationship fails.

DataEvolve closes a related loop at the curation-strategy level: judge scores and diagnostics on sampled original–cleaned document pairs revise per-category strategies before selected strategies are applied at corpus scale (Mi et al., 2025). Its later pre-training results validate the searched strategy, whereas BigBang makes post-training outcomes part of the controller for subsequent synthesis (PolarSeeker, 2026). Keeping these recursive boundaries explicit is more informative than labeling both systems simply "self-evolving."

4.3 Representative Adaptive Data-Orchestration Systems and Their Recursive Boundaries

The recursive boundary is the highest-level outcome that feeds back to change a later data decision. DataOrchestra represents input-conditioned, per-example planning without a reported online policy update (Huang et al., 2026). BigBang represents the stronger case in which post-training outcomes control later synthesis and evaluation strategies (PolarSeeker, 2026). Table 2 organizes representative systems between these boundaries by the decision that changes data and the point at which feedback returns. This prevents all automated data pipelines from being described as equivalent closed loops.

The table's contrast between preparation-policy recursion and downstream-controlled recursion does not imply an evidence ranking: a full loop can entangle changes in data, synthesis strategy, critic criteria, and training. Repeated adaptation can also expose the controller's selection benchmark to overfitting (Singh et al., 2025); stronger causal evidence still depends on controlled comparisons (Gadre et al., 2023). Thus, "representative" means that a mechanism clarifies the design space, not that it has broad reproduction or field consensus.

The explicit state separation summarized in Table 2 suggests a modular research direction: future systems can expose and separately calibrate the target profile, utility estimator, exploration rule, and promotion gate in Eq. 4. Such factorization makes a pipeline easier to compare, ablate, and transfer while preserving its data-centric contribution.

4.4 Controller Topologies as Implementations

MAST shows that multi-agent failures arise from coordination and inter-agent misalignment as well as from individual reasoning errors (Cemri et al., 2025). This motivates treating teachers, judges, critics, planners, schedulers, tools, and humans as implementations of the preceding decision functions rather than as the taxonomy itself. Their topology matters because it determines which errors are shared, which decisions are inspectable, and where independent evidence can enter.

The literature also reflects a transfer of control over feedback data. Reinforcement learning from human feedback (RLHF) pipelines established human demonstrations and comparisons as external data for supervised and reward-guided policy updates (Ouyang et al., 2022). Constitutional AI moved part of that path to model-generated critiques, revisions, and AI preferences constrained by a fixed constitution (Bai et al., 2022). Self-Rewarding Language Models then used the improving model itself to produce rewards for later iterative preference updates (Yuan et al., 2024a). This progression raises scale but can weaken signal independence, which is why selective escalation and role separation matter more than the nominal number of agents.

4.4.1 Single-Controller Selection

MATES uses local probes from the current model to refresh example selection during pre-training (Yu et al., 2024b). ADO instead uses current per-domain loss trajectories to update mixture weights during training (Jiang et al., 2025b). Both instantiate a single controller that consumes learner-state evidence and emits a new sampling decision. This topology is inexpensive and replayable, but a biased controller can recursively reshape the evidence on which it is recalibrated; learned-proxy overoptimization illustrates the resulting dependence risk (Gao et al., 2023). Conservative update sizes, protected slices, and periodic recalibration are therefore central safeguards.

4.4.2 Proposer–Critic and Proposer–Verifier Separation

CoVerRL alternates generator and verifier roles to escape a consensus trap, but its use of one model for both roles also shows that role separation alone does not create an independent signal (Pan et al., 2026). A proposer can synthesize or transform candidate data while a critic explains defects and a verifier decides whether evidence supports acceptance; the benefit comes from informational separation, not component count. An executable checker can reject a program using evidence unavailable to the generator, whereas a critic from the same model family may merely restate the proposal. BigBang's distinction between critic

Table 2: Representative data-centric orchestration systems. The table emphasizes the feedback-to-decision-to-data path and states where recursion closes; it is not an exhaustive catalogue of automated data construction.

Representative work	Feedback used	Data decision	Mutable object	Loop closure
DataEvolve (Mi et al., 2025)	LLM-judge scores and diagnostics on sampled original–cleaned document pairs	Revise the curation strategy across iterations	Per-category strategy, experience and strategy pools, and sampled cleaned outputs	Curation strategy updates from sample feedback; pre-training only validates it
DataEvolver for automatic data preparation (Deng et al., 2026)	Logical and execution checks, then discrepancies between trial outputs and fixed high-quality seeds	Evolve operators, a preparation DAG, and feedback memory	Operator library, DAG, data profile, experience memory, and trial outputs	Preparation policy updates from seed discrepancies; training only validates outputs
DataOrchestra (Huang et al., 2026)	Tool-execution feedback and stage-specific verifier judgments during offline plan evolution	Drop, leave untouched, or clean a chunk and select its processing stages	Chunk-level curation plan, rewriting instructions, and processed chunk	Offline feedback trains the orchestrator; no online policy update is reported
OpenDataArena-driven dataset engineering (Gao et al., 2025)	Value-anchored rankings and multi-dimensional analyses of candidate data	Construct difficulty-aware mathematics data and patch multi-domain SFT mixtures	Selected records, difficulty composition, and domain mixture	Rankings guide construction; downstream evaluation does not trigger another update
DataMaster (Du et al., 2026)	Downstream training and evaluation outcomes under a fixed learning algorithm	Select the next data branch, source, composition, or transformation	DataTree, Data Pool, and Global Memory	Downstream outcomes update the DataTree; the learning algorithm stays fixed
BigBang (PolarSeeker, 2026)	Critic judgments and held-out real-task performance after training	Revise domain, capability, difficulty, synthesis, and evaluation policies	Frontier-task distribution, synthesis programs and strategies, and critic criteria	Post-training outcomes update critic and synthesis policies
Frontis-MA1 (Yang et al., 2026a)	Execution status and scores; training-time reward, child variance, and visit cooling; search-time quality, progress, and novelty	Select parent programs and bounded operator-conditioned evidence, then apply a trained operator	Programs, trajectories, experience cards, and search board	Inference updates programs and experience, not model weights
Multimodal DataEvolver (Yan et al., 2026b)	Fixed-verifier quality statistics and rejection causes summarized into round-level feedback	Revise retrieval queries and generation prompts, and synthesize for under-covered regions	Candidate and pass sets, query and prompt policies, targeted samples, and semantic-feedback memory	Verifier feedback updates construction policy; fine-tuning only evaluates data

judgments and observed post-training outcomes illustrates how a higher-fidelity channel can recalibrate a cheaper one rather than simply vote with it (PolarSeeker, 2026).

This topology can still fail when the proposer and verifier share training data, reward shortcuts, or benchmark exposure. Preference leakage makes this correlation observable despite assigned role separation when the underlying models share identity, lineage, or family (Li et al., 2026a). Repeated agreement is not evidence of independence. The controller should therefore record verifier coverage and abstentions and route unsupported cases to a tool, model family, or human gate with a genuinely different evidence source.

4.4.3 Distributed and Multi-Agent Control

AutoVQA-G separates fine-grained visual verification from a prompt-optimization agent that mines critiques of failed samples to revise subsequent grounded-VQA annotation (Hu et al., 2026c). This illustrates how a multi-agent system can distribute diagnosis, retrieval, generation, verification, and scheduling when the data unit itself is structured. The multimodal DataEvolver in Table 2 provides a second pattern: rejection causes and round-level failure statistics are distilled into semantic feedback that revises later retrieval and targeted synthesis rather than disappearing after filtering (Yan et al., 2026b).

Automated multi-agent attribution shows that distributed trajectories make component-level causes harder to identify (Zhang et al., 2025d). Agent messages may alter prompts without leaving a structured data diff, correlated agents can amplify one another's error, and distribution can impose nontrivial coordination cost. MAST's taxonomy confirms that failures arise from coordination and inter-agent misalignment as well as from an individual model's reasoning error (Cemri et al., 2025). An auditable controller should therefore expose role-specific observations, decisions, and writes to shared state. If those effects cannot be separated, the system should be reported as a composite pipeline rather than attributing gain to one orchestration mechanism.

4.4.4 External and Human-Gated Control

InstructGPT demonstrates the high-cost external path in which humans supply demonstrations and rank model outputs for later training (Ouyang et al., 2022). OmniOPD demonstrates selective escalation by sending the student's peak-entropy reasoning forks to teacher verification instead of requesting feedback uniformly (Zhou et al., 2026c). These designs motivate reserving external tools or humans for low-coverage or high-consequence decisions: calibrating ambiguous slices, validating a proposed policy change, adjudicating regressions, or authorizing promotion. Independence is thereby also a budgeted feedback-allocation problem.

A trained multimodal fact verifier offers a specialized channel for evidence-grounded claims, although its use as an adaptive orchestration gate remains to be established (Lee et al., 2024b). On reasoning and planning tasks, sound external verification has produced gains where unsupported iterative self-critique can collapse (Stechly et al., 2025). Yet external feedback still has a support boundary: symbolic checks may be silent for open-ended quality, while human overseers can inherit the system's framing and confirmation bias (Recchia et al., 2025). Governance must retain signal source, scope, and confidence so that "external" is not mistaken for universally correct.

4.5 Routing Decisions Across Stages and Model Types

4.5.1 Stage-Specific Data Destinations

The same diagnosed gap can be routed to different mutable objects. In pre-training, ADO changes domain mixture weights while training proceeds (Jiang et al., 2025b). In SFT, error-driven multimodal tuning retrieves instructions aligned with diagnosed reasoning failures (Yao et al., 2025). In preference optimization, factually augmented RLHF changes grounded comparison data (Sun et al., 2024). In on-policy distillation (OPD), OmniOPD selects student-conditioned reasoning forks for teacher feedback (Zhou et al., 2026c). In context and memory adaptation, Frontis-MA1 selects experience cards and updates a search board rather than model weights (Yang et al., 2026a). These destinations differ in update cost, latency, persistence, and the kind of failure they can repair, so stage choice is itself a data-allocation decision.

BigBang routes held-out capability gaps toward new task distributions and synthesis strategies (PolarSeeker, 2026), whereas MED-VRAG iteratively revises queries and accumulates page-image evidence in runtime memory (Chen et al., 2026c). These routes are alternatives, not a mandatory pipeline. Routing to a convenient stage rather than the causal bottleneck can conceal a failure temporarily, so the controller should state why its destination should alter the diagnosed mechanism and which evaluation would falsify that expectation. Existing surveys organize most self-improvement methods within a preselected component or stage, leaving matched-budget evidence for choosing among pre-training, SFT, preference data, and runtime memory sparse (Deng et al., 2025).

4.5.2 Modality-Specific Evidence and Data Granularity

OmniOPD selects uncertainty-bearing reasoning forks as textual chunk-level supervision units (Zhou et al., 2026c), while MED-VRAG keeps retrieved evidence as document-page images (Chen et al., 2026c). These examples show that the control functions need not change across model types, but the signal space, reliability boundary, and mutable data unit do. A controller must preserve links to spans, image regions, frames, audio segments, renders, or actions when selecting and rewriting data; a fluent textual correction can otherwise erase the evidence that made the example useful.

Existing systems illustrate several such data units. Factually augmented RLHF makes externally grounded multimodal feedback part of preference construction rather than relying on response style alone (Sun et al., 2024). DriveStack-VLA supplies a distinct embodied example: it uses a render teacher to align supervision with bird's-eye-view geometry, so the selected correction targets a perceptual representation rather than only a textual answer (Wang et al., 2026b). In both cases, the orchestration implication is that the controller must route a diagnosed error to a data object whose modality preserves the missing evidence.

VLM judges can retain useful pairwise ranking while remaining unreliable for absolute score thresholds, making their role in a data gate use-dependent (Kumar et al., 2026). Recursive multimodal training also exhibits modality-specific collapse, and mitigation depends on which modality and label channel is refreshed (Hu et al., 2025). Modality-aware verification and mixture constraints are therefore needed; an unsupported signal should trigger abstention or escalation rather than a confident update.

4.6 Evidence Standards and Auditability

DataOrchestra trains an input-conditioned planner but does not report online updates to that deployed policy (Huang et al., 2026). BigBang instead uses accumulated post-training evidence to revise later task distributions, critic criteria, and synthesis strategies (PolarSeeker, 2026). This contrast shows why the presence of a teacher, judge, critic, planner, or agent is not by itself evidence of adaptive orchestration. A fixed accept/reject verdict implements only stage-local gating; the stronger claim begins when accumulated feedback changes a later target, operator, allocation, gate, or policy in a way distinguishable from the fixed rule.

Datasheets establishes a structured record of dataset motivation, composition, collection, and use (Gebru et al., 2021). The Data Provenance Initiative broadens that record to sources, creators, licensing conditions, and downstream reuse (Longpre et al., 2024a). Building on these documentation precedents, we propose that an orchestration study expose five linked objects:

1. **Signal and provenance:** the observation received by the controller, its source, uncertainty, support, and prior exposure;
2. **Decision space:** the feasible alternatives for z_t, o_t, π_t, b_t, and g_t, including abstention and stopping;
3. **Data delta:** the exact records, weights, order, metadata, or memory state changed by the selected action;
4. **Decision evidence:** a comparison with a fixed, random, or ablated policy that isolates the value of adaptive selection; and
5. **Trajectory and governance:** downstream outcomes across rounds, protected-slice regressions, costs, promotions, rollbacks, and termination reasons.

Implicit Execution Tracing shows a complementary controller-side problem: when ordinary logs are absent, even segment boundaries and agent attribution must be reconstructed from embedded signals (Nian et al., 2026). The five objects above form our proposed replayable record linking the observed signal, selected action a_t, realized data delta, and resulting learner state; reporting only a final dataset, score, or controller prompt hides the causal data delta.

DataComp supplies a controlled-comparison precedent by holding training recipe and compute scale fixed while changing the candidate multimodal dataset (Gadre et al., 2023). Adaptive orchestration requires an additional shift from static artifact documentation to intervention provenance, in which the triggering signal, feasible alternatives, selected data delta, promotion, and rollback remain linked across rounds.

Controller-level lineage is equally necessary. Automated multi-agent failure attribution asks which component caused a task failure and at which point in the trajectory (Zhang et al., 2025d). Behavioral-reproducibility work adds whether repeated invocations choose the same tools, in the same order, with the same arguments (Yagubyan, 2026). Together with execution tracing above, these results motivate recording provenance by design rather than inferring it after a dataset or checkpoint has already been promoted.

DataEvolver illustrates the preparation-loop boundary: downstream SFT validates its prepared output but does not control the reported inner preparation loop (Deng et al., 2026). DataMaster illustrates the operation-attribution boundary: downstream gains compare complete data-engineering branches but do not isolate every internal operation (Du et al., 2026). Arena analyses illustrate the audit-independence boundary by documenting incentives to adapt repeatedly to one selection environment (Singh et al., 2025). Thus final-only evaluation supports pipeline adaptation plus downstream validation, jointly changed variables support system-level rather than selector-level claims, and a reused selection benchmark is not an independent audit. Each mechanism should state these boundaries locally rather than delegating them to a separate limitations discussion.

4.7 From Orchestration to Execution

BigBang makes the separation concrete: held-out evaluation identifies capability gaps, its controller revises the task and synthesis distribution, and training realizes the selected intervention (PolarSeeker, 2026). Evaluation therefore establishes what has been observed; orchestration chooses a data transition; and execution realizes it. DataEvolver marks an earlier boundary, closing feedback at preparation-policy refinement while using downstream training only for validation (Deng et al., 2026). Section 5 instantiates these decisions across pre-training, SFT, reinforcement learning, on-policy distillation, and context or memory adaptation; across all stages, feedback counts as orchestration only when it changes a later data state whose effect is evaluated again.

5 Execution Mechanisms: Data-Centric Updates Across Training and Adaptation

Table 3: Representative execution-layer mechanisms in pre-training and supervised fine-tuning. Each row records how evaluation feedback updates a mutable data-related object.

Method	Stage	Data operation	Updated object	Feedback signal	Update and loop closure	Main risk
Ultra-FineWeb (Wang et al., 2025g)	PT	Corpus filtering/selection	Seed set and filter	Downstream validation	Revises seeds and filtering rules for later retained documents	Proxy overfitting; narrowed coverage
MATES (Yu et al., 2024b)	PT	Model-aware selection	Pre-training examples	Current-model influence	Refreshes sample selection for later training	Early-model bias against long-tail data
Group-MATES (Yu et al., 2025d)	PT	Group-level selection	Example groups	Group influence	Selects groups conditional on earlier selected data	Unstable complementarity estimates
DataEvolve (Mi et al., 2025)	PT	Rewriting/ refinement	Cleaning strategy and corpus	Judge scores and diagnostics	Revises cleaning strategies across rounds	Evaluator bias; semantic drift
ADO (Jiang et al., 2025b)	PT	Mixing/ reweighting	Domain sampling weights	Online loss trajectories	Updates domain proportions during training	Short-term gain bias
RegMix-D (Zhao et al., 2026b)	PT	Dynamic mixing	Domain mixture weights	Mixture-switch losses	Revises later data proportions from proxy trajectories	Accumulated allocation bias
DEITA (Liu et al., 2024b)	SFT	Instruction selection	Instruction pool	Quality, complexity, diversity	Selects examples for SFT	Proxy-metric overfitting

Continued on next page

Table 3 (continued)

Method	Stage	Data operation	Updated object	Feedback signal	Update and loop closure	Main risk
LESS (Xia et al., 2024)	SFT	Target-aware selection	Instruction examples	Target-task influence	Selects examples for targeted SFT	Target-distribution overfitting
VisionFoundry (Zhou et al., 2026a)	SFT	Generation/ filtering	Image-instruction pairs	Verifier accept/reject	Removes unsupported pairs before SFT	Shared visual blind spots
Self-Evolving Visual Questioner (Liang et al., 2026)	SFT	Iterative generation/rewriting	Question pool	Judge verdicts and rewrites	Updated VLM generates the next pool	Evaluator entanglement
MobileGen (Kang et al., 2026a)	SFT	Failure-driven repair	GUI instructions and trajectories	Capability profile	Changes next-round difficulty distribution	Incorrect failure attribution
AutoVQA-G (Hu et al., 2026c)	SFT	Grounded annotation repair	VQA annotations and prompts	Critiques of failed annotations	Revises prompts for the next data round	Shared perception errors
Socratic-Geo (Jiao et al., 2026)	SFT	Failure-driven synthesis	Geometry examples	Solver failures	Uses failures for targeted augmentation	Overfocus on observed failures

This section turns to the execution layer: the concrete stage-specific updates through which selected feedback changes the data-related objects available to later rounds. We organize execution mechanisms by training and adaptation stage: pre-training (PT), supervised fine-tuning (SFT), reinforcement learning (RL), on-policy distillation (OPD), and context or memory adaptation. This stage-based view is useful because the same feedback signal implies different interventions at different points in the pipeline.

5.1 Pre-training: Evaluation-Triggered Data Interventions

Pre-training is the earliest execution stage at which evaluation feedback can alter foundational training data and thereby influence subsequent model capabilities. Because the downstream effects of such interventions are often observed only after substantial training, data updates frequently rely on incomplete or early-stage evaluation signals. We treat a method as core evidence of data-centric recursive improvement when evaluation outcomes change pre-training data that are reused in a later training round; one-shot scoring and fixed corpus construction instead provide supporting evidence for the corresponding data operations. According to the executed data operation, we organize related work into four families: *Corpus filtering and selection*, *data rewriting and refinement*, *data synthesis and targeted mining*, and *data mixing and reweighting*.

5.1.1 Corpus filtering and selection.

Corpus filtering and selection determine which existing examples are retained for subsequent pre-training. DataComp-LM (Li et al., 2024c) and FineWeb (Penedo et al., 2024)compare alternative filtering and corpus-construction strategies through controlled pre-training, while QuRating (Wettig et al., 2024) and Meta-rater (Zhuang et al., 2025) introduce model-generated or model-validated quality signals to guide document selection. PDS (Gu et al., 2025) and BETR (Mizrahi et al., 2025) further estimate the training value of candidate data using training dynamics, data influence, diversity, or relevance to target tasks, while related work extends these selection mechanisms to training-loss-based pruning (Ye et al., 2026a), multilingual quality filtering (Turki et al., 2026), and multimodal pre-training data curation (Farina et al., 2026). Although these methods shift data selection away from fixed heuristics toward evaluation-informed decisions, the final training set is typically determined before the target training run begins.

More direct recursive updates arise when evaluation outcomes are used to modify subsequent data selection. Ultra-FineWeb (Wang et al., 2025g) validates candidate high-quality data through downstream model performance and uses the resulting evidence to revise its seed set and filtering classifier, thereby changing which web documents are retained. GREATS (Wang et al., 2024c) directly estimates candidate utility through the expected validation-loss reduction after updating the current model with each candidate, and reselects the samples used for the next parameter update at every iteration; although its large-scale experiments focus on fine-tuning, the same online-selection mechanism is also validated in a GPT-Small pre-training setting on OpenWebText.

Influence on the current model provides another direct criterion for data selection. MATES (Yu et al., 2024b) locally probes the current pre-training model to estimate data influence and uses these estimates to update data selection for the next training stage. Group-MATES (Yu et al., 2025d) extends this estimation to interactions among groups of examples. Layer-Aware Influence (Yang et al., 2025c) uses the current batch as a self-influence reference during pre-training, estimates sample influence online from output-layer gradients, and removes negatively influential samples before each update, so that the subset actually used for training changes with the model state. Data value can also depend more directly on the current training state. BLADE (Wang et al., 2026a) updates online selection according to the loss gap between candidate data and a dynamic reference that evolves with training, whereas OPUS (Wang et al., 2026d) estimates candidate utility in the optimizer-induced update space, incorporating the current optimizer state into the effective update geometry, and uses the resulting projected utility to guide subsequent sampling. Data value therefore becomes not only an intrinsic property of a sample, but also a quantity that changes with model state and training stage.

However, repeated reliance on current-model feedback can narrow corpus coverage by systematically reducing exposure to long-tail, low-resource, or poorly understood data that are undervalued early in training.

5.1.2 Data rewriting and refinement.

Data rewriting and refinement modify the content, structure, or representation of existing training examples so that data with useful information but quality defects can remain in pre-training. ProX (Zhou et al., 2025), RefineX (Bi et al., 2025), REWIRE (Nguyen et al., 2025b), and SwallowCode and SwallowMath (Fujii et al., 2026) improve existing text through example-specific processing programs, editing strategies, and generative rewriting, while AITQE (Huang et al., 2024) and PMC-InterCPT (Zhu et al., 2026a) extend similar operations to image–text consistency and multimodal context reconstruction. These studies show that rewriting and refinement can improve pre-training corpora, although their processing strategies are typically specified before large-scale training begins.

Building on this foundation, some methods use quality assessments of rewritten outputs to adjust the data-processing procedure itself, allowing evaluation outcomes to directly affect the content that ultimately enters pre-training. RePro (Yu & Xiong, 2025) constructs rewards from the quality and semantic faithfulness of rewritten text and uses them to optimize a rephraser, so that the updated policy produces a different rewritten pre-training corpus. CTRL (Liu et al., 2024c) iteratively generates revised texts, filters out candidates that degrade readability or helpfulness, and retains lower-perplexity ones for subsequent rounds of refinement; although it targets safety-oriented (query, response) pre-training texts rather than general web corpora, the iterative revision loop illustrates the same evaluation-driven rewriting pattern.

Other methods use verification of execution results to drive further data revision. DataOrchestra (Huang et al., 2026) revises example-level processing plans according to the actual outputs of data operations. For noise pruning, a verifier checks whether useful content has been mistakenly removed; for LLM rewriting, it checks whether noise has been removed without introducing factual errors or information loss. Failed rewrites are re-executed with corrective instructions, allowing execution feedback to directly affect the final processed data. DataEvolve (Mi et al., 2025) repeatedly performs quality-problem identification, candidate strategy generation, execution, and evaluation, and uses the outcomes of one round to revise the cleaning and refinement strategies applied in the next.

Nevertheless, repeated refinement can accumulate evaluator bias, leading to semantic drift, information loss, or stylistic homogenization in the resulting corpus.

5.1.3 Data synthesis and targeted mining.

Data synthesis and targeted mining both expand the current corpus by adding new training units, but they differ in where the added data come from. The former constructs new training content from existing corpora, model generations, or structured transformations, whereas the latter identifies and introduces relevant examples from external data sources according to specific needs. MIND (Akter et al., 2025), BeyondWeb (Maini et al., 2025), and FinePhrase (Niklaus et al., 2026) expand pre-training corpora by generating or restructuring existing content and evaluate alternative construction strategies through downstream training outcomes.

DoPAMine (Arannil et al., 2024) retrieves domain-relevant documents from large web corpora using domain seeds, while WRAP++ (Zhou et al., 2026b) exploits cross-document relations to discover and organize new training content. These approaches can effectively expand the training corpus, but their generation or acquisition targets are typically specified in advance and do not continually adapt to the current model state. Related mechanisms extend such synthesis to underrepresented modalities, for example synthetic interleaved speech–text data for pre-training (Zeng et al., 2025).

Building on these approaches, some methods begin to adjust newly added data according to the behavior or learning state of the current model. Self-Improving Pretraining (Tan et al., 2026) evaluates current-model rollouts, original suffixes, and externally rewritten suffixes for quality, safety, and factuality, and uses these judgments to select the continuations used in subsequent pre-training. Active Recap Learning (Hui, 2026) uses the current model's long–short gap to identify tokens that depend strongly on distant context, retrieves the preceding segments most useful for predicting those tokens, and converts them into recap-augmented sequences for long-context continued pre-training. BoLT (Ruan et al., 2025) iterates between latent-thought inference and model training, using latent thoughts inferred by the current model to construct new thought-augmented training data, after which the updated model participates in the next round of data construction. SynPro (Yu & Xiong, 2026) further updates rephrasing and reformatting generators using quality, faithfulness, and data-influence rewards, and then uses the updated generation policies to produce synthetic tokens for the next training stage. Together, these methods make the selection, construction, or generation of new data increasingly dependent on the current model state and evaluation outcomes.

A related risk is that data expansion driven by model state or evaluation signals can amplify existing errors and leave knowledge regions outside the current evaluation coverage underrepresented.

5.1.4 Data mixing and reweighting.

Data mixing and reweighting primarily adjust the training proportions and sampling probabilities of data from different domains, sources, languages, or modalities without changing the content of individual examples. DoReMi (Xie et al., 2023), DoGE (Fan et al., 2024b), Data Mixing Laws (Ye et al., 2025a), and RegMix (Liu et al., 2025b) use proxy training, gradient relations, or mixture-performance prediction to identify improved data mixtures. CLIMB (Diao et al., 2025) further iterates between proxy training and performance prediction to progressively refine candidate mixtures over semantic data clusters. Although this search process is iterative, in all of these approaches the selected mixture is fixed before the subsequent target-model training run. More recent work instead treats the sampling distribution as a mutable data state that can be updated continually as training progresses.

Some methods dynamically adjust subsequent data proportions according to training loss and learning progress. ODM (Albalak et al., 2023) updates domain sampling probabilities using recent loss reduction, while ADO (Jiang et al., 2025b) fits online loss scaling curves for individual domains and modifies the mixture according to the marginal gains expected from allocating additional data. Velocitune (Luo et al., 2025) and GRAPE (Fan et al., 2025) redistribute training proportions using different measures of learning progress. Aioli (Chen et al., 2025a) instead fits a unified optimization model to the observed training dynamics and periodically updates the mixture accordingly, while the online variant of RegMix-D (Zhao et al., 2026b) uses losses observed at mixture-switch points during target training to revise subsequent data proportions.

Other methods use gradient or influence information to estimate the value of different data sources for updating the current model. Dynamic Gradient Alignment (Fan et al., 2024a) adjusts sampling proportions according to gradient alignment between source domains and target data. PiKE (Li et al., 2025d) uses gradient norm and variance to modify the contributions of different domains in subsequent batches, while TiKMiX (Wang et al., 2025f) uses group influence to estimate the marginal effect of increasing the proportion of a domain on validation objectives and updates the subsequent mixture accordingly.

Some approaches further combine multiple training signals to update the subsequent sampling distribution. HDS (Dang et al., 2026) integrates data quality, cross-domain influence, and model state into a multi-objective reward to update domain sampling proportions online. OP-Mix (Hu et al., 2026b) instead trains lightweight domain-specific LoRA adapters from the current model, interpolates these adapters to approximate candidate data mixtures, and evaluates the resulting proxies to select the mixture used for

subsequent training. AC-ODM (Ma et al., 2026a) and Data Mixing Agent (Yang et al., 2026b) similarly learn domain-reweighting policies from actor-critic optimization or historical data-mixing trajectories.

A corresponding risk is that dynamic mixing can overallocate training to domains with larger short-term measured gains, allowing allocation bias to accumulate across subsequent updates.

Overall, the four families above are not data operations introduced by Data-Centric Recursive Improvement itself. The key change is that data decisions traditionally fixed before training are increasingly revised according to model evaluations and training outcomes. Pre-training execution therefore shifts from one-shot data preparation toward evaluation-conditioned data updates, while repeated feedback may also lead to narrower data coverage, distorted data content, and accumulated bias in the training distribution.

5.2 Supervised Finetuning: Evaluation-Guided Instruction Data Construction

Supervised fine-tuning adapts pretrained models to task-following behavior through updates on instruction data. At this stage, the mutable object is the instruction pool, and evaluation supports recursive improvement only when outputs from the current model or an evaluator change the pool used in a later update. Evaluation takes on a control role when these outputs determine which examples are generated, selected, repaired, or grounded; a full model–data loop further requires the updated model to change the candidates or distribution in the next round.

5.2.1 Instruction Selection as a Data-Update Operator

Instruction selection treats the SFT pool as a mutable distribution. Selection criteria assess sample difficulty or complexity, diversity and coverage, and downstream utility, typically producing scalar metric scores; preference-based methods additionally use preference comparisons. A fixed selection or weighting rule maps these outputs to retention, pruning, weighting, or resampling, providing direct feedback-to-data evidence. Iterative control requires outputs from the current model or evaluator to revise the instruction pool used in a later round.

DEITA combines quality and complexity scores with diversity-aware selection (Liu et al., 2024b). LESS estimates target-task utility through influence scores (Xia et al., 2024). Data Diversity Matters provides supporting evidence that diversity and coverage affect robustness (Bukharin et al., 2024).

For multimodal data, PreSel uses an Instruction Relevance Score to assign task-wise sampling budgets and Neighbor Centrality scores to select representative images before instruction generation (Safaei et al., 2025). TIVE combines task-difficulty and instance-influence scores (Liu et al., 2025d). ScalSelect derives importance scores from instruction-conditioned attention representations and statistical leverage scores (Wu et al., 2026). ICONS aggregates influence estimates across target tasks into a consensus for data selection (Wu et al., 2024). COINCIDE ranks data by concept-skill transferability (Lee et al., 2024a). Align2LLaVA uses human preference comparisons to train reward models whose scalar scores filter questions and answers, after which the inner LLM rewrites and reviews the retained instructions (Huang et al., 2025b).

Most of these methods implement stage-local execution rules rather than repeated model-dependent selection. The main risk is mixture overfitting to a proxy metric: selected examples may improve benchmark scores while narrowing coverage of difficult visual cases.

5.2.2 Instruction Generation and Rewriting

Instruction generation and rewriting treat instructions, responses, rationales, demonstrations, and visual dialogues as mutable SFT objects. Teachers supply candidate supervision, judges or verifiers produce scores and accept/reject verdicts, and critics provide targeted feedback or revision requests. These outputs provide direct control-role evidence when they trigger generation, filtering, or rewriting, and iterative evidence when the updated model generates the next instruction pool.

Self-Instruct generates instruction triples and maps validity and similarity checks to accept/reject decisions (Wang et al., 2023). WizardLM uses Evol-Instruct to rewrite seed instructions into more complex variants (Xu et al., 2024a). X-InstructBLIP transfers instruction tuning across modalities (Panagopoulou et al.,

2024), while Video-LLaVA extends instruction following to unified image and video inputs (Lin et al., 2024a). These methods establish reusable data operations or supervision interfaces, but do not show evaluation after a model update revising a later supervision distribution.

VisionFoundry provides direct feedback-to-data evidence through verifier-filtered construction. It generates questions, reference answers, and text-to-image prompts and then renders the corresponding images (Zhou et al., 2026a). A VLM verifier assesses whether each image provides sufficient visual support for the intended answer and returns an accept/reject verdict as an actionable evaluation output. A fixed action-selection rule removes unsupported instruction–image pairs from the SFT pool, making this a one-pass execution pipeline rather than an iterative model-dependent loop.

Self-Evolving Visual Questioner provides iterative control evidence. The current VLM generates image-grounded questions, while a frozen reference checkpoint acts as a judge/evaluator and produces accept/reject verdicts and rewrite requests as actionable evaluation outputs (Liang et al., 2026). These outputs drive action selection by retaining, removing, or revising candidate questions, and the retained examples support question-generation and question-answering objectives. After SFT, the updated model generates the next instruction pool, so evaluation plays a control role in closing the model–data feedback loop.

The main risk is evaluator entanglement: a generator, teacher, or evaluator may share the language priors or perceptual blind spots of the model being trained. Iterative-improvement claims should therefore report the evaluation output, the data update it triggered, and performance on held-out grounding checks independent of the filtering rule.

5.2.3 Grounded and Structured Supervision

This family applies grounding-to-filter to both region-level evidence and its structured OCR, layout, chart, and temporal special cases. The mutable objects include grounded QA, region captions, masks, boxes, OCR instructions, chart QA, and frame-aware video instructions. Their primary quality property is conditional correctness, while generated charts also require artifact fidelity. Depending on the evidence type, evaluation produces rule-verifiable judgments such as IoU or execution success, accept/reject verdicts, localization scores, or learned-evaluator rationales. Fixed acceptance, repair, annotation, or weighting rules map these outputs to SFT data updates.

Osprey introduces mask-conditioned visual instruction tuning (Yuan et al., 2024b), and Draw-and-Understand supports points, boxes, and free-form visual prompts (Lin et al., 2025). LLaVAR adds OCR-aware visual instructions for text-rich images (Zhang et al., 2023). ChartGen exposes executable chart structure through code-guided generation (Kondic et al., 2025). VideoRefer Suite makes spatial-temporal reference and localization part of the instruction object (Yuan et al., 2025b), while Chain-of-Frames associates reasoning with frame-level evidence (Ghazanfari et al., 2026). These methods make supervision inspectable, but the evidence remains supporting unless a grounding output controls acceptance, repair, or a later data round.

VeriEvol provides direct feedback-to-data evidence. A type-aware evolution module rewrites low-difficulty image–question seeds into harder image-grounded prompts, while the HTV-Agent evaluates the reliability of each candidate answer (Li et al., 2026b). Independent solution traces and programmatic checks produce rule-verifiable judgments, and direct visual inspection produces an additional grounding verdict. Their combined accept/reject output drives a fixed acceptance rule that retains a candidate only when multi-source counter-evidence fails to refute its answer. The cycle updates prompt difficulty and the conditional correctness of answers before verified examples enter SFT.

The main risk is evaluator entanglement: a learned evaluator may share the perception errors of the generator. Generic QA transformations may also discard conditioning evidence such as OCR layout, numerical relations, or event order. Claims of iterative control should therefore identify the evaluation output, the resulting data update, and whether a later round depends on the updated model.

5.2.4 Failure-Driven Instruction Repair

This family converts observed failures into retrieved repair samples, hard examples, negative instructions, or revised sampling weights. The relevant evaluation outputs include error categories, missing-capability diagnoses, hard-case subsets, vulnerable subgroups, and capability profiles. These outputs support failure-directed data construction when they determine the supervision used in the next update; repeated evaluation after that update allows the repair distribution to evolve with the model.

Error-driven LMM tuning evaluates a student model and uses a teacher to produce missing-capability diagnoses that guide targeted retrieval from a task-agnostic instruction collection (Yao et al., 2025). ARDS uses perturbation-based robustness evaluation to identify vulnerable subgroups and map their difficulty to a robustness-aware training mixture (Yang et al., 2025b). Learning from Reasoning Failures uses frontier models to analyze reasoning errors, propose targeted examples, and filter them for quality before instruction tuning (Stan et al., 2026). LRV-Instruction instead derives positive and negative supervision from a predefined hallucination taxonomy rather than a diagnosis of the current model (Liu et al., 2024a). These methods connect diagnosis to data construction, but do not report repeated evaluation after the model update.

MobileGen implements an iterative model–data loop by repeatedly evaluating a GUI agent and producing a capability profile over structural and semantic difficulty (Kang et al., 2026a). The capability profile is a measurement output; when it determines the difficulty distribution for next-round instructions and trajectories, evaluation takes on a control role. Re-evaluating the updated agent then changes the subsequent data distribution.

Anchor Evolution converts model failures into truth-anchored SFT data through an iterative SFT–RL cycle (Wang et al., 2026h). Ground-truth answer matching produces rule-verifiable accept/reject verdicts, which are aggregated into rollout success counts and mapped by a fixed data-routing rule to redundant, volatile, and failing-frontier subsets. For the failing frontier, a teacher performs error attribution and produces error categories, retrieval keywords, and actionable hints as diagnostic outputs. The keywords retrieve verified anchors, while the hints construct scaffold-augmented supervision; after the model update, re-evaluation determines the next-round failure frontier, so evaluation progresses from measurement and diagnosis to a control role.

Socratic-Geo dynamically couples data synthesis with model learning: a Teacher agent generates geometric reasoning data, the Solver learns from it, and Solver failures guide targeted augmentation in the next round (Jiao et al., 2026). AutoVQA-G uses a consistency evaluator to critique failed annotations, after which a prompt-optimization agent revises the generation prompts for the next VQA-G data round (Hu et al., 2026c). The former updates both the model and subsequent data, whereas the latter primarily evolves the data-generation process.

RSIBench-Data provides an agent-controlled example of iterative SFT data synthesis. A researcher agent proposes message-format examples, executable tasks, or tool-use trajectories, while a fixed sandboxed evaluator returns selection scores, verifier outcomes, task trajectories, and execution diagnostics (Meng et al., 2026). These evaluation outputs guide revisions to data sources, filtering, curriculum, mixture, and supervision format before a shared SFT backend trains the next candidate checkpoint. Because each candidate is trained from the same base model, the loop adapts the data-synthesis strategy rather than carrying model parameters forward across rounds.

The main risk is incorrect error attribution: an observed failure may result from model capacity, prompt design, evaluator bias, or contamination rather than missing instruction data. Claims of repeated improvement should therefore identify the diagnostic output, the resulting data update, and how the updated model changes the next repair distribution.

The four intervention families provide a common description of SFT-stage data updates. A method belongs to the SFT execution layer when an output from a teacher, judge, or verifier directly changes instruction examples or mixture weights. The reliability of that output is analyzed in Section 3, while a controller that chooses the evaluator, target stage, retry policy, or stopping condition belongs to the orchestration layer in Section 4. Static instruction tuning remains important background, but a full co-evolution signal requires a

traceable path from evaluation output to the next supervised data object and from the updated model to a later data distribution.

5.3 Reinforcement Learning and Preference Optimization

RL and preference optimization support policy-level recursive improvement when evaluation signals change the data used in later optimization rounds. From a data-centric perspective, this process contains two linked pipelines. The policy-training pipeline constructs preference pairs, collects rollouts, and refreshes them from successive policy checkpoints. Its data are defined both by their supervision form, such as a preference pair or verifier-labeled trajectory, and by the policy that generated their candidates. The evaluator-training pipeline instead constructs supervision for a reward model, critic, process reward model, or judge that will assess later policy data. RLHF (Ouyang et al., 2022), DPO (Rafailov et al., 2023), and GRPO (Shao et al., 2024) provide representative optimization mechanisms that consume these data objects. With previous Section 3.1 defining the underlying feedback forms, particularly preference comparisons, verifier labels, and learned-judge outputs, this section shifts the focus from what those signals represent to how they construct, label, and refresh data across policy updates.

5.3.1 Feedback-Guided Policy-Training Data Generation and Refresh

Preference-pair construction and revision. The data-centric question for preference optimization is not whether a method uses preference data, but whether feedback changes the construction rule or contents of the preference corpus across rounds. The mutable data usually take the form of chosen–rejected response pairs, sometimes augmented with localized corrections, counterfactual inputs, or task-specific annotations. A single DPO round on a fixed preference corpus may improve alignment, but it does not close a recursive loop because the preference data are not updated (Rafailov et al., 2023).

RLHF-V (Yu et al., 2024a) is a clear representative of recursive preference-data construction. Segment-level human corrections identify localized visual errors, convert them into corrective preference pairs, and return those pairs to preference optimization. The recursive object is therefore the preference-pair corpus, not only the policy parameters. LLaVA-RLHF (Sun et al., 2024) similarly maps response comparisons and factual augmentation into policy updates through a reward model, while mDPO (Wang et al., 2024a) uses image-conditioned comparisons to improve text–image alignment. These methods differ in the source of contrast, but they share the same data operation: feedback defines which responses should be paired and how the preference relation should be interpreted.

Preference-data recursion can also be policy-conditioned. On-policy LVLM alignment (Yu et al., 2026) samples hallucinated responses from the current policy, annotates them with preferences, and uses the resulting pairs to optimize that same policy. After the policy update, the next candidate pool is sampled from a changed model, so the preference corpus evolves with the policy. TangoFlux (Hung et al., 2026) provides a related example in audio generation. It repeatedly generates audio candidates, ranks them with a fixed CLAP evaluator, and rebuilds preference pairs for subsequent optimization. Because CLAP remains fixed, the recursive update occurs in the policy-training data rather than in the evaluator-training data.

The main risk is recursive preference misspecification. If constructed pairs encode annotator bias or superficial proxies such as style, verbosity, or formatting, later rounds can amplify these shortcuts and reduce coverage of rare failure modes. Preference-data recursion therefore requires explicit criteria, counterfactual pairs, coverage across failure types, held-out agreement checks, and provenance linking each comparison to its evidence and evaluator.

Verifier-labeled rollout generation, selection, and weighting. A second policy-training pathway uses verifier feedback to select or weight policy-generated trajectories. The mutable object is a trajectory or rollout group, and the feedback may come from an exact-match answer, task metric, executable checker, or rule-based verifier. In this pathway, the verifier is treated as a fixed feedback source. Recursion arises only when its outputs change which rollouts enter the policy update or how strongly they are weighted. A verifier used only for final reporting remains an ordinary evaluation mechanism.

Table 4: Representative execution-layer mechanisms in RL, on-policy distillation, and context or memory. The final column distinguishes standard on-policy or cross-interaction adaptation from methods that revise future data construction or management. OPD and memory adaptation do not by themselves imply a strict recursive loop.

Method	Stage	Data operation	Updated object	Feedback signal	Update mechanism	Adaptation depth / limit
Self-Rewarding LMs (Yuan et al., 2024a)	RL	Self-rewarded preferences	Responses and preference pairs	Model-as-judge scores	Regenerates judged pairs and applies iterative DPO	Iterative data–policy loop
On-policy LVLM (Yu et al., 2026)	RL	Preference refresh	Current-policy responses and pairs	Hallucination labels	Resamples, relabels, reweights, and optimizes each round	Iterative on-policy refresh
RLAIF-V (Yu et al., 2025c)	RL	Iterative preference construction	AI-labeled response pairs	Claim-level self-feedback	Rebuilds preference data for iterative DPO	Iterative feedback–data loop
Visual-RFT (Liu et al., 2025e)	RL	Verifier-scored rollouts	Current-policy rollout groups	Accuracy or IoU	Scores rollout groups and updates the policy with GRPO	Standard on-policy update
eva (Ye et al., 2025b)	RL	Prompt evolution	Training prompts and responses	Creator–solver reward	Prioritizes and generates prompts for later policy updates	Prompt–policy co-adaptation
Ornith-1.5 (Ornith Team, 2026)	RL	Joint data generation	Tasks, scaffolds, and rollouts	Validity, difficulty, novelty, and success	Regenerates and jointly optimizes all three with GRPO	Bounded meta-level loop
GKD (Agarwal et al., 2024)	OPD	Student-rollout generation	Student sequences and teacher targets	Teacher token distributions	Resamples student states for generalized distillation	On-policy adaptation with fixed collection rule
AsyncOPD (Kang et al., 2026b)	OPD	Stale-rollout correction	Cached rollouts and teacher scores	Learner-time KL and cache support	Corrects old-policy data under the current student	Stale-data correction with fixed updater
TrOPD (Xing et al., 2026)	OPD	Feedback validation	Student trajectories and outlier regions	Teacher reliability under distribution shift	Masks, clips, or redirects unreliable supervision	Current-round validation with fixed rule
OmniOPD (Zhou et al., 2026c)	OPD	Selective supervision	High-uncertainty chunks	Teacher responses and semantic similarity	Audits uncertain forks and distills without logits	Selective supervision with fixed scheduler
OPCD (Ye et al., 2026b)	OPD	Context-conditioned feedback	Student rollouts and privileged context	Context-conditioned teacher distributions	Distills contextual knowledge on student states	On-policy adaptation with fixed context
Flux-OPD (Wang et al., 2026f)	OPD	Evolving context	Student rollouts and contexts	Context difference and conflict	Revises contextual correction and later supervision	Cross-round context refresh
Reflexion (Shinn et al., 2023)	Mem.	Reflection memory	Verbal reflections	Environment outcomes	Writes failure lessons for later trials	Across-trial accumulation with fixed writer
AgentRR (Feng et al., 2025)	Mem.	Verified replay	Structured experience traces	Task outcome and replay checks	Abstracts, verifies, and reuses traces on later tasks	Cross-task replay with fixed replay rule
Voyager (Wang et al., 2024b)	Mem.	Skill accumulation	Executable skill library	Execution and self-verification	Stores verified skills for later composition	Skill accumulation with fixed manager
SelfMem (Yang et al., 2026c)	Mem.	Strategy refinement	Memory strategy	Downstream task feedback	Explores and revises storage and retrieval behavior	Strategy-level adaptation
EvolveMem (Liu et al., 2026b)	Mem.	Retrieval co-evolution	Knowledge and retrieval configuration	Failures, regression, and stagnation	Diagnoses failures, revises configuration, and rolls back	Retrieval-policy co-evolution
Frontis-MA1 (Yang et al., 2026a)	Mem.	Experience reuse	Program nodes, cards, and task board	Sandbox outcome, progress, and novelty	Selects parents, retrieves traces, and promotes verified data	Evaluated reuse with fixed selection weights

Visual-RFT (Liu et al., 2025e) is a representative case. It turns classification accuracy or IoU from a reporting metric into feedback for scored perception trajectories. MM-Eureka (Meng et al., 2025) provides a related example for image reasoning, where rule-based feedback is applied to current-policy rollouts. In both cases, the verifier does not merely measure performance; it changes the effective training set by filtering or weighting policy-generated data.

When verifier-scored rollouts are collected on policy, the loop becomes deeper because the policy that generates the data is also updated. GRPO alternates rollout collection with policy optimization, so each checkpoint changes the generation distribution for the next batch (Shao et al., 2024). The recursive object is therefore not only the selected rollout set, but also the data-generating policy. Ornith-1.5 (Ornith Team, 2026) serves as a representative model that incorporates such process into model post-training. It generates tasks, task-specific scaffolds, and solution rollouts jointly, and optimizes all three with GRPO using task validity, frontier difficulty, novelty, scaffold alignment, reward fidelity, hack resistance, and rollout success. Because frontier difficulty depends on current-policy performance and stronger policies generate the tasks and scaffolds used in later rounds, evaluation refreshes both the rollout distribution and part of its generation configuration. However, the outer reward definitions, environment, and GRPO update rule remain fixed, so the method represents bounded meta-level self-improvement rather than open-ended self-referential RSI. This creates a data-freshness requirement: rollout labels and policy versions should be tracked so that updates are based on data generated by a compatible policy.

The main risk is verifier narrowing. Because verifier labels decide which rollouts are reinforced, the policy may overfit measurable signals while neglecting unmeasured qualities such as grounding, consistency, plausibility, or safety. This risk grows when the verifier is treated as a complete proxy for task quality.

5.3.2 Learned Evaluator Data Construction and Updating

The learned-evaluator pathway updates the supervision used to train a reward model, critic, process reward model, or judge. Its mutable object is a preference annotation, rubric, or intermediate-step label, not the rollout set itself. Unlike verifier-guided rollout selection, which changes rollout inclusion or weights, this pathway changes the evaluator that will later label, rank, or weight policy data. It is recursive only when evaluation revises the evaluator's training data, rubric dimensions, or supervision structure; a fixed evaluator used only for filtering belongs to rollout selection or rejection sampling.

Omni-RRM (Kong et al., 2026) is a clear closed-loop example. It synthesizes rubric-grounded omni-modal preference supervision, trains an evaluator on that supervision, and uses the evaluator to assess later policy data. The recursive object is therefore not only the policy-data pool, but also the evaluator-training corpus that defines future judgments. VisionReward (Xu et al., 2026a) provides complementary evidence for learned-evaluator construction by building a multidimensional reward model from fine-grained human judgments over image and video candidates.

Evaluator supervision can also move from whole-output judgments to structured intermediate labels. Visual-PRM (Wang et al., 2025c) trains process-level supervision for visual reasoning. This changes the granularity of the feedback object: the unit of supervision becomes an intermediate reasoning step rather than a final response. If these labels are updated using later model failures, the evaluator can reshape which reasoning errors are detected and corrected in subsequent policy training.

The main risks are axis collapse and evaluator entanglement. Aggregate rewards may hide underrepresented quality dimensions, and evaluators trained on same-family model outputs or judgments can become self-confirming. Recursive use therefore requires calibration, external validation, disagreement checks, and clear provenance for evaluator supervision.

Overall, these pathways differ in recursive depth. Fixed preference data and a fixed reward model support only a one-shot policy update. Recursion begins when feedback changes later preference data, rollout selection or weights, evaluator supervision, or the policy that generates future candidates. A method approaches policy-level self-improvement only when feedback also changes the configuration of future improvement, such as what data to collect, which failures to target, which evaluator to trust, or which update operator to apply. Thus, we treat an RL method as co-evolutionary when evaluation feedback changes a concrete data

object, and as recursively improving when it also changes the rules governing future data collection, labeling, weighting, or optimization.

5.4 On-Policy Distillation from Teacher Feedback

On-policy distillation (OPD) supports teacher-mediated self-improvement by training on states visited by the current student and supervising those states with teacher feedback. Its data-centric feature is that the supervision data are mutable: as the student changes, the rollout-feedback records used for training also change. Each record links a student-visited state or prefix to the produced action or token, the teacher signal, and the decision about how that signal enters the update.

OPD differs from SFT in Section 5.2 because it does not rely only on a fixed instruction pool, and from RL in Section 5.3 because it usually provides denser feedback than scalar rewards or preferences. In this section, we ask whether teacher feedback only updates the student, changes the rollout-feedback records used in later updates, or also changes how future supervision is collected and constructed. We organize the discussion around two pipelines: the student-side rollout pipeline and the teacher-feedback pipeline, following the OPD survey (Song & Zheng, 2026) while focusing on mutable data objects rather than only feedback or loss types.

5.4.1 Student-Conditioned Rollout Generation and Validation

Generating supervision on current-student states. The student-side pipeline starts by sampling trajectories from the current student and attaching teacher feedback to the states or prefixes that the student actually visits. This is the basic data-centric operation in OPD: the supervised states are generated by the current student rather than drawn only from a fixed instruction pool.

MiniLLM (Gu et al., 2024) and GKD (Agarwal et al., 2024) instantiate this operation by combining student-sampled sequences with teacher token distributions and reverse-divergence updates. GKD can also mix on-policy and fixed-data distillation. These methods make the training data follow the changing student, but they do not by themselves establish recursive improvement because feedback does not revise the rule for collecting future supervision.

Correcting stale or unsupported rollout feedback. A rollout-feedback pair may become unreliable when it is stale, weakly supported by the teacher, or located in an outlying teacher–student region. The data-centric operation is to decide whether that pair should be used directly, corrected, downweighted, constrained, or excluded. This is the OPD form of the signal-validity issue discussed in Section 3.

AsyncOPD (Kang et al., 2026b) addresses staleness by treating cached rollouts as mutable records. Each record stores visited prefixes, sampled actions, behavior-policy information, teacher scores, and the student version. At update time, the learner recomputes the relevant quantities under the current student and applies old-to-current correction instead of treating the cached estimate as fixed. This controls temporal mismatch between rollout collection and student update, but it does not test teacher reliability on every visited state.

Trust Region OPD (Xing et al., 2026) addresses teacher–student mismatch. It distills only where teacher supervision is considered reliable and constrains updates so that outlier regions do not dominate learning. Together, these methods show record-level data co-evolution: evaluation changes how existing rollout-feedback pairs are used, but not the teacher's context or the policy for constructing future feedback.

5.4.2 Teacher-Feedback Construction and Adaptation

Selecting teacher access and supervision granularity. The teacher-feedback pipeline can change the feedback record even when on-policy sampling is fixed. It can choose which part of the rollout receives supervision and which teacher interface supplies the signal.

Existing methods instantiate this choice at different granularities. PRISM (Wang et al., 2026e) uses response-level supervision from a discriminator and does not require teacher logits. One-Token Rollout (Ming et al., 2025) applies feedback at a single supervised prefix. OmniOPD (Zhou et al., 2026c) selects uncertain chunks

and performs logit-free chunk-level distillation across models with different tokenizers. OPOD (Zhao et al., 2026c) routes student responses to specialist teachers and controls their influence separately.

These methods fit the data-centric view because they change how feedback records are constructed. They become recursive only if outcomes revise later teacher routing, supervision granularity, or teacher-influence policies. If those rules stay fixed, the methods adapt supervision but do not yet adapt the supervision policy itself.

Conditioning feedback on supervisory context. Teacher feedback can also be conditioned on context that the student does not see. In this setting, the context is part of the feedback record rather than part of the student's sampling input. If the context is fixed, it changes the teacher signal for the current update. If the context is selected or revised by feedback, it becomes part of the supervision policy.

On-Policy Context Distillation (OPCD) (Ye et al., 2026b) samples student trajectories without privileged context and evaluates each token with a teacher conditioned on historical solution traces or an optimized system prompt. It then uses reverse-KL distillation to transfer this contextual knowledge into the student. During training, however, the context for each input remains fixed. OPCD therefore adapts the student responses, but it does not revise the supervisory context itself.

Flux-OPD (Wang et al., 2026f) makes the context mutable. It extracts contexts from recent student behavior, compares context-conditioned and context-free teacher feedback, and downweights conflicting context corrections. Each round changes both the student rollouts and the context set used for later supervision. This is closer to recursive improvement because student behavior revises part of the future feedback-construction process, not only the next training examples.

Grounding feedback and assigning update credit. Dense teacher feedback must be grounded in task-relevant evidence. Vision-OPD (Yuan et al., 2026) does this by letting a crop-conditioned self-teacher observe an answer-critical region and provide token-level feedback for a student rollout from the full image. Visual-Advantage OPD (Liu et al., 2026d) goes further by comparing token scores under the original image and a visually degraded image, using the difference as an evidence-dependent update weight.

The same student-visited-state view extends beyond text prefixes to denoising states, video histories, and audio-conditioned rollouts (Huang et al., 2025c; Li et al., 2026c; Hu et al., 2026a; Cao et al., 2026). In the mechanisms reviewed here, the evidence signal mainly grounds or weights the current update rather than revising future evidence selection, teacher access, or student sampling.

5.4.3 When OPD Becomes Teacher-Mediated Recursive Improvement

The OPD loop is adaptive because the current student generates the rollouts used for the next update. It becomes record-level data co-evolution when evaluation changes how rollout-feedback records are corrected, filtered, or weighted. It approaches recursive self-improvement only when feedback also changes the future supervision policy, such as teacher routing, context construction, evidence selection, or supervision granularity.

This boundary is important for validation. Rollouts may become stale, teacher feedback may fail under teacher–student mismatch, contexts may conflict, and teacher-like fluency may improve without stronger use of task evidence. Freshness ablations, mismatch analysis, context-stability tests, evidence counterfactuals, and equal-compute comparisons with SFT or RL are therefore needed. OPD should not be counted as recursive improvement merely because it uses teacher feedback; the feedback must change how future supervision is collected or constructed.

5.5 Context and Memory as Data-Centric Adaptation

Context and memory support *experience-level self-improvement* by changing the evidence and prior behavior available to future interactions, even when model weights remain fixed. Their mutable data includes retrieved evidence, runtime context, persistent records, replayable traces, and reusable skills compiled from validated experience. The signals defined in Section 3, including relevance, grounding, environment outcomes, fresh-

ness, contradiction, and compression fidelity, govern how these objects are selected, compressed, written, revised, retained, replayed, compiled into skills, promoted, or deleted. We organize these interventions into two linked paths. The fast, non-parametric path selects and adapts runtime evidence and context, updates persistent memory, and verifies and reuses prior experience without changing model weights. The slower, weight-updating path promotes verified memory into SFT or RL data. Under our action-based boundary, either path is co-evolutionary only when evaluation changes the data available to a later interaction or training round, and it approaches recursive self-improvement when feedback also revises the strategy governing that change.

5.5.1 Fast Non-Parametric Experience Adaptation

Formulating memory through selection, compression, and persistent updates. The fast path first determines which evidence enters the runtime context and which information persists across interactions. Its mutable objects include retrieval rankings, over-budget contexts, and stored records, while recall, attribution, relevance, freshness, contradiction, task utility, and compression fidelity govern query reformulation, reranking, retention, merging, summarization, consolidation, revision, and deletion. These quantities are evaluation signals of the kind defined in Section 3.1, while the policy mapping them to later memory actions becomes adaptive orchestration only when feedback revises that policy. RAG (Lewis et al., 2020), RETRO (Borgeaud et al., 2022), VisRAG (Yu et al., 2025b), and ColPali (Faysse et al., 2025) establish retrieval over external or structured evidence but do not themselves close a co-evolutionary loop because answer evaluation does not change later retrieval. Iterative RAG updates accumulated evidence within an interaction according to retrieval relevance, answer quality, and attribution (Chen et al., 2026c), and MIRAGE takes a related step for visual understanding by iterating retrieval and answer revision, although such methods need not revise the retriever for future tasks (Asadi et al., 2026). As evidence accumulates, context systems manage its growth through sparse video memory in MovieChat (Song et al., 2024), online memory banks in MA-LMM (He et al., 2024), and tool-queryable state in VideoAgent (Fan et al., 2024c), while evaluation-guided compression methods select or merge evidence under task-quality and resource constraints, as in visual context compression (Chen et al., 2024c), VideoChat-Flash (Li et al., 2026e), and AVOC (Chen et al., 2026d). Persistent-memory systems extend this management across interactions by moving records between stores in MemGPT (Packer et al., 2023), consolidating interactions in Mem0 (Chhikara et al., 2025), organizing linked notes in A-Mem (Xu et al., 2025), or reconstructing explicit state in ReaSMoRy (He et al., 2026). Whether these mechanisms preserve useful experience without excessive context growth is evaluated over extended or dynamic interactions through retention and temporal consistency in LoCoMo (Maharana et al., 2024), long-memory retrieval in LongMemEval (Wu et al., 2025a), evolving-memory quality in EvoMemory (Wei et al., 2025), beyond-million-token stress tests (Tavakoli et al., 2026), and stored-record quality, downstream utility, and error propagation in memory-management evaluation (Xiong et al., 2026).

Using memory during inference. Once formulated, memory affects later behavior by retrieving and replaying evaluated trajectories, critiques, workflows, distilled lessons, or executable skills. Reflexion converts task experience into verbal reflections (Shinn et al., 2023), ExpeL distills reusable knowledge from experience (Zhao et al., 2024), Agent Record & Replay verifies and replays prior traces under replay-time validity (Feng et al., 2025), and M^2 compresses and retrieves prior traces according to abstraction quality or task relevance (Yan et al., 2026a). Skills extend this mechanism from recalling experience to retrieving procedures that directly structure future behavior. Voyager stores self-verified executable programs (Wang et al., 2024b), MUSE-Autoskill refines retrieved skills using unit tests and runtime feedback (Lin et al., 2026), and SAGE uses a skill-integrated reward to guide skill generation and reuse (Wang et al., 2025a). These methods accumulate system-level capability without necessarily changing model weights, and they become co-evolutionary when replay outcomes revise which experiences or skills are retained, updated, downweighted, or deleted.

Adapting the memory-management system. A stronger loop changes not only memory content but also the strategy governing storage, retrieval, and reuse. Frontis-MA1/OpenMLE illustrates the boundary between content-level and strategy-level adaptation by converting sandbox outcomes into experience cards and a task-level board that guide parent selection and bounded operation-conditioned context construc-

tion (Yang et al., 2026a). However, it retains every deterministic card and applies fixed selection weights, providing stronger evidence for evaluation-guided memory use than for co-evolution of the memory policy itself. SelfMem moves beyond fixed management rules by allowing an agent to evaluate and refine its memory strategy through memory tools and conversational feedback (Yang et al., 2026c). EvolveMem makes this adaptation more structured by exposing retrieval configuration as an action space and using failure diagnosis, regression-triggered reversion, and stagnation-triggered exploration to co-evolve stored knowledge and retrieval (Liu et al., 2026b). BigBang broadens the adaptation target from memory management to the overall improvement strategy. Its generator harness records the motivation, outcome, failure cause, and conclusion of each data-synthesis experiment, while critic and held-out evaluation feedback revise the synthesis program and evaluation criteria in later rounds (PolarSeeker, 2026). These records therefore support adaptive orchestration of the improvement strategy rather than direct learning of the memory policy, consistent with Section 4.6. Across these systems, learned deletion, contradiction-aware consolidation, versioned rollback, and transfer-aware skill deprecation remain underexplored despite their importance for preventing stale, conflicting, or harmful experience from accumulating.

5.5.2 Slow Weight-Updating Adaptation through Memory Promotion

Constructing training data from verified experience. Unlike the fast path, which changes the data used in later interactions without necessarily updating model weights, the slow path promotes verified runtime records into SFT or RL data, making the candidate trace pool and promoted training subset its mutable objects. Each candidate trace retains the observation or evidence, model action, tool state, outcome, and provenance needed for audit. An evaluator checks its utility, grounding, freshness, and robustness under replay or counterfactual tests, after which admission and normalization convert it into an SFT instruction or demonstration, as discussed in Section 5.2, or an RL preference pair or scored rollout, as discussed in Section 5.3. After training, held-out evaluation determines whether the promoted data improved the intended capability and whether the source traces or derived examples should be retained, reweighted, or removed in the next round, coupling the fast interaction-memory loop to a slower cycle of data construction, weight updates, and newly generated experience. Several works instantiate this promotion pathway at different levels of experience granularity. STaR records self-generated rationales and promotes those that yield correct answers into iterative SFT data (Zelikman et al., 2022). ReST applies the same generate-evaluate-promote principle to offline policy improvement by reusing policy-generated and reward-evaluated responses (Gulcehre et al., 2023). In executable software engineering, SWE-Gym uses unit tests to select successful agent trajectories for iterative rejection-sampling fine-tuning (Pan et al., 2025). Agent-R complements success-only selection by converting erroneous interactive trajectories into reflection data, identifying the first error and connecting it to a corrective search branch (Yuan et al., 2025a). Frontis-MA1 broadens this pathway to long-horizon program search by converting sandbox-verified trajectories into execution-grounded SFT and RL data (Yang et al., 2026a).

Evaluating memory-derived training corpora. Memory promotion reintroduces the signal contamination, judge bias, and attribution risks discussed in Section 3. Evaluator independence is crucial because a plausible error can become both memory and supervision when the same model generates, approves, stores, and trains on a trace. Safeguards include replay-time verification, as illustrated by Agent Record & Replay (Feng et al., 2025), as well as executable task outcomes, human review, separately calibrated verifiers, and held-out grounded evaluation. Each derived training example should also preserve a provenance link to its source trace and supporting evidence. Beyond evaluator contamination, memory-derived corpora can inherit stale or irrelevant records, lossy removal of decisive evidence, and invalid transfer of earlier experience. Independent auditing and traceable provenance instantiate the signal-independence safeguard discussed in Section 4.4 and determine whether promoted experience provides reliable supervision across stages. Although this promotion pathway remains less mature than runtime retrieval and replay, it connects memory adaptation to data-evaluation co-evolution.

Table 5 summarizes representative methods under the proposed Evaluation–Orchestration–Execution taxonomy, coding each work by the feedback signals it consumes, the orchestration decisions it makes, and the mutable data-related objects it updates.

Table 5: Representative data–evaluation co-evolution methods positioned along the Evaluation–Orchestration–Execution taxonomy. A filled bullet (•) marks dimensions where the method, as described in this survey, actively uses feedback to change a later data-related state; an open circle (○) marks dimensions that are absent, fixed, or used mainly for validation rather than loop control.

	Evaluation signal					Orchestration decision				Execution target				
Method	**Score**	**Failure**	**Verifier**	**Pref./Judge**	**Env.**	**Signal**	**Attrib.**	**Action**	**Gate**	**PT**	**SFT**	**RL/Pref.**	**Eval. data**	**Memory**
DataComp (Gadre et al., 2023)	•	○	○	○	○	○	○	•	○	•	○	○	○	○
GREATS (Wang et al., 2024c)	•	○	○	○	○	•	○	•	○	•	○	○	○	○
ICONS (Wu et al., 2024)	•	○	○	○	○	•	○	•	○	○	•	○	○	○
Ultra-FineWeb (Wang et al., 2025g)	•	○	•	○	○	•	○	•	•	•	○	○	○	○
MATES (Yu et al., 2024b)	•	○	○	○	○	•	○	•	○	•	○	○	○	○
DataOrchestra (Huang et al., 2026)	○	○	•	•	○	•	•	•	•	•	○	○	○	○
DataMaster (Du et al., 2026)	•	•	○	•	○	•	•	•	•	•	•	○	○	•
DataEvolver (Deng et al., 2026)	○	•	•	○	○	•	•	•	•	•	○	○	○	•
DataEvolve (Mi et al., 2025)	•	•	○	•	○	•	•	•	•	•	○	○	○	•
BigBang (PolarSeeker, 2026)	•	•	•	•	○	•	•	•	•	○	•	○	•	•
Frontis-MA1 (Yang et al., 2026a)	•	•	•	○	•	•	•	•	•	○	•	•	○	•
Multimodal DataEvolver (Yan et al., 2026b)	○	•	•	•	○	•	•	•	•	○	•	○	○	•
VisionFoundry (Zhou et al., 2026a)	○	○	•	○	○	•	○	•	•	○	•	○	○	○
Self-Evolving Visual Questioner (Liang et al., 2026)	○	•	•	•	○	•	•	•	•	○	•	○	○	○
Visual-RFT (Liu et al., 2025e)	•	○	•	○	○	•	○	•	•	○	○	•	○	○
MM-Eureka (Meng et al., 2025)	•	○	•	○	○	•	○	•	•	○	○	•	○	○
RLHF-V (Yu et al., 2024a)	○	•	○	•	○	•	•	•	•	○	○	•	•	○
RLAIF-V (Yu et al., 2025c)	○	•	○	•	○	•	•	•	•	○	○	•	•	○
Omni-RRM (Kong et al., 2026)	○	○	○	•	○	•	○	•	•	○	○	•	•	○
AsyncOPD (Kang et al., 2026b)	•	○	○	•	○	•	○	•	•	○	○	○	○	•
Flux-OPD (Wang et al., 2026f)	•	○	○	•	○	•	○	•	•	○	○	○	○	•
SelfMem (Yang et al., 2026c)	•	•	○	•	•	•	•	•	•	○	○	○	○	•
EvolveMem (Liu et al., 2026b)	•	•	○	•	•	•	•	•	•	○	○	○	○	•
SWE-Gym (Pan et al., 2025)	•	•	•	○	•	•	•	•	•	○	•	•	○	•
Agent-R (Yuan et al., 2025a)	•	•	•	•	•	•	•	•	•	○	•	•	○	•
Voyager (Wang et al., 2024b)	○	•	•	○	•	•	•	•	•	○	○	○	○	•
MUSE-Autoskill (Lin et al., 2026)	○	•	•	○	•	•	•	•	•	○	○	○	○	•

Abbreviations. Score: benchmark, downstream, or validation score. Failure: failure cluster or error diagnosis. Verifier: rule, tool, execution, IoU, exact-match, or fixed-evaluator signal. Pref./Judge: preference, reward-model, learned-judge, critique, or rationale signal. Env.: environment outcome or executable interaction result. Signal: signal selection. Attrib.: error attribution. Action: target, operator, allocation, or budget decision. Gate: acceptance, stopping, rollback, or promotion gate. PT: pre-training. RL/Pref.: reinforcement learning or preference optimization. Eval. data: evaluator-supervision data. Memory: runtime context, persistent memory, experience records, or skill stores. The table codes functional roles as described in this survey rather than implementation details. A method may contain additional components; a filled bullet is used only when the component participates in the feedback-to-update path emphasized here.

6 Failure Modes of Data–Evaluation Co-Evolution

A higher score is not sufficient evidence that a data–evaluation loop has improved. The score may measure the wrong capability, reuse information seen by the loop, hide unstable updates, overlook lost data support, or combine changes that cannot be attributed to one component. We use five requirements: signal validity and coverage, signal independence, update stability, data-support integrity, and system identifiability. Reproducible traces are evidence needed to establish system identifiability. These requirements are our synthesis of the surveyed evidence. They are separate diagnostic questions rather than claims that every recursive-improvement system must fail.

6.1 Invalid or Incomplete Feedback Signals

A feedback signal is invalid when it does not measure the capability that the loop is intended to improve. Limited target-space coverage creates the same problem because an accurate score on a narrow test may not represent the full target distribution. Repeated updates can then improve the measured proxy while leaving the intended capability unchanged. Multimodal evaluation makes this gap especially important: wide task coverage, low cost, and low variance are difficult to achieve together (Zhang et al., 2025c).

Grounding failure is a pointwise form of invalidity. A policy may answer through a language shortcut instead of using the relevant image, audio, video, document, spatial state, or action signal. The evaluator may miss the same shortcut. Visual encoders can overlook fine detail (Tong et al., 2024b), and strong vision–language models still fail perceptual tasks designed to resist memorized language patterns (Rahmanzadehgervi et al., 2024). Task-specific evidence also shows that automatic evaluators can remain biased when text and images provide different information (Liu et al., 2025c).

Aggregate reliability does not resolve this problem. A judge can return repeatable scores while remaining insensitive to required evidence. Because each judgment can decide which example enters the next update,

pointwise validity matters more than leaderboard consistency alone. The unresolved boundary is whether the signal responds to the intended capability across the relevant domains and modalities.

6.2 Dependent or Exposed Feedback Signals

A signal can measure the right capability and still fail to provide independent evidence. This occurs when evaluation examples have influenced training, when the same benchmark is reused for repeated selection, or when the generator and evaluator share correlated preferences. In each case, the loop can improve against information that is already part of the update process.

Contamination is difficult to rule out. If a model has seen only the training split, a train–test perplexity contrast may reveal exposure. If it has seen both splits, both values may fall together and resemble a clean model (Xu et al., 2024c). Other probability- and membership-based detectors also depend on assumptions that can fail across domains and training regimes (Fu et al., 2025b). Release-filtered and dynamic benchmarks reduce some exposure channels, but they can have smaller samples and higher measurement variance (Jain et al., 2025).

Generator–evaluator dependence creates a related problem without direct item leakage. Evaluators can recognize and favor outputs from their own model family (Panickssery et al., 2024). Self-preference is not proof that the selected output is worse, and current evidence does not isolate it as the cause of every downstream regression. The limitation is that a gain measured by a related evaluator may not persist under independent examples, judges, or external outcomes.

6.3 Unstable Updates and Non-Monotonic Progress

Update instability occurs when an accepted or deployed state regresses or oscillates under a fixed, independent evaluation. Variation among candidates that were never accepted is normal search behavior and is not itself instability. The risk arises when discrete changes to prompts, memories, workflows, code, or data are accepted using textual critique or noisy judge scores that do not provide a reliable update direction. Intrinsic self-critique can reduce performance where sound external verification improves it (Stechly et al., 2025), and self-refinement can strengthen a model's preference for its own outputs without improving the intended metric (Xu et al., 2024d).

The balance between corrected and introduced errors provides one direct stability test. A recent preprint models this balance with an error-correction rate and an error-introduction rate. Several tested models degraded or oscillated when correction did not offset newly introduced errors (Liu & Meng, 2026). This result is task- and model-specific, but it shows why an update that fixes some failures may still reduce total performance.

Search reports must separate candidate exploration from accepted-state performance. The Darwin Gödel Machine finds strong coding agents through lineages that sometimes include lower-scoring descendants (Zhang et al., 2026). These lower-scoring descendants are useful exploration, not evidence that the deployed system regressed. The example shows why candidate scores and the best stored score should be reported separately. A recent self-evolving-agent preprint reports that an unguarded context update can regress, while a disjoint held-out gate can reject the measured regression (Nguyen et al., 2026). Such gates protect the selection split; they do not guarantee true out-of-distribution improvement.

6.4 Data-Support Loss under Recursive Generation

Data-support loss occurs when the update policy repeatedly removes rare but relevant parts of the training distribution. Recursive generation can copy model errors and narrow the range of later training data (Shumailov et al., 2024). Average reward or fluency may rise while rare classes, styles, failure cases, or out-of-distribution behaviors disappear. This loss can remain hidden behind a higher task score. Iterative SFT, DPO, and mixed SFT–DPO can raise pass@1 while diversity and out-of-distribution performance decline (Wu et al., 2025b). Maximum-likelihood sharpening can concentrate probability mass without adding knowledge (Huang et al., 2025a), and curated feedback can improve expected reward while variance falls

(Ferbach et al., 2024). Variance collapse is therefore one possible symptom of support loss, not a common cause of all five requirements.

The outcome depends on how generated data is used. Replacing real data with recursively generated data collapsed in analyzed settings, while retaining real data across rounds avoided collapse in the corresponding accumulation setting (Gerstgrasser et al., 2024). Asymptotic analyses retain a more pessimistic boundary: repeated recursion can distort the learned distribution even when each round contains only a small synthetic fraction (Dohmatob et al., 2025).

The same support question applies across modalities. If text examples are easier to generate, verify, or optimize, a mixed data policy may expand text supervision faster than image, audio, video, spatial, or action supervision. Cross-modal recursion may also behave differently when one modality is regenerated and another remains fixed (Hu et al., 2025). Current evidence does not establish a universally safe real-to-synthetic ratio or modality mixture. The boundary depends on the generator, filtering policy, model, domain, accumulation rule, and number of rounds.

6.5 Unattributable and Irreproducible System Changes

The behavior of a co-evolution system is jointly determined by the base model, controller, judge, prompts, tools, external APIs, and environment state. Reports that retain only the final score cannot identify which component or interaction caused a change. System identifiability requires localizing that effect through controlled component changes and replay. This is component attribution: it asks which part of the running system produced the change. It differs from training-data attribution, which asks which examples caused a model behavior and which data intervention may repair it.

Judge biases are well documented (Zheng et al., 2023), but multi-agent systems still lack a detailed account of why their control processes fail (Cemri et al., 2025). Automated component attribution is only beginning to be studied (Zhang et al., 2025d). Reconstructing an execution path after only the final output remains partial and costly (Nian et al., 2026).

Replayable state is a prerequisite for this attribution. The record must include model and API versions, prompts, tool schemas, retrieval state, rejected candidates, and intermediate controller decisions. Otherwise a component swap may change both the target component and hidden execution conditions. Stochastic inference adds another source of variation across steps. One recent preprint reports that tool sequences are more repeatable than their argument values and that much behavioral divergence begins early in the pipeline (Yagubyan, 2026). This evidence is preliminary, but it shows why a final model, dataset, or score is insufficient for replay. The unresolved limitation is whether a reported gain can be localized to a component or interaction under a controlled, replayable configuration.

7 Future Directions

Section 6 identified five requirements that a measured gain can fail to satisfy. This section presents five corresponding research programs. Some programs directly address current failures. Others develop the multimodal data, feedback, and control mechanisms needed for future co-evolution systems. The common goal is to establish when a loop produces reliable improvement rather than a higher score on its own update signal.

7.1 Build Valid, Coverage-Aware Multimodal Signals

Any signal that selects data or changes a model should be validated at the level where it is used. Aggregate model rankings are not enough because a single incorrect judgment can change which example enters the next update. Evaluation should therefore test consistency, target coverage, grounding, and uncertainty separately. A repeatable judge can still measure the wrong capability, and one score can hide conflicting changes in factuality, visual quality, safety, or temporal coherence.

Multimodal signals require direct grounding tests. A judge may reward a fluent answer without using the relevant image, audio, video, or document. Current multimodal judges vary across tasks and can miss changes in hallucination, spatial reasoning, factual grounding, and visual fidelity (Norman et al., 2026; Kumar et al., 2026; Khan et al., 2026). Recent reward benchmarks cover multimodal and interleaved inputs, but process-level validation remains limited (Yasunaga et al., 2025; Hu et al., 2026d).

Evidence ablation provides one practical test. The evaluator can be run again after the required modality or premise is masked. If its judgment does not change, the signal needs further validation before it is used for training (Khan et al., 2026; Khanmohammadi et al., 2026). Premise-level checks can also record which visual facts support a reward (Wang et al., 2026c). Repeated scoring can estimate variance. Low-confidence judgments should retain their uncertainty instead of being converted directly into training labels.

Future evaluators should also report separate dimensions rather than one aggregate score. For generation, these dimensions may include prompt alignment, factuality, visual quality, identity, physical plausibility, safety, and temporal coherence. For embodied tasks, they may include outcome correctness, use of the required modality, valid tool use, and recovery after failure. Each dimension should be checked against human judgments and held-out domains.

Unified understanding–generation models may help construct these tests. An editing model can change one visual fact and create a counterfactual pair for an evaluator. Understanding errors can also become generation tasks, and generated counterexamples can be used to test understanding (Yang et al., 2025a; Tong et al., 2026; Zhang et al., 2025b; Liu et al., 2026c). These counterfactuals should then pass the independent-validation controls in Section 7.2.

7.2 Establish and Scale Independent Feedback

A valid signal is not sufficient if it has already influenced training or shares the model's preferences. Future loops need feedback that is separated from data generation, update selection, or model training. This separation can be provided by private examples, unrelated judges, executable rules, human review, or outcomes from an external environment.

Dynamic and private evaluation can reduce repeated exposure. Release-date filtering (Jain et al., 2025), continuous refresh (White et al., 2025), and generated instances (Zhu et al., 2024) reduce direct reuse of fixed test items (Chen et al., 2025b). They also introduce costs. Release filtering can reduce sample size, and generated tests can share assumptions with the model being evaluated. Reports should therefore state who generated each test, who verified it, which components share training data, and how much uncertainty remains.

Independent evaluators provide another check. Cross-family judges, tools, and humans may reduce correlated errors, but model family alone does not establish independence. Evaluation should measure agreement and error correlation on held-out examples. Evaluator disagreement should be recorded and handled by the escalation policy in Section 7.3.

Rule-verifiable tasks provide machine-checkable feedback. A curriculum agent and executor can co-evolve from no external task data (Xia et al., 2025), and a generator and verifier can co-evolve to reduce repeated consensus errors (Pan et al., 2026). Scoring rules and test cases can also change with the system they evaluate (Li et al., 2026d; Lee & Kahng, 2025). These methods are most useful when the verifier is based on an external rule, program, or environment transition rather than another unconstrained model judgment.

Real interaction provides a different source of evidence. People, deployed software, and the physical world can supply corrections, interruptions, preferences, task outcomes, environment transitions, and recovery attempts. Multimodal interaction models and embodied agents may lower the cost of collecting these signals (Luo et al., 2026; Guo et al., 2025b; Driess et al., 2023). Interactive benchmarks capture part of this process (Xie et al., 2024; Koh et al., 2024; Rawles et al., 2025), but they do not replace long-term feedback from real deployment.

Future collection systems should retain the context, outcome, and uncertainty of each interaction. They should report useful feedback per unit cost, human burden, privacy risk, safety controls, interaction diversity,

and transfer to new users and environments. Simulation can supplement this evidence, but it should not be reported as independent real-world feedback.

7.3 Measure and Control Update Stability

Current work does not provide a reliable rule for choosing the number of feedback rounds or rollouts per round. The canonical self-rewarding method reports three iterations and leaves longer runs open (Yuan et al., 2024a). The solver–verifier capability gap can limit self-improvement (Sun et al., 2026), reward-filtered fine-tuning can improve and then saturate (Liu et al., 2026a), and weak external signal can increase variance (Zenil, 2026). These results identify relevant variables, but they do not provide a general scaling law.

7.3.1 Track Trajectories

Future studies should report the full update trajectory. Each round should include task performance, proposed candidates, accepted states, the best state found so far, rollout count, verifier acceptance rate, reversals, rollbacks, and cost. This separates actual round behavior from a best-score curve that is non-decreasing by construction. Support and out-of-distribution metrics should be reported separately under the data-support program in Section 7.4.

Stability should be measured directly. For refinement, reports can compare the number of corrected errors with the number of new errors (Liu & Meng, 2026). For search, they can report candidate rejection, reversal, and rollback rates. The same candidates should be evaluated under stable criteria across rounds so that changes in the judge are not mistaken for changes in the model.

7.3.2 Treat Indistinguishable States as Ties

Near a benchmark ceiling, small score differences require special care. Repeated evaluation, confidence intervals, and paired held-out tests can show whether two states are distinguishable. If the difference is within evaluation noise, the controller should treat the result as a tie. The loop should then stop, collect more evidence, or move to a harder independent test.

7.3.3 Predefine Escalation and Rollback

Human escalation and rollback should use explicit triggers. Useful triggers include high evaluator uncertainty, disagreement between independent judges, repeated reversals, falling diversity, held-out regression, weak transfer, or a safety alarm. Model judges and human reviewers both have systematic errors (Goel et al., 2025; Recchia et al., 2025). Debate can help in some settings, but it can also optimize for judge preference (Lang et al., 2025; Carro et al., 2025). The escalation policy should therefore state which signal caused the handoff and what evidence the human or stronger evaluator must check.

7.4 Scale Data while Preserving Support

Sample count alone does not define data scale. The update policy should preserve rare capabilities, difficult cases, modalities, and out-of-distribution behavior across rounds. This requires direct comparison of replacement, accumulation, and curation policies over several real-to-synthetic ratios.

Fresh real data remains important. Accumulating real and generated data can behave differently from replacing real data with generated data (Gerstgrasser et al., 2024). Provenance records should identify the generator, source, round, filtering rule, and whether a sample was accumulated or replaced. Current provenance infrastructure remains incomplete (Longpre et al., 2024b;a). The real-to-synthetic ratio should be reported as both a data policy and a cost.

Several mechanisms can increase on-policy multimodal data. Multimodal reinforcement learning with verifiable rewards (RLVR) uses rule-checked tasks, as shown by TRON for visual reasoning and VideoRLVR for spatial and temporal trajectories (Yang et al., 2026d; Zhu et al., 2026c). Executable artifacts provide another check: Video2Code converts interaction videos into webpages that can be tested on state transitions (Xu et al., 2026b).

Generated and learned environments can provide resettable state and parallel rollouts. Agent World Model and ScaleEnv generate tool-use environments (Wang et al., 2026i; Tu et al., 2026), while VLAW combines real-robot data, a video world model, synthetic rollouts, and policy updates (Guo et al., 2026). These routes differ in realism and verification, but they share one unresolved question: whether more generated experience preserves capability, modality, tail, and out-of-distribution coverage.

Environment selection matters as much as environment count. Adding more multimodal environment types can cause negative transfer, while ability-aware selection and curricula can perform better than training on the full pool (Zhu et al., 2026b). Reports should include capability coverage, difficulty, temporal horizon, modality use, accepted verified rollouts per attempt, verifier confidence, and held-out transfer. They should also record whether the agent exploited the verifier or ignored a required modality (Roth et al., 2026; Chen et al., 2026a).

Table 6: Recommended reporting fields for a data–evaluation co-evolution loop.

Field	What to record	Purpose
Evaluation signal	Source, modality, version, verifier type, uncertainty, and validity checks	Shows what information drives the loop
Updated object	Data, model stage, rollout group, memory, tool, workflow, or environment	Locates the intervention
Controller policy	Selection rules, prompts, thresholds, and schedule	Makes update decisions reproducible
Candidate history	Proposed, accepted, rejected, and rolled-back candidates with reasons	Separates search history from the final state
Provenance and support	Generator, real-to-synthetic ratio, accumulation policy, diversity, tails, and modality balance	Records how the data distribution changes
Human involvement	Trigger, task, judgment type, decision, and cost	Supports escalation and oversight claims
Rounds and stopping	Round count, monitored values, halt rule, and rollback state	Shows why the loop continued or stopped
Cost	Task generation, rollouts, verifiers, humans, and fresh data	Enables fixed-budget comparison
Transfer tests	Held-out tasks, users, environments, and safety checks	Tests whether the update generalizes

7.5 Localize and Reproduce System Changes

7.5.1 Localize the System Change

A loop should first identify which running component or interaction caused a change. Studies can swap or ablate the model, controller, judge, prompts, tools, APIs, and environment, then test whether the same effect appears during replay. Automated component attribution and trace reconstruction provide initial tools for this task (Zhang et al., 2025d; Nian et al., 2026). The result should reproduce under a matched execution state before it is used to guide a repair.

7.5.2 Choose a Repair After Localization

Training-data attribution can then ask which examples contributed to the behavior and which data change may repair it. Text methods can select data based on predicted task effects (Engstrom et al., 2024) or identify harmful examples before repair (Jain et al., 2024). Influence methods scale to language models, although their identified examples are not always reliable (Grosse et al., 2023; Li et al., 2025e). Fixed, off-policy targets may not represent errors produced after an update. On-policy attribution can reduce this mismatch (Wang et al., 2026g). The repair can target a data or training stage, or a runtime component such as memory, tools, workflow, or environment. Stage-selection policies should be compared with fixed-stage baselines under the same diagnosed failure. A state-distribution view can compare offline SFT with RL and on-policy distillation on states produced by the current learner (Nie, 2026), while freshness controls can reduce updates from stale rollouts (Chen et al., 2026b). The repair is useful only if it improves held-out behavior without reducing retention or safety.

7.5.3 Compare Under Controlled Conditions

Controlled testbeds are needed to compare these choices. They should allow planned component changes while recording model versions, prompts, data, evaluators, tools, rounds, and budgets. Every loop should also be compared with a compute-matched static baseline. The baseline should spend the same budget without repeated feedback, for example on more seed data, more epochs, one generation batch, or one-shot distillation. This comparison tests whether iteration adds value. A fixed-budget comparison should include task generation, policy rollouts, verifier execution, human judgment, and fresh external data. It should report performance, transfer, coverage, wall-clock time, and component cost.

7.5.4 Keep a Replayable Record

Logs should include generated, accepted, rejected, and rolled-back candidates; evaluator outputs; prompts; tool calls; component versions; environment state; human actions; costs; and stopping decisions. These records support the study of orchestration failures and runtime governance (Cemri et al., 2025; Kaptein et al., 2026). The system should publish the monitored value, threshold, action, and rollback state for every stop rule.

7.5.5 Cross-Program Reporting

The five research programs above require a shared reporting protocol. A recursive-improvement claim should not report only the final score; it should also record the feedback signals, updated objects, controller decisions, candidate history, data provenance, human involvement, stopping rules, costs, and transfer tests that define the loop. Table 6 gives a recommended reporting template.

8 Conclusion

Data-centric recursive improvement marks a shift in foundation model development from static data construction and post-hoc evaluation toward closed-loop systems in which evaluation actively shapes future data and model behavior. As foundation models are increasingly improved through synthetic data, preference feedback, verifier signals, retrieval, memory, and agent-environment interaction, understanding how evaluation signals are converted into data-centric interventions has become essential. In this survey, we provide a structured review of this trend through the lens of data–evaluation co-evolution. We organize existing work around a signal-decision-update loop: what signal diagnoses the current system, who or what decides how to act on that signal, and what data-related object is updated to affect the next round. This perspective connects methods across pre-training, supervised fine-tuning, preference optimization, on-policy distillation, context adaptation, and memory-based systems. We further highlight the risks that arise when evaluation becomes part of the improvement loop, including unreliable feedback, feedback overfitting, unstable updates, data-support loss, and irreproducible system changes. Addressing these challenges will require more grounded feedback signals, independent audits, stable update mechanisms, and reproducible records of data and control decisions. We hope this survey provides a foundation for researchers and practitioners to design, analyze, and advance more adaptive, robust, and accountable recursive foundation model improvement method that is a potential key to ASI.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024.

Syeda Nahida Akter, Shrimai Prabhumoye, John Kamalu, Sanjeev Satheesh, Eric Nyberg, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Mind: Math informed synthetic dialogues for pretraining llms. In *International Conference on Learning Representations*, 2025.

Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL `https://arxiv.org/abs/2312.02406`.

Vinayak Arannil, Neha Narwal, Sourav Sanjukta Bhabesh, Sai Nikhil Thirandas, Darren Yow-Bang Wang, Graham Horwood, Alex Anto Chirayath, and Gouri Pandeshwar. Dopamine: Domain-specific pre-training adaptation from seed-guided data mining, 2024. URL https://arxiv.org/abs/2410.00260.

Mohammad Asadi, Jack W. O'Sullivan, Fang Cao, Tahoura Nedaee, Kamyar Rajabalifardi, Fei-Fei Li, Ehsan Adeli, and Euan Ashley. Mirage: The illusion of visual understanding, 2026. URL https://arxiv.org/abs/2603.21687.

Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Baolong Bi, Shenghua Liu, Xingzhang Ren, Dayiheng Liu, Junyang Lin, Yiwei Wang, Lingrui Mei, Junfeng Fang, Jiafeng Guo, and Xueqi Cheng. Refinex: Learning to refine pre-training data at scale from expert-guided programs, 2025. URL http://arxiv.org/abs/2507.03253v2.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, pp. 2206–2240, 2022.

Alexander Bukharin, Shiyang Li, Zhengyang Wang, Jingfeng Yang, Bing Yin, Xian Li, Chao Zhang, Tuo Zhao, and Haoming Jiang. Data diversity matters for robust instruction tuning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 3411–3425. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.195.

Di Cao, Dongjie Fu, Hai Yu, Siqi Zheng, Xu Tan, and Tao Jin. X-OPD: Cross-modal on-policy distillation for capability alignment in speech LLMs. *arXiv preprint arXiv:2603.24596*, 2026.

María Victoria Carro, Denise Alejandra Mester, Facundo Nieto, Oscar Agustín Stanchi, Guido Ernesto Bergman, Mario Alejandro Leiva, Eitan Sprejer, Luca Nicolás Forziati Gangi, et al. Ai debaters are more persuasive when arguing in alignment with their own beliefs. *arXiv preprint arXiv:2510.13912*, 2025.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya G. Parameswaran, Dan Klein, Kannan Ramchandran, Matei A. Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail? In *Advances in Neural Information Processing Systems*, volume 38, 2025. Datasets and Benchmarks Track.

Daoyuan Chen, Yilun Huang, Zhijian Ma, Hesen Chen, Xuchen Pan, Ce Ge, Dawei Gao, Yuexiang Xie, Zhaoyang Liu, Jinyang Gao, Yaliang Li, Bolin Ding, and Jingren Zhou. Data-juicer: A one-stop data processing system for large language models. In *Companion of the 2024 International Conference on Management of Data*, pp. 120–134. ACM, 2024a.

Dongping Chen, Ruoxi Chen, Shilin Zhang, Yaochen Wang, Yinuo Liu, Huichi Zhou, Qihui Zhang, Yao Wan, Pan Zhou, and Lichao Sun. Mllm-as-a-judge: Assessing multimodal llm-as-a-judge with vision-language benchmark. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 6562–6595, 2024b.

Dongping Chen, Xuanao Huang, Zhihan Hu, Qingyuan Shi, Dianqi Li, and Tianyi Zhou. Sandboxed coding agents are competitive omni-modal task solvers. *arXiv preprint arXiv:2606.00579*, 2026a. URL https://arxiv.org/abs/2606.00579.

Jieneng Chen, Luoxin Ye, Ju He, Zhao-Yang Wang, Daniel Khashabi, and Alan Yuille. Efficient large multi-modal models via visual context compression. In *Advances in Neural Information Processing Systems*, volume 37, 2024c.

Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In *Advances in Neural Information Processing Systems*, volume 37, 2024d.

Mayee F. Chen, Michael Y. Hu, Nicholas Lourie, Kyunghyun Cho, and Christopher Ré. Aioli: A unified optimization framework for language model data mixing. In *International Conference on Learning Representations*, 2025a.

Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, et al. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation. *arXiv preprint arXiv:2502.17521*, 2025b.

Xianwei Chen, Shimin Zhang, and Jibin Wu. f-opd: Stabilizing long-horizon on-policy distillation with freshness-aware control. *arXiv preprint arXiv:2605.17862*, 2026b.

Xupeng Chen, Binbin Shi, Chenqian Le, Jiaqi Zhang, Kewen Wang, Ran Gong, Jinhan Zhang, and Chihang Wang. Iterative multimodal retrieval-augmented generation for medical question answering. *arXiv preprint arXiv:2604.27724*, 2026c.

Yijing Chen, Wenhui Tan, Xiaoyi Yu, Yuyue Wang, Xin Cheng, Kaisi Guan, Hao Jiang, Xiangyang Li, Guojie Zhu, and Ruihua Song. Avoc: Enhancing hour-level audio-video understanding in omni-modal llms via retrieval-inspired token compression. *arXiv preprint arXiv:2606.24286*, 2026d.

Zhaorun Chen, Zichen Wen, Yichao Du, Yiyang Zhou, Chenhang Cui, Siwei Han, Jen Weng, Chaoqi Wang, Zhengwei Tong, Leria Huang, Canyu Chen, Haoqin Tu, Qinghao Ye, Zhihong Zhu, Yuqing Zhang, Jiawei Zhou, Zhuokai Zhao, Rafael Rafailov, Chelsea Finn, and Huaxiu Yao. Mj-bench: Is your multimodal reward model really a good judge for text-to-image generation? In *Advances in Neural Information Processing Systems*, volume 38, 2025c. doi: 10.52202/085713-2081. Datasets and Benchmarks Track.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. In *ECAI 2025 - 28th European Conference on Artificial Intelligence*, pp. 2993–3000, 2025. doi: 10.3233/FAIA251160.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael I. Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating llms by human preference. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 8359–8388, 2024.

Chenhao Dang, Jing Ma, and Mingjie Liao. Holistic data scheduler for llm pre-training via multi-objective reinforcement learning. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, pp. 176–187. ACM, April 2026. doi: 10.1145/3770854.3780325. URL `http://dx.doi.org/10.1145/3770854.3780325`.

Chao Deng, Shaolei Zhang, Ju Fan, and Xiaoyong Du. Dataevolver: Automatic data preparation for large language models through multi-level self-evolving, 2026. URL `https://arxiv.org/abs/2606.07001`.

Shijian Deng, Kai Wang, Tianyu Yang, Harsh Singh, and Yapeng Tian. Self-improvement in multimodal large language models: A survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 1987–2006. Association for Computational Linguistics, 2025.

Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. Nemotron-CLIMB: Clustering-based iterative data mixture bootstrapping for language model pre-training. In *Advances in Neural Information Processing Systems*, volume 38, 2025. Datasets and Benchmarks Track.

Elvis Dohmatob, Yunzhen Feng, Arjun Subramonian, and Julia Kempe. Strong model collapse. In *International Conference on Learning Representations*, 2025.

Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 8469–8488, 2023. URL https://proceedings.mlr.press/v202/driess23a.html.

Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, Xinyu Zhu, Wenhao Wang, Linfeng Zhang, Chen Qian, and Siheng Chen. Datamaster: Data-centric autonomous ai research. *arXiv preprint arXiv:2605.10906*, 2026.

Logan Engstrom, Axel Feldmann, and Aleksander Madry. Dsdm: Model-aware dataset selection with datamodels. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 12491–12526, 2024.

Simin Fan, David Grangier, and Pierre Ablin. Dynamic gradient alignment for online data mixing, 2024a. URL https://arxiv.org/abs/2410.02498.

Simin Fan, Matteo Pagliardini, and Martin Jaggi. DOGE: Domain reweighting with generalization estimation. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 12895–12915, 2024b.

Simin Fan, Maria Ios Glarou, and Martin Jaggi. Grape: Optimize data mixture for group robust multi-target adaptive pretraining. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/e7bba023351f8262c22e3328e42645ec-Abstract-Conference.html.

Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *Computer Vision – ECCV 2024*, volume 22, pp. 75–92. Springer, 2024c.

Alex Fang, Albin Madappally Jose, Amit Jain, Ludwig Schmidt, Alexander Toshev, and Vaishaal Shankar. Data filtering networks. In *International Conference on Learning Representations*, 2024.

Matteo Farina, Vishaal Udandarao, Thao Nguyen, Selim Kuzucu, Maximilian Böther, Andreas Hochlehnert, Adhiraj Ghosh, Marianna Nezhurina, Karsten Roth, Joschka Struber, Yuhui Zhang, Sebastian Dziadzio, Elaine Sui, Soumya Jahagirdar, Dhruba Ghosh, Hasan Hammoud, Thomas De Min, Simone Caldarella, Jehanzeb Mirza, Sedrick Keh, Mehdi Cherti, Hilde Kuehne, Bernt Schiele, Serena Yeung-Levy, Muhammad Ferjad Naeem, Federico Tombari, Ana Klimovic, Elisa Ricci, Matthias Bethge, Sewoong Oh, Ameya Prabhu, Alessio Tonioni, Jenia Jitsev, Massimiliano Mancini, Ludwig Schmidt, and Nikhil Parthasarathy. Datacomp-vlm: Improved open datasets for vision-language models, 2026. URL https://arxiv.org/abs/2606.28551.

Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations*, 2025.

Erhu Feng, Wenbo Zhou, Zibin Liu, Le Chen, Yunpeng Dong, Cheng Zhang, Yisheng Zhao, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. Get experience from practice: LLM agents with record & replay. *arXiv preprint arXiv:2505.17716*, 2025.

Damien Ferbach, Quentin Bertrand, Avishek Joey Bose, and Gauthier Gidel. Self-consuming generative models with curated data provably optimize human preferences. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Ling Fu, Zhebin Kuang, Jiajun Song, Mingxin Huang, Biao Yang, Yuzhe Li, Linghao Zhu, Qidi Luo, Xinyu Wang, Hao Lu, Zhang Li, Guozhi Tang, Bin Shan, Chunhui Lin, Qi Liu, Binghong Wu, Hao Feng, Hao Liu, Can Huang, Jingqun Tang, Wei Chen, Lianwen Jin, Yuliang Liu, and Xiang Bai. Ocrbench v2: An

improved benchmark for evaluating large multimodal models on visual text localization and reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pp. 107930–107981, 2025a. doi: 10.52202/085713-3253. Datasets and Benchmarks Track.

Yujuan Velvin Fu, Özlem Uzuner, Meliha Yetişgen, and Fei Xia. Does data contamination detection work (well) for llms? a survey and evaluation on detection assumptions. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 5235–5256. Association for Computational Linguistics, 2025b.

Kazuki Fujii, Yukito Tajima, Sakae Mizuki, Masaki Kawamura, Hinari Shimada, Taihei Shiotani, Koshiro Saito, Masanari Oi, Taishi Nakamura, Takumi Okamoto, Shigeki Ishida, Kakeru Hattori, Youmi Ma, Hiroya Takamura, Rio Yokota, Jun Sakuma, and Naoaki Okazaki. Rewriting pre-training data boosts llm performance in math and code. In *International Conference on Learning Representations*, 2026.

Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruba Ghosh, Jieyu Zhang, Eyal Orgad, Rahim Entezari, Giannis Daras, Sarah M. Pratt, Vivek Ramanujan, Yonatan Bitton, Kalyani Marathe, Stephen Mussmann, Richard Vencu, Mehdi Cherti, Ranjay Krishna, Pang Wei Koh, Olga Saukh, Alexander Ratner, Shuran Song, Hannaneh Hajishirzi, Ali Farhadi, Romain Beaumont, Sewoong Oh, Alex Dimakis, Jenia Jitsev, Yair Carmon, Vaishaal Shankar, and Ludwig Schmidt. Datacomp: In search of the next generation of multimodal datasets. In *Advances in Neural Information Processing Systems*, volume 36, 2023. Datasets and Benchmarks Track.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *Transactions on Machine Learning Research*, 2026. URL `https://mlanthology.org/tmlr/2026/gao2026tmlr-survey/`.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 10835–10866, 2023.

Xin Gao, Xiaoyang Wang, Yun Zhu, Mengzhang Cai, Conghui He, and Lijun Wu. Closing the data loop: Using opendataarena to engineer superior training datasets. *arXiv preprint arXiv:2601.09733*, 2025.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, et al. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Tomasz Korbak, Henry Sleight, Rajashree Agrawal, John Hughes, Dhruv Bhandarkar Pai, Andrey Gromov, Dan Roberts, Diyi Yang, David L. Donoho, and Sanmi Koyejo. Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data. In *Proceedings of the First Conference on Language Modeling*, 2024.

Sara Ghazanfari, Francesco Croce, Nicolas Flammarion, Prashanth Krishnamurthy, Farshad Khorrami, and Siddharth Garg. Chain-of-frames: Advancing video understanding in multimodal llms via frame-aware reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2746–2755, 2026. URL `https://openaccess.thecvf.com/content/CVPR2026/html/Ghazanfari_Chain-of-Frames_Advancing_Video_Understanding_in_Multimodal_LLMs_via_Frame-Aware_Reasoning_CVPR_2026_paper.html`.

Dhruba Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. Geneval: An object-focused framework for evaluating text-to-image alignment. *Advances in Neural Information Processing Systems*, 36:52132–52152, 2023.

Shashwat Goel, Joschka Strüber, Ilze Amanda Auzina, Karuna K. Chandra, Ponnurangam Kumaraguru, Douwe Kiela, Ameya Prabhu, Matthias Bethge, and Jonas Geiping. Great models think alike and this undermines ai oversight. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Roger Grosse, Juhan Bae, Cem Anil, Nelson Elhage, Alex Tamkin, Amirhossein Tajdini, Benoit Steiner, Dustin Li, et al. Studying large language model generalization with influence functions. *arXiv preprint arXiv:2308.03296*, 2023.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.

Yuxian Gu, Li Dong, Hongning Wang, Yaru Hao, Qingxiu Dong, Furu Wei, and Minlie Huang. Data selection via optimal control for language models. In *International Conference on Learning Representations*, 2025.

Caglar Gulcehre et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645:633–638, 2025a. doi: 10.1038/s41586-025-09422-z. URL `https://doi.org/10.1038/s41586-025-09422-z`.

Qingpei Guo, Kaiyou Song, Zipeng Feng, Ziping Ma, Qinglong Zhang, Sirui Gao, Xuzheng Yu, Yunxiao Sun, Tai-Wei Chang, Jingdong Chen, et al. M2-omni: Advancing omni-mllm for comprehensive modality support with competitive performance. *arXiv preprint arXiv:2502.18778*, 2025b.

Yanjiang Guo, Tony Lee, Lucy Xiaoyang Shi, Jianyu Chen, Percy Liang, and Chelsea Finn. VLAW: Iterative co-improvement of vision-language-action policy and world model. *arXiv preprint arXiv:2602.12063*, 2026. URL `https://arxiv.org/abs/2602.12063`.

Bo He, Hengduo Li, Young Kyun Jang, Menglin Jia, Xuefei Cao, Ashish Shah, Abhinav Shrivastava, and Ser-Nam Lim. Ma-lmm: Memory-augmented large multimodal model for long-term video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Jixuan He, Xueting Li, Chieh Hubert Lin, and Ming-Hsuan Yang. Reasmory: 3d reconstruction as explicit memory for vlms spatial reasoning. *arXiv preprint arXiv:2606.00963*, 2026.

Jing Hu, Danxiang Zhu, Xianlong Luo, Dan Zhang, Shuwei He, Yishu Lei, Haitao Zheng, Shikun Feng, Jingzhou He, Yu Sun, Hua Wu, and Haifeng Wang. CORD: Bridging the audio-text reasoning gap via weighted on-policy cross-modal distillation. *arXiv preprint arXiv:2601.16547*, 2026a.

Michael Y. Hu, Apurva Gandhi, Kyunghyun Cho, Tal Linzen, and Pratyusha Sharma. Always learning, always mixing: Efficient and simple data mixing all the time, 2026b. URL `https://arxiv.org/abs/2605.15220`.

Rongsheng Hu, Runwei Guan, Yicheng Di, Jiayu Bao, and Yuan Liu. Autovqa-g: Self-improving agentic framework for automated visual question answering and grounding annotation. *arXiv preprint arXiv:2604.17488*, 2026c.

Yushi Hu, Reyhane Askari-Hemmat, Melissa Hall, Emily Dinan, Luke Zettlemoyer, and Marjan Ghazvininejad. Multimodal rewardbench 2: Evaluating omni reward models for interleaved text and image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 36904–36915, 2026d. URL `https://openaccess.thecvf.com/content/CVPR2026/html/Hu_Multimodal_RewardBench_2_Evaluating_Omni_Reward_Models_for_Interleaved_Text_CVPR_2026_paper.html`.

Zizhao Hu, Mohammad Rostami, and Jesse Thomason. Multimodal synthetic data finetuning and model collapse: Insights from vlms and diffusion models. In *Proceedings of the 27th ACM International Conference on Multimodal Interaction (ICMI)*. Association for Computing Machinery, 2025.

Audrey Huang, Adam Block, Dylan J. Foster, Dhruv Rohatgi, Cyril Zhang, Max Simchowitz, Jordan T. Ash, and Akshay Krishnamurthy. Self-improvement in language models: The sharpening mechanism. In *International Conference on Learning Representations*, 2025a.

Han Huang, Yuqi Huo, Zijia Zhao, Haoyu Lu, Shu Wu, Bingning Wang, Qiang Liu, Weipeng Chen, and Liang Wang. Beyond filtering: Adaptive image-text quality enhancement for mllm pretraining, 2024. URL https://arxiv.org/abs/2410.16166.

Hongzhe Huang, Jiang Liu, Zhewen Yu, Li Cai, Dian Jiao, Wenqiao Zhang, Siliang Tang, Juncheng Li, Hao Jiang, Haoyuan Li, and Yueting Zhuang. Align2llava: Cascaded human and large language model preference alignment for multi-modal instruction curation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 8759–8781. Association for Computational Linguistics, 2025b. doi: 10.18653/v1/2025.findings-acl.458. URL http://dx.doi.org/10.18653/v1/2025.findings-acl.458.

Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *Advances in Neural Information Processing Systems*, 2025c.

Zhen Huang, Yikun Wang, Shijie Xia, and Pengfei Liu. Dataorchestra: Learning to orchestrate per-example curation of pretraining data, 2026. URL https://arxiv.org/abs/2607.24717.

Chenyu Hui. Towards robust long-context understanding of large language model via active recap learning, 2026. URL https://arxiv.org/abs/2601.13734.

Chia-Yu Hung, Navonil Majumder, Zhifeng Kong, Ambuj Mehrish, Amir Zadeh, Chuan Li, Rafael Valle, Bryan Catanzaro, and Soujanya Poria. Tangoflux: Super fast and faithful text to audio generation with flow matching and clap-ranked preference optimization. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/f475c4cf43826411ae61c94bab360a0d-Abstract-Conference.html.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025.

Saachi Jain, Kimia Hamidieh, Kristian Georgiev, Andrew Ilyas, Marzyeh Ghassemi, and Aleksander Madry. Data debiasing with datamodels (d3m): Improving subgroup robustness via data selection. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Dongzhi Jiang, Ziyu Guo, Renrui Zhang, Zhuofan Zong, Hao Li, Le Zhuo, Shilin Yan, Pheng-Ann Heng, and Hongsheng Li. T2i-r1: Reinforcing image generation with collaborative semantic-level and token-level cot. In *Advances in Neural Information Processing Systems*, volume 38, 2025a. URL https://papers.neurips.cc/paper_files/paper/2025/hash/38fc6254f73450813db3b3e04397a9fc-Abstract-Conference.html.

Yiding Jiang, Allan Zhou, Zhili Feng, Sadhika Malladi, and J. Zico Kolter. Adaptive data optimization: Dynamic sample selection with scaling laws. In *International Conference on Learning Representations*, 2025b.

Zhengbo Jiao, Zifan Zhang, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, and Linfeng Zhang. Socratic-geo: Synthetic data generation and cross-modal geometric reasoning via multi-agent interaction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 23795–23804, 2026.

Linjia Kang, Zhimin Wang, Yongkang Zhang, Duo Wu, Jinghe Wang, Ming Ma, Haopeng Yan, and Zhi Wang. Learning with challenges: Adaptive difficulty-aware data generation for mobile gui agent training, 2026a. URL https://arxiv.org/abs/2601.22781.

Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjun Kang, Sanghyun Park, Donghoon Kim, Minjae Lee, Minseo Kim, Rishabh Tiwari, Yuchen Zeng, Hyung Il Koo, and Kangwook Lee. Asyncopd: How stale can on-policy distillation be? *arXiv preprint arXiv:2606.24143*, 2026b.

Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime governance for ai agents: Policies on paths. *arXiv preprint arXiv:2603.16586*, 2026.

Mohammed Safi Ur Rahman Khan, Sanjay Suryanarayanan, Tushar Anand, and Mitesh M. Khapra. Seeing isn't believing: Uncovering blind spots in evaluator vision-language models. *arXiv preprint arXiv:2604.21523*, 2026.

Reza Khanmohammadi, Erfan Miahi, Simerjot Kaur, Charese H. Smiley, Ivan Brugere, Kundan Thind, and Mohammad M. Ghassemi. Grounded or guessing? lvlm confidence estimation via blind-image contrastive ranking. *arXiv preprint arXiv:2605.10893*, 2026.

Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in nlp. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4110–4124. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.324.

Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. *Advances in Neural Information Processing Systems*, 36:36652–36663, 2023.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024.

Jovana Kondic, Pengyuan Li, Dhiraj Joshi, Zexue He, Shafiq Abedin, Jennifer Sun, Ben Wiesel, Eli Schwartz, Ahmed Nassar, Bo Wu, Assaf Arbelle, Aude Oliva, Dan Gutfreund, Leonid Karlinsky, and Rogerio Feris. Chartgen: Scaling chart understanding via code-guided synthetic chart generation, 2025. URL `https://arxiv.org/abs/2507.19492`.

Zicheng Kong, Dehua Ma, Zhenbo Xu, Alven Yang, Yiwei Ru, Haoran Wang, Zixuan Zhou, Fuqing Bie, Liuyu Xiang, Huijia Wu, Jian Zhao, and Zhaofeng He. Omni-rrm: Advancing omni reward modeling via automatic rubric-grounded preference synthesis, 2026. URL `https://arxiv.org/abs/2602.00846`.

Divake Kumar, Sina Tayebati, Devashri Naik, Ranganath Krishnan, and Amit Ranjan Trivedi. Vlm judges can rank but cannot score: Task-dependent uncertainty in multimodal evaluation. *arXiv preprint arXiv:2604.25235*, 2026.

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, Lester James V. Miranda, Bill Yuchen Lin, Khyathi Raghavi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. Rewardbench: Evaluating reward models for language modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 1755–1797. Association for Computational Linguistics, 2025.

Hao Lang, Fei Huang, and Yongbin Li. Debate helps weak-to-strong generalization. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, 2025.

Jaewoo Lee, Boyang Li, and Sung Ju Hwang. Concept-skill transferability-based data selection for large vision-language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 5060–5080. Association for Computational Linguistics, 2024a.

Jaeyoung Lee, Ximing Lu, Jack Hessel, Faeze Brahman, Youngjae Yu, Yonatan Bisk, Yejin Choi, and Saadia Gabriel. How to train your fact verifier: Knowledge transfer with multimodal open models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 13060–13077. Association for Computational Linguistics, 2024b.

Minjae Lee and Minsuk Kahng. Data-prompt co-evolution: Growing test sets to refine llm behavior. *arXiv preprint arXiv:2510.12728*, 2025.

Seongyun Lee, Seungone Kim, Sue Hyun Park, Geewook Kim, and Minjoon Seo. Prometheus-vision: Vision-language model as a judge for fine-grained evaluation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 11286–11315. Association for Computational Linguistics, 2024c.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020.

Baiqi Li, Zhiqiu Lin, Deepak Pathak, Jiayao Li, Yixin Fei, Kewen Wu, Tiffany Ling, Xide Xia, Pengchuan Zhang, Graham Neubig, et al. Genai-bench: Evaluating and improving compositional text-to-visual generation. *arXiv preprint arXiv:2406.13743*, 2024a.

Baiqi Li, Zhiqiu Lin, Wenxuan Peng, Jean de Dieu Nyandwi, Daniel Jiang, Zixian Ma, Simran Khanuja, Ranjay Krishna, Graham Neubig, and Deva Ramanan. Naturalbench: Evaluating vision-language models on natural adversarial samples. In *Advances in Neural Information Processing Systems*, volume 37, 2024b. Datasets and Benchmarks Track.

Dawei Li, Renliang Sun, Yue Huang, Ming Zhong, Bohan Jiang, Jiawei Han, Xiangliang Zhang, Wei Wang, and Huan Liu. Preference leakage: A contamination problem in llm-as-a-judge. In *International Conference on Learning Representations*, 2026a.

Haoling Li, Kai Zheng, Jie Wu, Can Xu, Qingfeng Sun, Han Hu, and Yujiu Yang. Verievol: Scaling multimodal mathematical reasoning via verifiable evol-instruct, 2026b. URL `https://arxiv.org/abs/2606.23543`.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruba Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. DataComp-LM: In search of the next generation of training sets for language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024c. doi: 10.52202/079017-0455. Datasets and Benchmarks Track.

Jian Li, Weiheng Lu, Hao Fei, Meng Luo, Ming Dai, Min Xia, Yizhang Jin, Zhenye Gan, Ding Qi, Chaoyou Fu, et al. A survey on benchmarks of multimodal large language models. *arXiv preprint arXiv:2408.08632*, 2024d.

Lei Li, Zhihui Xie, Mukai Li, Shunian Chen, Peiyi Wang, Liang Chen, Yazheng Yang, Benyou Wang, and Lingpeng Kong. Silkie: Preference distillation for large visual language models. *arXiv preprint arXiv:2312.10665*, 2023.

Lei Li, Yuancheng Wei, Zhihui Xie, Xuqing Yang, Yifan Song, Peiyi Wang, Chenxin An, Tianyu Liu, Sujian Li, Bill Yuchen Lin, Lingpeng Kong, and Qi Liu. Vl-rewardbench: A challenging benchmark for vision-language generative reward models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 24657–24668, 2025a. URL `https://openaccess.thecvf.com/content/CVPR2025/html/Li_VL-RewardBench_A_Challenging_Benchmark_for_Vision-Language_Generative_Reward_Models_CVPR_2025_paper.html`.

Quanhao Li, Junqiu Yu, Kaixun Jiang, Yujie Wei, Zhen Xing, Pandeng Li, Ruihang Chu, Shiwei Zhang, Yu Liu, and Zuxuan Wu. DiffusionOPD: A unified perspective of on-policy distillation in diffusion models. *arXiv preprint arXiv:2605.15055*, 2026c.

Shuyue Stella Li, Rui Xin, Yike Wang, Teng Xiao, Rulin Shao, Zoey Hao, Melanie Sclar, Faeze Brahman, Pang Wei Koh, and Yulia Tsvetkov. Evolm: Self-evolving language models through co-evolved discriminative rubrics. In *International Conference on Learning Representations*, 2026d.

Xiang Lisa Li, Farzaan Kaiyom, Evan Zheran Liu, Yifan Mai, Percy Liang, and Tatsunori Hashimoto. Autobencher: Towards declarative benchmark construction. In *International Conference on Learning Representations*, 2025b.

Xinhao Li, Yi Wang, Jiashuo Yu, Xiangyu Zeng, Yuhan Zhu, Haian Huang, Jianfei Gao, Kunchang Li, Yinan He, Chenting Wang, Yu Qiao, Yali Wang, and Limin Wang. Videochat-flash: Hierarchical compression for long-context video modeling. In *International Conference on Learning Representations*, 2026e.

Yi Li, Haonan Wang, Qixiang Zhang, Boyu Xiao, Chenchang Hu, Hualiang Wang, and Xiaomeng Li. Unieval: Unified holistic evaluation for unified multimodal understanding and generation. *arXiv preprint arXiv:2505.10483*, 2025c.

Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Pike: Adaptive data mixing for large-scale multi-task learning under low gradient conflicts, 2025d. URL `https://arxiv.org/abs/2502.06244`.

Zhe Li, Wei Zhao, Yige Li, and Jun Sun. Do influence functions work on large language models? In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 14367–14382. Association for Computational Linguistics, 2025e.

Yijun Liang, Hengguang Zhou, Ming Li, Lichen Li, Cho-Jui Hsieh, and Tianyi Zhou. Self-evolving visual questioner, 2026. URL `https://arxiv.org/abs/2606.13929`.

Hunter Lightman et al. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

Bin Lin, Yang Ye, Bin Zhu, Jiaxi Cui, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 5971–5984. Association for Computational Linguistics, 2024a. doi: 10.18653/v1/2024.emnlp-main.342.

Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation. *arXiv preprint arXiv:2605.27366*, 2026. URL `https://arxiv.org/abs/2605.27366`.

Weifeng Lin, Xinyu Wei, Ruichuan An, Peng Gao, Bocheng Zou, Yulin Luo, Siyuan Huang, Shanghang Zhang, and Hongsheng Li. Draw-and-understand: Leveraging visual prompts to enable mllms to comprehend what you want. In *International Conference on Learning Representations*, 2025.

Zhiqiu Lin, Deepak Pathak, Baiqi Li, Jiayao Li, Xide Xia, Graham Neubig, Pengchuan Zhang, and Deva Ramanan. Evaluating text-to-visual generation with image-to-text generation. In *Computer Vision – ECCV 2024*, pp. 366–384. Springer, 2024b.

Aofan Liu and Jingxiang Meng. Self-correction as feedback control: Error dynamics, stability thresholds, and prompt interventions in LLMs. *arXiv preprint arXiv:2604.22273*, 2026. URL `https://arxiv.org/abs/2604.22273`.

Chenruo Liu, Yijun Dong, Yiqiu Shen, and Qi Lei. A task-centric theory for iterative self-improvement with easy-to-hard curricula. In *International Conference on Learning Representations*, 2026a.

Fuxiao Liu, Kevin Lin, Linjie Li, Jianfeng Wang, Yaser Yacoob, and Lijuan Wang. Mitigating hallucination in large multi-modal models via robust instruction tuning. In *International Conference on Learning Representations*, 2024a.

Jiaqi Liu, Xinyu Ye, Peng Xia, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Evolvemem: Self-evolving memory architecture via AutoResearch for LLM agents. *arXiv preprint arXiv:2605.13941*, 2026b.

Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-grpo: Training flow matching models via online rl. In *Advances in Neural Information Processing Systems*, volume 38, pp. 45650–45685, 2025a. doi: 10.52202/085713-1362. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9ed5598827c66866837632a2d0d8af1c-Abstract-Conference.html.

Jinyu Liu, Xincheng Shuai, Henghui Ding, and Yu-Gang Jiang. Unison: Benchmarking unified multimodal models via synergistic understanding and generation. *arXiv preprint arXiv:2606.26984*, 2026c.

Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. Regmix: Data mixture as regression for language model pre-training. In *International Conference on Learning Representations*, 2025b.

Ruiqi Liu, Xiaolei Lv, Gengsheng Li, Ximo Zhu, Zhiheng Wang, Zhengbo Zhang, Junkai Chen, Zhiheng Li, Bo Li, Jun Gao, and Shu Wu. Visual-advantage on-policy distillation for vision-language models. *arXiv preprint arXiv:2605.21924*, 2026d.

Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and Junxian He. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. In *International Conference on Learning Representations*, 2024b.

Xiaoqun Liu, Jiacheng Liang, Muchao Ye, and Zhaohan Xi. Robustifying safety-aligned large language models through clean data curation, 2024c. URL https://arxiv.org/abs/2405.19358.

Yinhong Liu, Jianfeng He, Hang Su, Ruixue Lian, Yi Nian, Jake W. Vincent, Srikanth Vishnubhotla, Robinson Piramuthu, and Saab Mansour. MDSEval: A meta-evaluation benchmark for multimodal dialogue summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 14707–14727, 2025c. doi: 10.18653/v1/2025.findings-emnlp.794. URL https://aclanthology.org/2025.findings-emnlp.794/.

Zikang Liu, Kun Zhou, Wayne Xin Zhao, Dawei Gao, Yaliang Li, and Ji-Rong Wen. Less is more: High-value data selection for visual instruction tuning. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 3712–3721. Association for Computing Machinery, 2025d. doi: 10.1145/3746027.3755160. URL https://doi.org/10.1145/3746027.3755160.

Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-rft: Visual reinforcement fine-tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 2034–2044, 2025e.

Shayne Longpre, Robert Mahari, Anthony Chen, Naana Obeng-Marnu, Damien Sileo, William Brannon, Niklas Muennighoff, Nathan Khazam, Jad Kabbara, Kartik Perisetla, Xinyi Wu, Enrico Shippole, Kurt Bollacker, Tongshuang Wu, Luis Villa, Sandy Pentland, and Sara Hooker. A large-scale audit of dataset licensing and attribution in ai. *Nature Machine Intelligence*, 6(8):975–987, 2024a. doi: 10.1038/s42256-024-00878-8.

Shayne Longpre, Robert Mahari, Naana Obeng-Marnu, William Brannon, Tobin South, Katy Gero, Sandy Pentland, and Jad Kabbara. Data authenticity, consent, & provenance for ai are all broken: what will it take to fix them? *arXiv preprint arXiv:2404.12691*, 2024b.

Yujie Lu, Dongfu Jiang, Wenhu Chen, William Yang Wang, Yejin Choi, and Bill Yuchen Lin. Wildvision: Evaluating vision-language models in the wild with human preferences. In *Advances in Neural Information Processing Systems*, volume 37, 2024. Datasets and Benchmarks Track.

Run Luo, Xiaobo Xia, Lu Wang, Longze Chen, Renke Shan, Jing Luo, Min Yang, and Tat-Seng Chua. Next-omni: Towards any-to-any omnimodal foundation models with discrete flow matching. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/ee60f53717bd9c2abdcca66dfbec65da-Abstract-Conference.html.

Zheheng Luo, Xin Zhang, Xiao Liu, Haoling Li, Yeyun Gong, Qi Chen, and Peng Cheng. Velocitune: A velocity-based dynamic domain reweighting method for continual pre-training. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16644–16656. Association for Computational Linguistics, 2025.

Jing Ma, Chenhao Dang, and Mingjie Liao. Ac-odm: Actor–critic online data mixing for sample-efficient llm pretraining. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026a.

Yinghao Ma, Haiwen Xia, Hewei Gao, Weixiong Chen, Yuxin Ye, Yuchen Yang, Sungkyun Chang, Mingshuo Ding, Yizhi Li, Ruibin Yuan, Simon Dixon, and Emmanouil Benetos. Cmi-rewardbench: Evaluating music reward models with compositional multimodal instruction, 2026b. URL https://arxiv.org/abs/2603.00610.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024.

Pratyush Maini, Vineeth Dorna, Parth Doshi, Aldo Carranza, Fan Pan, Jack Urbanek, Paul Burstein, Alex Fang, Alvin Deng, Amro Abbas, Brett Larsen, Cody Blakeney, Charvi Bannur, Christina Baek, Darren Teh, David Schwab, Haakon Mongstad, Haoli Yin, Josh Wills, Kaleigh Mentzer, Luke Merrick, Ricardo Monti, Rishabh Adiga, Siddharth Joshi, Spandan Das, Zhengping Wang, Bogdan Gaza, Ari Morcos, and Matthew Leavitt. Beyondweb: Lessons from scaling synthetic data for trillion-scale pretraining, 2025. URL https://arxiv.org/abs/2508.10975.

Fanqing Meng, Lingxiao Du, Zongkai Liu, Zhixiang Zhou, Quanfeng Lu, Daocheng Fu, Tiancheng Han, Botian Shi, Wenhai Wang, Junjun He, et al. Mm-eureka: Exploring the frontiers of multimodal reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2503.07365*, 2025.

Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. RSIBench-Data: Benchmarking data-centric research for recursive self-improvement. *arXiv preprint arXiv:2607.25886*, 2026.

Tiantian Mi, Dongming Shan, Zhen Huang, Yiwei Qin, Muhang Xie, Yuxuan Qiao, Yixiu Liu, Chenyang Zhou, and Pengfei Liu. Data darwinism part ii: Dataevolve – ai can autonomously evolve pretraining data curation. *arXiv preprint arXiv:2603.14420*, 2025.

Rui Ming, Haoyuan Wu, Shoubo Hu, Zhuolun He, and Bei Yu. One-token rollout: Guiding supervised fine-tuning of llms with policy gradient. *arXiv preprint arXiv:2509.26313*, 2025.

David Mizrahi, Anders Boesen Lindbo Larsen, Jesse Allardice, Suzie Petryk, Yuri Gorokhov, Jeffrey Li, Alex Fang, Josh Gardner, Tom Gunter, and Afshin Dehghan. Language models improve when pretraining data matches target tasks, 2025. URL https://arxiv.org/abs/2507.12466.

Dung Nguyen, Minh Khoi Ho, Huy Ta, Thanh Tam Nguyen, Qi Chen, Kumar Rav, Quy Duong Dang, Satwik Ramchandre, Son Lam Phung, Zhibin Liao, Minh-Son To, Johan Verjans, Phi Le Nguyen, and Vu Minh Hieu Phan. Localizing before answering: A benchmark for grounded medical visual question answering. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pp. 7670–7678, 2025a. doi: 10.24963/ijcai.2025/853.

Michael Nguyen, Quoc Nguyen, and Paul Vuong. Recursive self-evolving agents via held-out selection. *arXiv preprint arXiv:2606.28374*, 2026. URL https://arxiv.org/abs/2606.28374.

Thao Nguyen, Yang Li, Olga Golovneva, Luke Zettlemoyer, Sewoong Oh, Ludwig Schmidt, and Xian Li. Recycling the web: A method to enhance pre-training data quality and quantity for language models. In *Conference on Language Modeling (COLM)*, 2025b. URL http://arxiv.org/abs/2506.04689v3.

Yi Nian, Haosen Cao, Shenzhe Zhu, Henry Peng Zou, Qingqing Luan, and Yue Zhao. When only the final text survives: Implicit execution tracing for multi-agent attribution. *arXiv preprint arXiv:2603.17445*, 2026.

Dong Nie. Post-training is about states, not tokens: A state distribution view of sft, rl, and on-policy distillation. *arXiv preprint arXiv:2605.22731*, 2026.

Joel Niklaus, Atsuki Yamaguchi, Michal Štefánik, Guilherme Penedo, Hynek Kydlíček, Elie Bakouch, Lewis Tunstall, Edward Emanuel Beeching, Thibaud Frere, Colin Raffel, Leandro von Werra, and Thomas Wolf. How can we synthesize high-quality pretraining data? a systematic study of prompt design, generator model, and source data, 2026. URL http://arxiv.org/abs/2604.13977v1.

Justin D. Norman, Michael U. Rivera, and D. Alex Hughes. Reliability without validity: A systematic, large-scale evaluation of llm-as-a-judge models across agreement, consistency, and bias. *arXiv preprint arXiv:2606.19544*, 2026.

Ornith Team. Ornith-1.5: From self-scaffolding to self-improvement. Ornith Blog, August 2026. URL https://ornith.ai/ornith_1_5.html.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Teng Pan, Yuchen Yan, Zixuan Wang, Ruiqing Zhang, Guiyang Hou, Wenqi Zhang, Weiming Lu, Jun Xiao, and Yongliang Shen. Coverrl: Breaking the consensus trap in label-free reasoning via generator-verifier co-evolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 29833–29853, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1376. URL https://aclanthology.org/2026.acl-long.1376/.

Artemis Panagopoulou, Le Xue, Ning Yu, Junnan Li, Dongxu Li, Shafiq Joty, Ran Xu, Silvio Savarese, Caiming Xiong, and Juan Carlos Niebles. X-instructblip: A framework for aligning image, 3d, audio, video to llms and its emergent cross-modal reasoning. In *European Conference on Computer Vision (ECCV)*, pp. 177–197. Springer, 2024. doi: 10.1007/978-3-031-72995-9_11. URL https://arxiv.org/abs/2311.18799.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. Llm evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Guilherme Penedo et al. Fineweb: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

PolarSeeker. BigBang: Pursuing open-ended intelligence through self-evolving synthesis of verifiable frontier tasks. Technical report, PolarSeeker, 2026. URL https://endlessfrontier.tech/assets/paper.pdf. Preprint. Technical Report.

Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, Jie Tang, and Yuxiao Dong. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025.

Zhenting Qi, Susanna Maria Baby, Stefanie Anna Baby, Kan Yuan, Andrew Tomkins, Tu Vu, Da-Cheng Juan, and Cyrus Rashtchian. On the generalization gap in self-evolving language model reasoning. *arXiv preprint arXiv:2606.01075*, 2026. ICML 2026.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 2023.

Pooyan Rahmanzadehgervi, Logan Bolton, Mohammad Reza Taesiri, and Anh Totti Nguyen. Vision language models are blind: Failing to translate detailed visual features into words. In *Asian Conference on Computer Vision (ACCV)*, 2024. URL `https://arxiv.org/abs/2407.06581`.

Chris Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, 2025.

Gabriel Recchia, Chatrik Singh Mangat, Jinu Nyachhyon, Mridul Sharma, Callum Canavan, Dylan Epstein-Gross, and Muhammed Abdulbari. Confirmation bias: A challenge for scalable oversight. *arXiv preprint arXiv:2507.19486*, 2025.

Amit Roth, Ankur Samanta, Matan Halevy, Yoav Levine, and Yonathan Efroni. Hack-verifiable environments: Towards evaluating reward hacking at scale. *arXiv preprint arXiv:2605.20744*, 2026.

Yangjun Ruan, Neil Band, Chris J. Maddison, and Tatsunori Hashimoto. Reasoning to learn from latent thoughts, 2025. URL `http://arxiv.org/abs/2503.18866v2`.

Bardia Safaei, Faizan Siddiqui, Jiacong Xu, Vishal M. Patel, and Shao-Yuan Lo. Filter images first, generate instructions later: Pre-instruction data selection for visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14247–14256, 2025. doi: 10.1109/CVPR52734.2025.01329. URL `https://arxiv.org/abs/2503.07591`.

Atharva Sehgal, Patrick Yuan, Ziniu Hu, Yisong Yue, Jennifer J. Sun, and Swarat Chaudhuri. Self-evolving visual concept library using vision-language critics. *arXiv preprint arXiv:2504.00185*, 2025. CVPR 2025.

Nimrod Shabtay, Felipe Maia Polo, Sivan Doveh, Wei Lin, Muhammad Jehanzeb Mirza, Leshem Choshen, Mikhail Yurochkin, Yuekai Sun, Assaf Arbelle, Leonid Karlinsky, and Raja Giryes. Livexiv – a multi-modal live benchmark based on arxiv papers content. In *International Conference on Learning Representations*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross J. Anderson, and Yarin Gal. Ai models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y.

Shivalika Singh, Yiyang Nan, Alex Wang, Daniel D'Souza, Sayash Kapoor, Ahmet Üstün, Sanmi Koyejo, Yuntian Deng, Shayne Longpre, Noah A. Smith, et al. The leaderboard illusion. In *Advances in Neural Information Processing Systems*, volume 38, pp. 86910–86964, 2025. doi: 10.52202/085713-2620.

Enxin Song, Wenhao Chai, Guanhong Wang, Yucheng Zhang, Haoyang Zhou, Feiyang Wu, Haozhe Chi, Xun Guo, Tian Ye, Yanting Zhang, Yan Lu, Jenq-Neng Hwang, and Gaoang Wang. Moviechat: From dense token to sparse memory for long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18221–18232, 2024. doi: 10.1109/CVPR52733.2024.01725.

Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.

Gabriela Ben Melech Stan, Estelle Aflalo, Avinash Madasu, Vasudev Lal, and Phillip Howard. Learning from reasoning failures via synthetic data generation. In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, 2026.

Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/f3c5e56274140e0420baa3916c529210-Abstract-Conference.html.

Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, Ben Kao, Guohao Li, Junxian He, Yu Qiao, and Zhiyong Wu. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5555–5579. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.277. URL https://aclanthology.org/2025.acl-long.277/.

Yifan Sun, Yushan Liang, Zhen Zhang, Xin Liu, and Jiaye Teng. Theoretical modeling of large language model self-improvement training dynamics through solver-verifier gap. In *International Conference on Learning Representations*, 2026.

Zhiqing Sun, Sheng Shen, Shengcao Cao, Haotian Liu, Chunyuan Li, Yikang Shen, Chuang Gan, Liangyan Gui, Yu-Xiong Wang, Yiming Yang, Kurt Keutzer, and Trevor Darrell. Aligning large multimodal models with factually augmented rlhf. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 13088–13110. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.775.

Ellen Xiaoqing Tan, Jack Lanchantin, Shehzaad Dhuliawala, Danwei Li, Thao Nguyen, Jing Xu, Ping Yu, Ilia Kulikov, Sainbayar Sukhbaatar, Jason Weston, Xian Li, and Olga Golovneva. Self-improving pretraining: using post-trained models to pretrain better models, 2026. URL https://arxiv.org/abs/2601.21343.

Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J. Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in llms. In *International Conference on Learning Representations*, 2026.

Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, Austin Wang, Rob Fergus, Yann LeCun, and Saining Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. In *Advances in Neural Information Processing Systems*, volume 37, 2024a. doi: 10.52202/079017-2771.

Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. Eyes wide shut? exploring the visual shortcomings of multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9568–9578, 2024b.

Yujun Tong, Dongliang Chang, Zijin Yin, Xintong Liu, Yuanchen Fang, and Zhanyu Ma. Reversing the flow: Generation-to-understanding synergy in large multimodal models. *arXiv preprint arXiv:2605.15792*, 2026.

Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yi-Kai Zhang, Zhikang Xia, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, Qi Gu, Hui Su, and Xunliang Cai. ScaleEnv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training. *arXiv preprint arXiv:2602.06820*, 2026. URL https://arxiv.org/abs/2602.06820.

Yassine Turki, Vinko Sabolčec, Bettina Messmer, and Martin Jaggi. Toward cross-lingual quality classifiers for multilingual pretraining data selection, 2026. URL https://arxiv.org/abs/2604.20549.

Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8228–8238, 2024. doi: 10.1109/CVPR52733.2024.00786.

Fei Wang, Wenxuan Zhou, James Y. Huang, Nan Xu, Sheng Zhang, Hoifung Poon, and Muhao Chen. mdpo: Conditional preference optimization for multimodal large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 8078–8088. Association for Computational Linguistics, 2024a.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024b.

Jiachen T Wang, Tong Wu, Dawn Song, Prateek Mittal, and Ruoxi Jia. Greats: Online selection of high-quality data for llm training in every iteration. *Advances in Neural Information Processing Systems*, 37: 131197–131223, 2024c.

Jiaxing Wang, Deping Xiang, Jin Xu, Zirui Liu, Zicheng Zhang, Guoqiang Gong, Jun Fang, Chao Liu, Pengzhang Liu, Tongxuan Liu, Ke Zhang, and Qixia Jiang. Blade: Scalable bi-level adaptive data selection for llm training, 2026a. URL `https://arxiv.org/abs/2606.18650`.

Jingke Wang, Zhenru Zhao, Shuangming Lei, Hao Su, Yuehao Huang, Yijia Xie, Kai Tang, Guanglin Xu, AiXue Ye, Yukai Ma, and Yong Liu. Drivestack-vla: Render-teacher alignment for bev-based deepstack vision-language-action model. *arXiv preprint arXiv:2606.24051*, 2026b.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025a. URL `https://arxiv.org/abs/2512.17102`.

Junxin Wang, Dai Guan, Weijie Qiu, Zhihang Li, Yongbo Gai, Zhengyi Yang, Mengyu Zhou, Erchao Zhao, et al. Grounding the score: Explicit visual premise verification for reliable vision-language process reward models. *arXiv preprint arXiv:2603.16253*, 2026c.

Shaobo Wang, Xuan Ouyang, Tianyi Xu, Yuzheng Hu, Jialin Liu, Guo Chen, Tianyu Zhang, Junhao Zheng, Kexin Yang, Xingzhang Ren, Dayiheng Liu, and Linfeng Zhang. Opus: Towards efficient and principled data selection in large language model pre-training in every iteration, 2026d. URL `https://arxiv.org/abs/2602.05400`.

Siyuan Wang, Zhuohan Long, Zhihao Fan, Zhongyu Wei, and Xuanjing Huang. Benchmark self-evolving: A multi-agent framework for dynamic llm evaluation. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, 2025b.

Sudong Wang, Weiquan Huang, Xiaomin Yu, Zuhao Yang, Hehai Lin, Keming Wu, Chaojun Xiao, Chen Chen, Wenxuan Wang, Beier Zhu, Yunjian Zhang, and Chengwei Qin. Beyond sft-to-rl: Pre-alignment via black-box on-policy distillation for multimodal rl. *arXiv preprint arXiv:2604.28123*, 2026e.

Weiyun Wang, Zhe Chen, Wenhai Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Jinguo Zhu, Xizhou Zhu, Lewei Lu, Yu Qiao, et al. Enhancing the reasoning ability of multimodal large language models via mixed preference optimization. *arXiv preprint arXiv:2411.10442*, 2024d.

Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, et al. Visualprm: An effective process reward model for multimodal reasoning. *arXiv preprint arXiv:2503.10291*, 2025c.

Xiaokun Wang, Peiyu Wang, Jiangbo Pei, Wei Shen, Yi Peng, and Yunzhuo Hao. Skywork-vl reward: An effective reward model for multimodal understanding and reasoning. *arXiv preprint arXiv:2505.07263*, 2025d.

Yibin Wang, Yuhang Zang, Hao Li, Cheng Jin, and Jiaqi Wang. Unified reward model for multimodal understanding and generation. *arXiv preprint arXiv:2503.05236*, 2025e.

Yifan Wang, Binbin Liu, Fengze Liu, Yuanfan Guo, Jiyao Deng, Xuecheng Wu, Weidong Zhou, Xiaohuan Zhou, and Taifeng Wang. Tikmix: Take data influence into dynamic mixture for language model pre-training, 2025f. URL `https://arxiv.org/abs/2508.17677`.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *Proceedings of ACL*, 2023.

Yudong Wang, Zixuan Fu, Jie Cai, Peijun Tang, Hongya Lyu, Yewei Fang, Zhi Zheng, Jie Zhou, Guoyang Zeng, Chaojun Xiao, et al. Ultra-fineweb: Efficient data filtering and verification for high-quality llm training data. *arXiv preprint arXiv:2505.05427*, 2025g.

Yuran Wang, Zekun Wang, Bohan Zeng, Ruixu Zhang, Wenxuan Liu, Liu Yang, Yifan Dai, Yang Shi, Bozhou Li, Chengzhuo Tong, Daili Hua, Yuanxing Zhang, and Wentao Zhang. Flux-OPD: On-policy distillation with evolving contexts. *arXiv preprint arXiv:2607.28022*, 2026f.

Zefan Wang, Lincheng Li, Tianyu Yu, and Yuan Yao. Drift: Refining instruction data via on-policy data attribution. *arXiv preprint arXiv:2606.18307*, 2026g.

Zehao Wang, Yihan Zeng, Zidong Gong, Yuanfan Guo, Feng Zhu, Hongzhi Zhang, Wei Zhang, and Wangmeng Zuo. Ane: Pushing the reasoning frontier of multimodal llms via anchor evolution, 2026h. URL `https://arxiv.org/abs/2605.25571`.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026i. URL `https://arxiv.org/abs/2602.10090`.

Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.

Alexander Wettig et al. Qurating: Selecting high-quality data for training language models. In *Proceedings of the International Conference on Machine Learning*, 2024.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh Agrawal, Sandeep Singh Sandha, Siddartha V. Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A challenging, contamination-limited LLM benchmark. In *International Conference on Learning Representations*, 2025.

Changti Wu, Jiahuai Mao, Yuzhuo Miao, Shijie Lian, Bin Yu, Xiaopeng Lin, Cong Huang, Lei Zhang, and Kai Chen. Scalselect: Scalable training-free multimodal data selection for efficient visual instruction tuning, 2026. URL `https://arxiv.org/abs/2602.11636`.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*, 2025a.

Ting Wu, Xuefeng Li, and Pengfei Liu. Progress or regress? self-improvement reversal in post-training. In *International Conference on Learning Representations*, 2025b.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023.

Xindi Wu, Mengzhou Xia, Rulin Shao, Zhiwei Deng, Pang Wei Koh, and Olga Russakovsky. Icons: Influence consensus for vision-language data selection, 2024. URL `https://arxiv.org/abs/2501.00654`.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. Less: Selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 54104–54132. PMLR, 2024. URL `https://proceedings.mlr.press/v235/xia24c.html`.

Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025.

Sang Michael Xie et al. Doremi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems*, 2023.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, 2024. Datasets and Benchmarks Track.

Xingrun Xing, Haoqing Wang, Boyan Gao, Ziheng Li, and Yehui Tang. Trust region on-policy distillation. *arXiv preprint arXiv:2606.01249*, 2026.

Tianyi Xiong, Xiyao Wang, Dong Guo, Qinghao Ye, Haoqi Fan, Quanquan Gu, Heng Huang, and Chunyuan Li. Llava-critic: Learning to evaluate multimodal models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13618–13628, 2025.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 623–645, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.27. URL `https://aclanthology.org/2026.acl-long.27/`.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. WizardLM: Empowering large pre-trained language models to follow complex instructions. In *International Conference on Learning Representations*, 2024a.

Hu Xu, Saining Xie, Xiaoqing Ellen Tan, Po-Yao Huang, Russell Howes, Vasu Sharma, Shang-Wen Li, Gargi Ghosh, Luke Zettlemoyer, and Christoph Feichtenhofer. Demystifying clip data. In *International Conference on Learning Representations*, 2024b.

Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. Imagereward: Learning and evaluating human preferences for text-to-image generation. *Advances in Neural Information Processing Systems*, 36:15903–15935, 2023.

Jiazheng Xu, Yu Huang, Jiale Cheng, Yuanming Yang, Jiajun Xu, Yuan Wang, Wenbo Duan, Shen Yang, Qunlin Jin, Shurun Li, Jiayan Teng, Zhuoyi Yang, Wendi Zheng, Xiao Liu, Dan Zhang, Ming Ding, Xiaohan Zhang, Shiyu Huang, Xiaotao Gu, Minlie Huang, Jie Tang, and Yuxiao Dong. Visionreward: Fine-grained multi-dimensional human preference learning for image and video generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(13):11269–11277, 2026a. doi: 10.1609/aaai.v40i13.38107.

Mingde Xu, Zhen Yang, Yan Wang, Yu Wang, Xijun Liu, Zijun Dou, Wenyi Hong, Xiaotao Gu, Bin Xu, and Jie Tang. Video2Code: Generating interactive webpages from UI videos via action-aware revisit. *arXiv preprint arXiv:2606.20711*, 2026b. URL `https://arxiv.org/abs/2606.20711`.

Ruijie Xu, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. Benchmarking benchmark leakage in large language models. *arXiv preprint arXiv:2404.18824*, 2024c. URL `https://arxiv.org/abs/2404.18824`.

Wenda Xu, Guanglei Zhu, Xuandong Zhao, Liangming Pan, Lei Li, and William Wang. Pride and prejudice: LLM amplifies self-bias in self-refinement. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15474–15492, 2024d. doi: 10.18653/v1/2024.acl-long.826. URL `https://aclanthology.org/2024.acl-long.826/`.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Abel Yagubyan. How consistent are LLM agents? measuring behavioral reproducibility in multi-step tool-calling pipelines. *arXiv preprint arXiv:2605.28840*, 2026. URL https://arxiv.org/abs/2605.28840.

Dawei Yan, Haokui Zhang, Guangda Huzhang, Yang Li, Yibo Wang, Qing-Guo Chen, Zhao Xu, Weihua Luo, Ying Li, Wei Dong, and Chunhua Shen. M^2: Dual-memory augmentation for long-horizon web agents via trajectory summarization and insight retrieval. *arXiv preprint arXiv:2603.00503*, 2026a.

Siyu Yan, Yizhen Gao, Yilin Wang, Dongxing Mao, and Alex Jinpeng Wang. Dataevolver: Self-evolving multi-agent data construction for text-rich image generation, 2026b. URL https://arxiv.org/abs/2606.31537.

Junlin Yang, Che Jiang, Yu Fu, Tianwei Luo, Can Ren, Weizhi Wang, Kaikai Zhao, Hongyi Liu, Yuxin Zuo, Yuru Wang, Yuchen Fan, Kai Tian, Zhenzhao Yuan, Xiaojian Lin, Li Sheng, Rushi Qiang, Guoli Jia, Xingtai Lv, Ermo Hua, Dianqiao Lei, Youbang Sun, Ning Ding, Bowen Zhou, and Kaiyan Zhang. FrontisMA1: Training an AI4AI model towards recursive self-improvement in machine learning engineering. *arXiv preprint arXiv:2607.28568*, 2026a.

Kailai Yang, Xiao Liu, Lei Ji, Hao Li, Xiao Liang, Zhiwei Liu, Yeyun Gong, Peng Cheng, and Mao Yang. Data mixing agent: Learning to re-weight domains for continual pre-training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9447–9473. Association for Computational Linguistics, 2026b. doi: 10.18653/v1/2026.acl-long.427. URL https://aclanthology.org/2026.acl-long.427/.

Ling Yang, Xinchen Zhang, Ye Tian, Shiyi Zhang, Chenming Shang, Minghao Xu, Wentao Zhang, and Bin Cui. Hermesflow: Seamlessly closing the gap in multimodal understanding and generation. In *Advances in Neural Information Processing Systems*, volume 38, 2025a.

Shu Yang, Junchao Wu, Derek F. Wong, and Di Wang. Selfmem: Self-optimizing memory for AI agents. *arXiv preprint arXiv:2607.03726*, 2026c.

Tianze Yang, Yucheng Shi, Ruitong Sun, Jingyuan Huang, Ninghao Liu, and Jin Sun. TRON: Targeted rule-verifiable online environments for visual reasoning RL. *arXiv preprint arXiv:2606.01599*, 2026d. URL https://arxiv.org/abs/2606.01599.

Xu Yang, Chen Liu, and Ying Wei. Data selection matters: Towards robust instruction tuning of large multimodal models. In *Advances in Neural Information Processing Systems*, 2025b. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/0d77ccb50a558035f19089096f933e8e-Abstract-Conference.html.

Ziao Yang, Longbo Huang, and Hongfu Liu. Layer-aware influence for online data valuation estimation, 2025c. URL https://arxiv.org/abs/2510.16007.

Barry Menglong Yao, Qifan Wang, and Lifu Huang. Error-driven data-efficient large multimodal model tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. URL https://aclanthology.org/2025.acl-long.992/.

Michihiro Yasunaga, Luke Zettlemoyer, and Marjan Ghazvininejad. Multimodal rewardbench: Holistic evaluation of reward models for vision language models. *arXiv preprint arXiv:2502.14191*, 2025.

Jiasheng Ye, Peiju Liu, Tianxiang Sun, Jun Zhan, Yunhua Zhou, and Xipeng Qiu. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. In *International Conference on Learning Representations*, 2025a.

Jiayuan Ye, Vitaly Feldman, and Kunal Talwar. Cram less to fit more: Training data pruning improves memorization of facts, 2026a. URL https://arxiv.org/abs/2604.08519.

Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026b.

Ziyu Ye, Rishabh Agarwal, Tianqi Liu, Rishabh Joshi, Sarmishta Velury, Quoc V. Le, Qijun Tan, and Yuan Liu. Reward-guided prompt evolving in reinforcement learning for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 71910–71937, 2025b. URL https://proceedings.mlr.press/v267/ye25a.html.

Chengzhi Yu, Yifan Xu, Yifan Chen, and Wenyi Zhang. Optimizing lvlms with on-policy data for effective hallucination mitigation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale. In *Advances in Neural Information Processing Systems*, volume 38, 2025a. doi: 10.52202/085713-3775.

Shi Yu, Chaoyue Tang, Bokai Xu, Junbo Cui, Junhao Ran, Yukun Yan, Zhenghao Liu, Shuo Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. Visrag: Vision-based retrieval-augmented generation on multi-modality documents. In *International Conference on Learning Representations*, 2025b.

Tianyu Yu, Yuan Yao, Haoye Zhang, Taiwen He, Yifeng Han, Ganqu Cui, Jinyi Hu, Zhiyuan Liu, Hai-Tao Zheng, Maosong Sun, et al. Rlhf-v: Towards trustworthy mllms via behavior alignment from fine-grained correctional human feedback. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13807–13816, 2024a.

Tianyu Yu, Haoye Zhang, Qiming Li, Qixin Xu, Yuan Yao, Da Chen, Xiaoman Lu, Ganqu Cui, Yunkai Dang, Taiwen He, Xiaocheng Feng, Jun Song, Bo Zheng, Zhiyuan Liu, Tat-Seng Chua, and Maosong Sun. Rlaif-v: Open-source ai feedback leads to super gpt-4v trustworthiness. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19985–19995, 2025c. doi: 10.1109/CVPR52734.2025.01861. URL https://openaccess.thecvf.com/content/CVPR2025/html/Yu_RLAIF-V_Open-Source_AI_Feedback_Leads_to_Super_GPT-4V_Trustworthiness_CVPR_2025_paper.html.

Zichun Yu and Chenyan Xiong. Repro: Training language models to faithfully recycle the web for pretraining, 2025. URL https://arxiv.org/abs/2510.10681.

Zichun Yu and Chenyan Xiong. Generating pretraining tokens from organic data for data-bound scaling, 2026. URL https://arxiv.org/abs/2605.17849.

Zichun Yu, Spandan Das, and Chenyan Xiong. Mates: Model-aware data selection for efficient pretraining with data influence models. In *Advances in Neural Information Processing Systems*, volume 37, 2024b.

Zichun Yu, Fei Peng, Jie Lei, Arnold Overwijk, Scott Yih, and Chenyan Xiong. Group-level data selection for efficient pretraining. In *Advances in Neural Information Processing Systems*, volume 38, 2025d.

Qianhao Yuan, Jie Lou, Xing Yu, Hongyu Lin, Le Sun, Xianpei Han, and Yaojie Lu. Vision-OPD: Learning to see fine details for multimodal LLMs via on-policy self-distillation. *arXiv preprint arXiv:2605.18740*, 2026.

Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. Agent-R: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*, 2025a.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 57905–57923, 2024a.

Yuqian Yuan, Wentong Li, Jian Liu, Dongqi Tang, Xinjie Luo, Chi Qin, Lei Zhang, and Jianke Zhu. Osprey: Pixel understanding with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 28202–28211, 2024b. doi: 10.1109/CVPR52733.2024.02664.

Yuqian Yuan, Hang Zhang, Wentong Li, Zesen Cheng, Boqiang Zhang, Long Li, Xin Li, Deli Zhao, Wenqiao Zhang, Yueting Zhuang, Jianke Zhu, and Lidong Bing. Videorefer suite: Advancing spatial-temporal object understanding with video llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18970–18980, 2025b.

Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhu Chen, and Graham Neubig. Mmmu-pro: A more robust multi-discipline multimodal understanding benchmark. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15134–15186. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.736. URL `https://aclanthology.org/2025.acl-long.736/`.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Self-taught reasoner bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

Aohan Zeng, Zhengxiao Du, Mingdao Liu, Lei Zhang, Shengmin Jiang, Yuxiao Dong, and Jie Tang. Scaling speech-text pre-training with synthetic interleaved data. In *International Conference on Learning Representations*, 2025.

Hector Zenil. On the limits of self-improving in large language models: The singularity is not near without symbolic model synthesis. *arXiv preprint arXiv:2601.05280*, 2026.

Chi Zhang, Huaping Zhong, Kuan Zhang, Chengliang Chai, Rui Wang, Xinlin Zhuang, Tianyi Bai, Jiantao Qiu, Lei Cao, Ju Fan, Ye Yuan, Guoren Wang, and Conghui He. Harnessing diversity for important data selection in pretraining large language models. In *International Conference on Learning Representations*, 2025a.

Jenny Zhang, Shengran Hu, Cong Lu, Robert T. Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, 2026.

Jieyu Zhang, Weikai Huang, Zixian Ma, Oscar Michel, Dong He, Tanmay Gupta, Wei-Chiu Ma, Ali Farhadi, Aniruddha Kembhavi, and Ranjay Krishna. Task me anything. In *Advances in Neural Information Processing Systems*, volume 37, 2024a. Datasets and Benchmarks Track.

Jihai Zhang, Tianle Li, Linjie Li, Zhengyuan Yang, and Yu Cheng. Are unified vision-language models necessary: Generalization across understanding and generation. *arXiv preprint arXiv:2505.23043*, 2025b.

Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. LMMs-Eval: Reality check on the evaluation of large multimodal models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 881–916, 2025c. doi: 10.18653/v1/2025.findings-naacl.51. URL `https://aclanthology.org/2025.findings-naacl.51/`.

Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Yu Qiao, Peng Gao, and Hongsheng Li. MATHVERSE: Does your multi-modal LLM truly see the diagrams in visual math problems? In *Computer Vision – ECCV 2024*, volume 8, pp. 169–186. Springer, 2024b. doi: 10.1007/978-3-031-73242-3_10.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*. PMLR, 2025d.

Yanzhe Zhang, Ruiyi Zhang, Jiuxiang Gu, Yufan Zhou, Nedim Lipka, Diyi Yang, and Tong Sun. Llavar: Enhanced visual instruction tuning for text-rich image understanding, 2023. URL https://arxiv.org/abs/2306.17107.

Zhihong Zhang, Xiaojian Huang, Jin Xu, Zhuodong Luo, Xinzhi Wang, Jiansheng Wei, and Xuejin Chen. Videorewardbench: Comprehensive evaluation of multimodal reward models for video understanding, 2025e. URL https://arxiv.org/abs/2509.00484.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17): 19632–19642, 2024. doi: 10.1609/aaai.v38i17.29936.

Enhan Zhao, Wei Wu, Yuanrui Zhang, Xueliang Zhao, and Di He. Ouroboros-spatial: Closing the data-model loop for spatial reasoning, 2026a. URL https://arxiv.org/abs/2606.11719.

Kaiyan Zhao, Zhongtao Miao, Akiko Aizawa, and Yoshimasa Tsuruoka. Regmix-d: Dynamic data mixing via proxy training trajectories, 2026b. URL https://arxiv.org/abs/2606.18663.

Tong Zhao, Yuyang Hu, Reed Li, Yu Lu, Haibo Shi, Yutao Zhu, and Zhicheng Dou. OPOD: On-policy omni distillation. *arXiv preprint arXiv:2607.20918*, 2026c.

Yiran Zhao, Lu Zhou, Xiaogang Xu, Zhe Liu, Jiafei Wu, and Liming Fang. Fair in mind, fair in action? a synchronous benchmark for understanding and generation in umllms. *arXiv preprint arXiv:2603.00590*, 2026d.

Zhiyuan Zhao, Bin Wang, Linke Ouyang, Xiaoyi Dong, Jiaqi Wang, and Conghui He. Beyond hallucinations: Enhancing lvlms through hallucination-aware direct preference optimization. *arXiv preprint arXiv:2311.16839*, 2023.

Dian Zheng, Ziqi Huang, Hongbo Liu, Kai Zou, Yinan He, Fan Zhang, Lulu Gu, Yuanhan Zhang, Jingwen He, Wei-Shi Zheng, et al. Vbench-2.0: Advancing video generation benchmark suite for intrinsic faithfulness. *arXiv preprint arXiv:2503.21755*, 2025.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 2023.

Fan Zhou, Zengzhi Wang, Qian Liu, Junlong Li, and Pengfei Liu. Programming every example: Lifting pre-training data quality like experts at scale. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 79281–79327. PMLR, 2025. URL https://proceedings.mlr.press/v267/zhou25aa.html.

Guanyu Zhou, Yida Yin, Wenhao Chai, Shengbang Tong, Xingyu Fu, and Zhuang Liu. Visionfoundry: Teaching vlms visual perception with synthetic images, 2026a. URL https://arxiv.org/abs/2604.09531.

Jiang Zhou, Yunhao Wang, Xing Wu, Tinghao Yu, and Feng Zhang. Wrap++: Web discovery amplified pretraining, 2026b. URL https://arxiv.org/abs/2604.06829.

Yiyang Zhou, Chenhang Cui, Rafael Rafailov, Chelsea Finn, and Huaxiu Yao. Aligning modalities in vision large language models via preference fine-tuning. *arXiv preprint arXiv:2402.11411*, 2024a.

Yiyang Zhou, Zhiyuan Fan, Dongjie Cheng, Sihan Yang, Zhaorun Chen, Chenhang Cui, Xiyao Wang, Yun Li, Linjun Zhang, and Huaxiu Yao. Calibrated self-rewarding vision language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024b.

Yuhang Zhou, Lizhu Zhang, Yifan Wu, Mingyi Wang, Bo Peng, Jiayi Liu, Xiangjun Fan, and Zhuokai Zhao. Omniopd: Logit-free on-policy distillation via speculative verification. *arXiv preprint arXiv:2606.01476*, 2026c.

Guanghao Zhu, Zeyu Liu, Zhitian Hou, Pengkai Wang, Zhijie Sang, Yang Yu, Minheng Ni, Wenjun Wang, Yanggan Gu, Shuo Cai, Congkai Xie, Jianmin Wu, and Hongxia Yang. Beyond captions: Context-grounded reconstruction for biomedical multimodal continued pretraining, 2026a. URL https://arxiv.org/abs/2606.01049.

Kaijie Zhu, Jiaao Chen, Jindong Wang, Neil Zhenqiang Gong, Diyi Yang, and Xing Xie. Dyval: Dynamic evaluation of large language models for reasoning tasks. In *International Conference on Learning Representations*, 2024.

Kejian Zhu, Zhuoran Jin, Dongqi Huang, Hongbang Yuan, Yupu Hao, Kang Liu, and Jun Zhao. Beyond simply environment scaling: Designing effective environment distributions for multimodal agent learning. *arXiv preprint arXiv:2608.03571*, 2026b. URL https://arxiv.org/abs/2608.03571.

Tinghui Zhu, Sheng Zhang, James Y. Huang, Selena Song, Xiaofei Wen, Yuankai Li, Hoifung Poon, and Muhao Chen. Video models can reason with verifiable rewards. *arXiv preprint arXiv:2605.15458*, 2026c. URL https://arxiv.org/abs/2605.15458.

Xinlin Zhuang, Jiahui Peng, Ren Ma, Yinfan Wang, Tianyi Bai, Xingjian Wei, Jiantao Qiu, Chi Zhang, Ying Qian, and Conghui He. Meta-rater: A multi-dimensional data selection method for pre-training language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10856–10896, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.533. URL https://aclanthology.org/2025.acl-long.533/.

Chengke Zou, Xingang Guo, Rui Yang, Junyu Zhang, Bin Hu, and Huan Zhang. Dynamath: A dynamic visual benchmark for evaluating mathematical reasoning robustness of vision language models. In *International Conference on Learning Representations*, 2025.